
CBMPy Documentation

Release 0.8.9

Brett G. Olivier

Apr 27, 2026

CONTENTS:

1	Quick Start	3
2	Contents	5
2.1	Installation Guide	5
2.2	Flux Module Analysis	7
2.3	Matroid Module	7
2.4	Decomposition Module	11
2.5	Sparse Rationals Module	15
2.6	CBMPy Core Classes	16
2.7	Solver Implementations	115
2.8	CBMPy: CBRead module	122
2.9	CBMPy: CBReadtxt module	125
2.10	CBMPy: CBWrite module	126
2.11	CBMPy: CBXML module	138
2.12	CBMPy: CBNetDB module	155
2.13	Visualization	161
2.14	GUI Frontends	162
2.15	Multi-processing	163
2.16	Model Utilities	165
2.17	Release Notes	168
2.18	0.8.9 (unreleased)	168
2.19	0.8.8 (2023)	169
2.20	0.8.4 (2022)	169
2.21	0.8.2 (2021)	169
3	Indices and tables	171
	Python Module Index	173
	Index	175

CBMPy is a platform for constraint-based modelling and analysis of biological systems. It implements Flux Balance Analysis (FBA), Flux Variability Analysis (FVA), and other constraint-based methods using linear and mixed-integer programming.

QUICK START

After installing CBMPy:

```
import cbmpy

# Read a model from SBML
cmod = cbmpy.readSBML3FBC('example_model.xml')

# Perform Flux Balance Analysis
result = cbmpy.doFBA(cmod)

# Print the optimal objective value
print(f"Optimal flux: {result[0]}")
```


CONTENTS

2.1 Installation Guide

2.1.1 Installing CBMPy

CBMPy supports Python 3.8 through 3.12. You can install it using conda or pip.

2.1.2 Prerequisites

System Requirements:

- Python 3.8 - 3.12
- One of the following solvers: - IBM CPLEX (commercial) - GLPK (open source, via *swiglpk*)
- numpy $\geq$ 1.17.0
- python-libsbml $\geq$ 5.18.0 (for SBML support)
- scipy (for numerical operations)

2.1.3 Using Conda (Recommended)

1. Create a conda environment:

```
conda create -n cbmpy3 python=3.10
```

2. Activate the environment:

```
conda activate cbmpy3
```

3. Install CBMPy:

```
conda install cbmpy
```

Alternatively, create an environment with all dependencies:

```
conda env create -f environment.yml
```

2.1.4 Using pip

1. Create a virtual environment:

```
python -m venv cbmpy3
source cbmpy3/bin/activate # On Linux/Mac
cbmpy3\Scripts\activate.bat # On Windows
```

2. Install CBMPy:

```
pip install -e ".[dev]"
```

Or from the requirements file:

```
pip install -r requirements.txt
```

2.1.5 Verification

Verify the installation by running:

```
import cbmpy
cmmod = cbmpy.readSBML3FBC('cbmpy_test_core')
cbmpy.doFBA(cmmod)
```

2.1.6 Troubleshooting

Error: “No linear solver present”

You need to install either CPLEX or GLPK. On Linux:

```
# Install GLPK via swiglpk
pip install swiglpk
```

Error: “python-libsmbml not found”

Install python-libsmbml:

```
pip install python-libsmbml
```

On Windows with CPLEX:

1. Download the CPLEX Python package from IBM
2. Add the CPLEX directory to your PYTHONPATH

3. Install CBMPy

2.1.7 Update CBMPy

After installation, you can update CBMPy using:

```
# Conda
conda update cbmpy

# pip
pip install --upgrade cbmpy
```

Or update in-place:

```
pip install -e "[dev]"
```

2.1.8 Dependencies

Core dependencies: - numpy >= 1.17.0 - packaging - pyparsing - python-libsmbml >= 5.18.0 - lxml - xlrd - xlwt - swiglpk - scipy - XlsxWriter

Optional dependencies: - sympy >= 1.7.1 (for symbolic matrix support)

Development dependencies: - pytest >= 7.0 - pytest-cov - docformatter >= 1.7.7 - sphinx >= 7.0 - sphinx_rtd_theme - myst-parser - tox

2.2 Flux Module Analysis

2.3 Matroid Module

2.3.1 CBMPy: fluxmodules matroid module

PySCeS Constraint Based Modelling (<http://cbmpy.sourceforge.net>)

This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program. If not, see <<http://www.gnu.org/licenses/>>

Author: Arne C. Reimers Contact email: arne.c.reimers@gmail.com

class cbmpy.fluxmodules.matroid.**Matroid**(*rep, basis, nbasis*)

Bases: **object**

A linear matroid.

It has the fields: *elems* - lists the names of the elements *basis* - lists the names of the basis elements *nbasis* - lists the names of the non-basis elements (dual basis) *rep* - reduced representation matrix,

rows correspond to basis-elements and columns correspond to non-basis elements

__init__(*rep, basis, nbasis*)

Create matroid from given reduced representation matrix.

The names of the rows are given by *basis*. The names of the columns are given by *nbasis*.

findModules()

Computes the separators (modules) of this matroid.

Each module is returned as a matroid.

dual()

Returns the dual matroid.

By this operation the original coefficient matrix is lost for the dual.

copy()

Copies this matroid.

contract(*toContract*)

Computes self / *toContract*.

The matroid with *toContract* contracted is returned. This instance is not modified.

exchange(*a, b*)

Exchanges a basis element with a non-basis element.

requirement is that the fundamental circuit of the non-basis element uses the basis element

delete(*toDelete*)

Computes self *toDelete*.

The matroid with *toDelete* deleted is returned. No elements of this instance are deleted. However, the current basis can change.

parallel()

List all groups of parallel elements.

The result is a set of frozen sets, each set listing the ids of the parallel elements

coparallel()

List all groups of coparallel elements.

In metabolic network terminology the coparallel elements are the fully coupled reactions.

The result is a set of frozensets, each set listing the ids of the coparallel elements

isParallel()**isCoparallel()****fundamentalCircuit(*e*)**

Returns the fundamental circuit of an element.

If *e* is a non-basis element, the fundamental circuit, i.e. the support of the column of *e*, is returned.

If *e* is a basis element, the cofundamental circuit, i.e. the support of the row of *e*, is returned.

The returned fundamental circuit does not contain *e* itself

fundamentalCircuitValues(*e*)

Returns the fundamental circuit of an element with the coefficients.

If *e* is a non-basis element, the fundamental circuit, i.e. the support of the column of *e*, is returned.

If *e* is a basis element, the cofundamental circuit, i.e. the support of the row of *e*, is returned.

The returned fundamental circuit does not contain *e* itself. If needed, it should be added manually afterwards with a coefficient of 1.

rank(*elems*)

Computes the rank of *elems*.

This method puts as many elements of *elems* into the basis. The number of elements from *elems* in the final basis is returned.

corank(*elems*)

Computes the corank of *elems*.

This method puts as many elements of *elems* into the non-basis. The number of elements from *elems* in the final non-basis is returned.

isIndependent(*elems*)

Checks if the given set of elements is independent.

This does not modify this matroid, but may change its internal basis. If this method returns true, the current basis contains all elements of *elems*

isCoIndependent(*elems*)

Checks if the given set of elements is coindependent.

This does not modify this matroid, but may change its internal basis. If this method returns true, the current basis contains all no elements of elems

This is the same as isIndependent, only on the dual

connectivity(*separation*)

Evaluate the connectivity function for the given separation.

separation is a set of elements encoding one side of the separation. The other side is implied.

getInterface(*separation*)

Compute a list of circuits that define the interface of the given separation.

similarityMatrix(*stype*)

Computes a similarity matrix on the elements.

For each pair of elements the similarity matrix has an entry that gives the similarity.

stype gives what kind of similarity matrix should be computed. Possibilities are: direct
- direct from the columns of the original coefficient matrix

(needs the original coefficient matrix and in the case of contractions the result is not well defined)

__cols - normalized from the columns (rows are orthogonal) rows - normalized from the rows (columns are orthogonal) combined - takes max(__cols, rows)

cbmpy.fluxmodules.matroid.fromMatrix(*matrix, cols*)

Builds the reduced representation matrix.

Given a matrix of columns (self.matrix) it builds the reduced representation matrix (self.rep) that has rows for each basis element and columns for each non-basis element.

From this form the fundamental cycles can easily be read of and other operations like deletion and contraction are easy.

class cbmpy.fluxmodules.matroid.**SimilarityMatrix**(*matrix, elems*)

Bases: `object`

__init__(*matrix, elems*)

Builds a similarity matrix out of a matrix and element names.

the matrix must be square and have as many rows/columns as elems has entries

max(*simmatrix*)

2.4 Decomposition Module

2.4.1 CBMPy: fluxmodules decompositon module

PySCeS Constraint Based Modelling (<http://cbmpy.sourceforge.net>)

This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program. If not, see <<http://www.gnu.org/licenses/>>

Author: Arne C. Reimers Contact email: arne.c.reimers@gmail.com

class cbmpy.fluxmodules.decomposition.**Edge**(*a, b*)

Bases: `object`

Every edge connects two vertices.

__init__(*a, b*)

Initialize edge connecting vertices *a* and *b*.

getOther(*vertex*)

Give the other vertex then the specified one.

The specified vertex must be one of this edges vertices

class cbmpy.fluxmodules.decomposition.**AbstractVertex**

Bases: `object`

A vertex in a graph.

It can be a leaf or an inner node

maxId = 0

For giving unique ids for hashing.

__init__()

id()

label()

replaceAdjacent(*old, new*)

Replace the adjacent node *old* by *new*.

getAdjacent()

List adjacent nodes.

getLeaves(*fromVertex*)

Enumerates all the leaves of this vertex that are reachable without using *fromVertex*.

fromVertex must be a vertex adjacent to this vertex

class cbmpy.fluxmodules.decomposition.**Vertex**

Bases: [*AbstractVertex*](#)

Represents an inner vertex (i.e., not a leaf)

Every Vertex contains a map (edges) of adjacent edges resp. vertices. Each Edge has a list of elements associated to it (the interface). Each Vertex stores a matroid, whose elements are contributed by the edges.

The map edges has the following form: The keys are the incident vertices. The values are the list of elements contributed by the edge

__init__()

label()

setUpRoot(*matroid*)

Set ups this vertex as the root node for the given matroid.

This means we add leaves for all the elements of the matroid to this vertex

splitOff(*adjacent*)

Splits of a list of adjacent vertices.

adjacent must be a list of vertices that are adjacent to this node. The list *adjacent* must contain more than one element

replaceAdjacent(*old*, *new*)

Replace the adjacent node *old* by *new*.

getAdjacent()

List adjacent nodes.

getLeaves(*fromVertex*)

Enumerates all the leaves of this vertex that are reachable without using *fromVertex*.

If *fromVertex* is not a vertex incident to this vertex, then all leaves are enumerated.

isSimple()

class cbmpy.fluxmodules.decomposition.**Leaf**(*element*, *adjacent*)

Bases: [*AbstractVertex*](#)

__init__(*element, adjacent*)

Creates leaf representing an matroid element.

element is the matroid element to represent *adjacent* is the vertex to which this leaf is added

label()

replaceAdjacent(*old, new*)

Replace the adjacent node *old* by *new*.

getAdjacent()

List adjacent nodes.

getLeaves(*fromVertex*)

Enumerates all the leaves of this vertex that are reachable without using *fromVertex*.

fromVertex must be a vertex adjacent to this vertex

class cbmpy.fluxmodules.decomposition.**Decomposition**(*matroid*)

Bases: `object`

Hierarchical branch decomposition.

Actually can also represent a partial branch decomposition as intermediate results.

It has the following fields: *matroid* - the matroid of this branch decomposition *root* - a vertex of the graph representing the branch decomposition

TODO: more details

__init__(*matroid*)

Initializes as star branch decomposition.

listNonLeaves()

List all non-leaf vertices.

listNonLeavesRec(*vertex, source*)

getVertex(*vertexid*)

Returns the vertex with the given id.

If no vertex with the given id exists, `None` is returned.

splitZeroModules(*vertex*)

Branch given vertex into 0 modules.

Requires that all adjacent vertices have at most 1-dimensional interface

splitSimple()

Split all vertices into 0-modules and 1-modules.

splitOneModules(*vertex*)

Branch given vertex into 1 modules.

TODO:we have to deal with grouped elements

makeSubcubic()

Split all nodes that have degree ≥ 4 in an arbitrary manner.

isParallelSeries()

Checks if all nodes have only parallel or only coparallel elems.

This does not check if the matroid is parallel-series.

verifyFullyBranched()

Verifies that this branch decomposition is fully branched.

The branch decomposition is considered fully branched if all vertices satisfy: - is degree 3 or - all elements are parallel - all elements are coparallel

If successful, True is returned. Otherwise, a `NotFullyBranchedException` is raised

verifyEdgeWidth()

Verifies if the edge with really coincides with the interfaces.

In case of success, true is returned. Otherwise an `EdgeWidthException` is raised.

getWidth()

Computes the width of this branch decomposition.

poolmanMethod(*vertex*, *similarity=None*)

Decomposes the matroid using the method suggested by Poolman et al.

This method iteratively groups these elements together that are most similar. Similarity by sets of nodes is done via the mean. For more details see doi:10.1016/j.jtbi.2007.08.005

This method takes a similarity matrix as input, which allows it to also apply the method to other similarity matrices then those suggested in the original work. If no similarity matrix is given, then a row similarity matrix is used, which corresponds to the matrix suggested by Poolman et al.

makeGraphViz(*cmo*=None)

Create graphviz string to draw this decomposition.

write(*filename*)

Writes this decomposition to file.

exception `cbmpy.fluxmodules.decomposition.NotFullyBranchedException`(*vertex*)

Bases: `Exception`

Thrown if the branch decomposition is not fully branched.

```
__init__(vertex)
```

```
exception cbmpy.fluxmodules.decomposition.EdgeWidthException(vertex1, vertex2,  
                                                                conn)
```

Bases: `Exception`

```
__init__(vertex1, vertex2, conn)
```

2.5 Sparse Rationals Module

2.5.1 CBMPy: fluxmodules sparse rationals module

PySCeS Constraint Based Modelling (<http://cbmpy.sourceforge.net>)

This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program. If not, see <<http://www.gnu.org/licenses/>>

Author: Arne C. Reimers Contact email: arne.c.reimers@gmail.com

```
class cbmpy.fluxmodules.sparserationals.Matrix(listRep=None)
```

Bases: `object`

Sparse matrix of rational numbers (fractions)

The matrix grows by demand.

```
__init__(listRep=None)
```

“Creates empty matrix.

```
rows()
```

```
cols()
```

```
getCol(idx)
```

Returns the column at index `idx` as a frozen set of key-value pairs.

The returned dictionary does not contain any superfluous zeros.

```
transpose(scalar=1)
```

Creates a transpose of the matrix.

Optionally, the values can be multiplied by a scalar. This is for example used to compute the dual matroid.

copy()

Copies this matrix.

reset(*rows*, *cols*)

Reset all elements in rows x __cols to zero.

This is equivalent (but a bit faster) to `self[rows, __cols] = 0`

coladd(*col*, *v*, *scalar=1*)

Computes `self[:,col] += scalar * v`.

col can only be a single index

addMetabolicNetwork(*cmod*)

Adds the stoichiometric matrix of a metabolic network.

This is the default method to load the stoichiometric matrix into this matrix. Just create an empty matrix and call this function.

A list of column names is returned.

addListRep(*matrix*)

Adds matrix in list representation (ordered by rows)

toNumpy()

Returns floating point numpy representation.

The numpy matrix is a full matrix!

checkRows()

isZero()

`cbmpy.fluxmodules.sparserationals.revslice(x, sl)`

Note

The enumerate module is not documented here due to a missing dependency (`cbmpy.solver`). See the source code in `cbmpy/fluxmodules/enumerate.py`.

2.6 CBMPy Core Classes

The core module documents all main `cbmpy` modules.

2.6.1 CBMPy: `__init__.py`

PySCeS Constraint Based Modelling (<http://cbmpy.sourceforge.net>) Copyright (C) 2009-2024 Brett G. Olivier, VU University Amsterdam, Amsterdam, The Netherlands

This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program. If not, see <<http://www.gnu.org/licenses/>>

Author: Brett G. Olivier PhD Contact developers: <https://github.com/SystemsBioinformatics/cbmpy/issues> Last edit: \$Author: bgoli \$ (\$Id: `__init__.py` 710 2020-04-27 14:22:34Z bgoli \$)

```
cbmpy.MinimizeSumOfAbsFluxes(*args, **kwargs)
```

```
cbmpy.doFBAMinSum(*args, **kwargs)
```

```
cbmpy.test()
```

2.6.2 CBMPy: CBModel module

PySCeS Constraint Based Modelling (<http://cbmpy.sourceforge.net>) Copyright (C) 2009-2024 Brett G. Olivier, VU University Amsterdam, Amsterdam, The Netherlands

This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program. If not, see <<http://www.gnu.org/licenses/>>

Author: Brett G. Olivier PhD Contact developers: <https://github.com/SystemsBioinformatics/cbmpy/issues> Last edit: \$Author: bgoli \$ (\$Id: `CBModel.py` 706 2020-03-23 21:31:49Z bgoli \$)

```
class cbmpy.CBModel.Fbase
```

```
    Bases: object
```

```
    Base class for CB Model objects.
```

id = None

name = None

annotation = None

annotation_ext = None

compartment = None

miriam = None

notes = ''

getModel()

Get the parent model object linked to in objref, can return model or None for unlinked object.

Overwritten by Model

getPid()

Return the object ID.

getId()

Return the object ID.

getMetaId()

Return the object metaId.

getName()

Return the object name.

getNotes()

Return the object's notes.

getAnnotations()

Return the object annotation dictionary.

getAnnotation(*key*)

Return the object annotation associated with:

Parameters

key – the annotation key

hasAnnotation(*key*)

Returns a boolean representing the presence/absence of the key in the object annotation.

Parameters

key – the annotation key

setName(*name*)

Set the object name:

Parameters

name – the name string

setNotes(*notes*)

Sets the object's notes:

Parameters

notes – the note string, should preferably be (X)HTML for SBML

setAnnotation(*key*, *value*, *ext=None*)

Set an objects annotation as a key : value pair.

Parameters

- **key** – the annotation key
- **value** – the annotation value
- **ext** – a dictionary of extended properties in FBCv3 id, name, uri

deleteAnnotation(*key*)

Unsets (deltes) an objects annotation with key.

Parameters

key – the annotation key

setPid(*fid*)

Sets the object Id.

Parameters

fid – a valid c variable style id string

setId(*fid*)

Sets the object Id.

Parameters

fid – a valid c variable style id string

setMetaId(*mid=None*)

Sets the object Id.

Parameters

mid – [default=None] a valid c variable style metaid string, if None it will be set as meta+id

clone()

Return a clone of this object.

Cloning performs a deepcopy on the object which will also clone any objects that exist as attributes of this object, in other words an independent copy of the original. If this is not the desired behaviour override this method when subclassing or implement your own.

serialize(*protocol=0*)

Serialize object, returns a string by default.

Parameters

protocol – [default=0] serialize to a string or binary if required,

serializeToDisk(*filename, protocol=2*)

Serialize to disk using pickle protocol:

Parameters

- **filename** – the name of the output file
- **protocol** – [default=2] serialize to a string or binary if required,

addMIRIAMannotation(*qual, entity, mid*)

Add a qualified MIRIAM annotation or entity:

Parameters

- **qual** – a Biomodels biological qualifier e.g. “is” “isEncodedBy”
- **entity** – a MIRIAM resource entity e.g. “ChEBI”
- **mid** – the entity id e.g. CHEBI:17158 or fully qualifies url (if only_qual_uri)

addMIRIAMuri(*qual, uri*)

Add a qualified MIRIAM annotation or entity:

Parameters

- **qual** – a Biomodels biological qualifier e.g. “is” “isEncodedBy”
- **uri** – the fully qualified entity id e.g. <http://identifiers.org/chebi/CHEBI:12345> (no validity checking is done)

getMIRIAMannotations()

Returns a dictionary of all MIRIAM annotations associated with this object or None if there are none defined.

deleteMIRIAMannotation(*qual, entity, mid*)

Deletes a qualified MIRIAM annotation or entity:

Parameters

- **qual** – a Biomodels biological qualifier e.g. “is” “isEncodedBy”

- **entity** – a MIRIAM resource entity e.g. “ChEBI”
- **mid** – the entity id e.g. CHEBI:17158

getCompartmentId()

Return the compartment id where this element is located.

setCompartmentId(*compartment*)

Set the compartment id where this element is located.

getSBOTerm()

Return the SBO term for this object.

setSBOTerm(*sbo*)

Set the SBO term for this object.

Parameters

sbo – the SBOterm with format: SBO:nnnnnnn”

class cbmpy.CBModel.**Model**(*pid*)

Bases: *Fbase*

Container for constraint based model, adds methods for manipulating:

- objectives
- constraints
- reactions
- species
- compartments
- groups
- parameters
- N a structmatrix object

prefix = ''

suffix = ''

obj_func = None

activeObjIdx = None

N = None

sourcefile = ''

description = ''

CM = None

sensitivity = None

SCALED_REDUCED_COSTS = False

SOLUTION_STATUS_INT = None

SOLUTION_STATUS = None

DATE_CREATED = None

DATE_MODIFIED = None

__init__(*pid*)

Container for constraint based model.

Parameters

pid – a string with the model ID

objectives = None

flux_bounds = None

reactions = None

species = None

compartments = None

groups = None

genes = None

gpr = None

parameters = None

user_defined_constraints = None

user_constraints = None

MODEL_CREATORS = None

registerObjectInGlobalStore(*obj*)

Parameters

object

unRegisterObjectInGlobalStore(*sid*)

Parameters

sid

getModel()

Overrides the FBase inherited method, returns own instance.

clone()

Return a clone of this object.

getObject(*pid*)

Parameters

pid – returns a model object with pid or None

hasObject(*pid*)

Parameters

pid – returns a boolean if there is a registered object with pid-

serialize(*protocol=0*)

Serialize object, returns a string by default.

Parameters

protocol – [default=0] serialize to a string or binary if required,

serializeToDisk(*filename, protocol=2*)

Serialize to disk using pickle protocol:

Parameters

- **filename** – the name of the output file
- **protocol** – [default=2] serialize to a string or binary if required,

addMIRIAMannotation(*qual, entity, mid*)

Add a qualified MIRIAM annotation or entity:

Parameters

- **qual** – a Biomodels biological qualifier e.g. “is” “isEncodedBy”
- **entity** – a MIRIAM resource entity e.g. “ChEBI”
- **mid** – the entity id e.g. CHEBI:17158

setDescription(*html*)

Sets the model description which translates into the SBML <notes> field.

Parameters

html – any valid html or the empty string to clear “

getDescription()

Returns the model description which was stored in the SBML <notes> field.

setCreatedDate(*date=None*)

Set the model created date tuple(year, month, day, hour, minute, second)

Parameters

date – [default=None] default is now (automatic) otherwise (year, month, day, hour, minute, second) e.g. (2012, 09, 24, 13, 34, 00)

setModifiedDate(*date=None*)

Set the model modification date: tuple(year, month, day, hour, minute, second)

Parameters

date – [default=None] default is now (automatic) otherwise (year, month, day, hour, minute, second) e.g. (2012, 09, 24, 13, 34, 00)

addModelCreator(*firstname, lastname, organisation=None, email=None*)

Add a model creator to the list of model creators, only the first and fmaily names are mandatory:

Parameters

- **firstname**
- **lastname**
- **organisation** – [default=None]
- **email** – [default=None]

getModelCreators()

Return model creator information.

addObjective(*obj, active=False*)

Add an instantiated Objective object to the FBA model.

Parameters

- **obj** – an instance of the Objective class
- **active** – [default=False] flag this objective as the active objective (fba.activeObjIdx)

createObjectiveFunction(*rid, coefficient=1, osense='maximize', active=True, delete_current_obj=True*)

Create a single variable objective function:

- **rid** The
- **coefficient** [default=1]
- **osense** [default='maximize']

- **active** [default=True]
- **delete_current_obj** [default=True]

createSpecies(*sid*, *boundary=False*, *name=""*, *value=nan*, *compartment=None*,
charge=None, *chemFormula=None*)

Create a new species and add it to the model:

- **id** the unique species id
- **boundary** [default=False] whether the species is a variable (False) or is a boundary parameter (fixed)
- **name** [default=""] the species name
- **value** [default=nan] the value *not currently used*
- **compartment** [default=None] the compartment the species is located in
- **charge** [default=None] the species charge
- **chemFormula** [default=None] the chemical formula

createCompartment(*cid*, *name=None*, *size=1*, *dimensions=3*, *volume=None*)

Create a new compartment and add it to the model if the id does not exist.

Parameters

- **cid** – compartment id
- **name** – [None] compartment name
- **size** – [1] compartment size
- **dimensions** – [3] compartment size dimensions
- **volume** – [None] compartment volume

createParameter(*pid*, *value*, *constant=True*)

Instantiates a parameter.

createReaction(*rid*, *name=None*, *reversible=True*, *create_default_bounds=True*,
silent=False, *new=False*)

Create a new blank reaction and add it to the model:

Parameters

- **id** – the unique reaction ID
- **name** – the reaction name
- **reversible** – [default=True] the reaction reversibility. True is reversible, False is irreversible

- **create_default_bounds** – create default reaction bounds, irreversible $0 \leq J \leq \text{INF}$, reversible $-\text{INF} \leq J \leq \text{INF}$
- **silent** – [default=False] if enabled this disables the printing of information messages

createReactionNew(*rid*, *name=None*, *reversible=True*, *create_default_bounds=True*, *silent=False*, *new=False*)

Create a new blank reaction and add it to the model:

Parameters

- **id** – the unique reaction ID
- **name** – the reaction name
- **reversible** – [default=True] the reaction reversibility. True is reversible, False is irreversible
- **create_default_bounds** – create default reaction bounds, irreversible $0 \leq J \leq \text{INF}$, reversible $-\text{INF} \leq J \leq \text{INF}$
- **silent** – [default=False] if enabled this disables the printing of information messages

createReactionReagent(*reaction*, *metabolite*, *coefficient*, *silent=False*)

Add a reagent to an existing reaction, both reaction and metabolites must exist.

Parameters

- **reaction** – a reaction id
- **metabolite** – a species/metabolite id
- **coefficient** – the reagent coefficient

createReactionLowerBound(*reaction*, *value*)

Create a new lower bound for a reaction: $\text{value} \leq \text{reaction}$

- **reaction** the reaction id
- **value** the value of the bound

createReactionUpperBound(*reaction*, *value*)

Create a new upper bound for a reaction: $\text{reaction} \leq \text{value}$

- **reaction** the reaction id
- **value** the value of the bound

createReactionBounds(*reaction*, *lb_value*, *ub_value*)

Create a new lower bound for a reaction: $\text{value} \leq \text{reaction}$

- **reaction** the reaction id

- **lb_value** the value of the lower bound
- **ub_value** the value of the upper bound

createUserDefinedConstraint(*pid, lb, ub, components=None*)

Create an FBCv3 UserDefinedConstraint.

Parameters

- **pid** – unique id
- **lb** – lower bound float/parameter
- **ub** – upper bound float/parameter
- **components** – optional, the user defined constraint components in the form of a list

addUserDefinedConstraint(*udc*)

Add a User Defined Constraint object to the FBA model.

Parameters

- **obj** – an instance of the UserDefinedConstraint class

addFluxBound(*fluxbound, fbexists=None*)

Add an instantiated FluxBound object to the FBA model.

Parameters

- **fluxbound** – an instance of the FluxBound class
- **fbexists** – [default=None] this is a list of strings which contains fluxbounds that have been added to the model, see sample code below.

addSpecies(*species*)

Add an instantiated Species object to the FBA model.

Parameters

- **species** – an instance of the Species class

addGene(*gene*)

Add an instantiated Gene object to the FBA model.

Parameters

- **gene** – an instance of the G class

addParameter(*par*)

Add an instantiated Parameter object to the model.

Parameters

- **par** – an instance of the Parameter class

addCompartment(*comp*)

Add an instantiated Compartment object to the CBM model.

Parameters

comp – an instance of the Compartment class

addReaction(*reaction*, *create_default_bounds=False*, *silent=False*)

Adds a reaction object to the model.

Parameters

- **reaction** – an instance of the Reaction class
- **create_default_bounds** – create default reaction bounds, irreversible $0 \leq J \leq \text{INF}$, reversible $-\text{INF} \leq J \leq \text{INF}$
- **silent** – [default=False] if enabled this disables the printing of information messages

convertUserConstraintsToUserDefinedConstraints()

If a model is loaded with the old CBMPy specific constraint data structures json files and dictionaries, this function will upmark it to the new FBCv3 data structures.

copyUserDefinedConstraintsToUserConstraints()

This is a workaround until I complete full UserDefinedConstraints support.

addUserConstraint(*pid*, *fluxes=None*, *operator='>='*, *rhs=0.0*)

Add a user defined constraint to FBA model, this is additional to the automatically determined Stoichiometric constraints.

Parameters

- **pid** – user constraint name/id, use *None* for auto-assign
- **fluxes** – a list of (coefficient, reaction id) pairs where coefficient is a float
- **operator** – is one of '=', '>=' or '<=' (< and > will be interpreted as >= or <=)
- **rhs** – a float

deleteReactionAndBounds(*rid*)

Delete all reaction and bounds connected to reaction.

Parameters

rid – a valid reaction id

deleteObjective(*objective_id*)

Delete objective function:

objective_id the id of the objective function. If *objective_id* is given as 'active' then the active objective is deleted.

deleteBoundsForReactionId(*rid*, *lower=True*, *upper=True*)

Delete bounds connected to reaction, *rid*.

Parameters

- **rid** – a valid reaction id
- **upper** – [default=True] delete the upper bound
- **lower** – [default=True] delete the lower bound

createGeneAssociationsFromAnnotations(*annotation_key='GENE ASSOCIATION'*,
replace_existing=True)

Add genes to the model using the definitions stored in the annotation key. If this fails it tries some standard annotation keys: GENE ASSOCIATION, GENE_ASSOCIATION, gene_association, gene association.

Parameters

- **annotation_key** – the annotation dictionary key that holds the gene association for the protein/enzyme
- **replace_existing** – [default=True] replace existing annotations, otherwise only new ones are added

addGPRAssociation(*gpr*, *update_idx=True*)

Add a GeneProteinAssociation instance to the model.

Parameters

- **gpr** – an instantiated GeneProteinAssociation object

createGeneProteinAssociation(*protein*, *assoc*, *gid=None*, *name=None*,
gene_pattern=None, *update_idx=True*,
altlabels=None)

Create and add a gene protein relationship to the model, note genes are mapped on protein objects which may or may not be reactions.

Parameters

- **protein** – in this case the reaction
- **assoc** – the COBRA style gene protein association
- **gid** – the unique id
- **name** – the optional name
- **gene_pattern** – deprecated, not needed anymore
- **update_idx** – update the model gene index, not used

- **altlabels** – [default=None] alternative labels for genes, default uses geneIds

createGeneProteinAssociationFromTree(*protein, gpmtree, gid=None, name=None, gene_pattern=None, update_idx=True, altlabels=None*)

Create and add a gene protein relationship to the model, note genes are mapped on protein objects which may or may not be reactions.

Parameters

- **protein** – in this case the reaction
- **gpmtree** – the CBMPy GPR dictionary tree
- **gid** – the unique id
- **name** – the optional name
- **gene_pattern** – deprecated, not needed anymore
- **update_idx** – update the model gene index, not used
- **altlabels** – [default=None] alternative labels for genes, default uses geneIds

getAllProteinGeneAssociations(*use_labels=False*)

Returns a dictionary of the proteins associated with each gene.

Parameters

- use_labels** – use V2 gene labels rather than ID's

getAllGeneProteinAssociations(*use_labels=False*)

Returns a dictionary of genes associated with each protein.

Parameters

- use_labels** – use V2 gene labels rather than ID's

getGene(*gid*)

Returns a gene object that has the identifier:

Parameters

- gid** – the gene identifier

getGPRassociation(*gprid*)

Returns a gene protein association object that has the identifier:

Parameters

- gprid** – the gene protein identifier

getGPRforReaction(*rid*)

Return the GPR associated with the reaction id:

Parameters**rid** – a reaction id**getGPRforReactionAsDict**(*rid*, *useweakref=True*)

Return the GPR associated with the reaction id as a nested dictionary structure:

Parameters**rid** – a reaction id**getGPRIdAssociatedWithGeneId**(*gid*)

Return the GPR(s) associated with the gene id:

Parameters**gid** – a gene id**getGPRIdAssociatedWithGeneLabel**(*label*)

Return the GPR Id's associated with the gene label:

Parameters**label** – a gene label**getReactionActivity**(*rid*)

If there is a GPR and genes associated with the reaction ID then return either active=True or inactive=False Note if there is no gene associated information then this will return active.

Parameters**rid** – a reaction id**getGeneIds**(*substring=None*)

Returns a list of gene Ids, applies a substring search if substring is defined.

Parameters**substring** – search for this pattern anywhere in the id**getGeneObjects**(*substring=None*)

Returns a list of gene objects, applies a substring search if substring is defined.

Parameters**substring** – search for this pattern anywhere in the id**getGeneObjectsByLabel**(*substring=None*)

Returns a list of gene objects, applies a substring search if substring is defined.

Parameters**substring** – search for this pattern anywhere in the label**getGPRIds**(*substring=None*)

Returns a list of GPR Id's, applies a substring search if substring is defined.

Parameters

substring – search for this pattern anywhere in the id

getGPRObjects(*substring=None*)

Returns a list of GPR objects, applies a substring search if substring is defined.

Parameters

substring – search for this pattern anywhere in the id

getGeneLabels(*substring=None*)

Returns a list of gene labels (locus tags), applies a substring search if substring is defined.

Parameters

substring – search for this pattern anywhere in the label

getAllGeneActivities()

Returns a dictionary of genes (if defined) and whether they are active or not.

getAllProteinActivities()

Returns a dictionary of reactions (if genes and GPR's are defined) and whether they are active or not.

setAllProteinActivities(*activities, lower=0.0, upper=0.0*)

Given a dictionary of activities [rid : boolean] pairs set all the corresponding reactions:

Parameters

- **activities** – a dictionary of [rid : boolean] pairs
- **lower** – [default=0.0] the lower bound of the deactivated flux
- **upper** – [default=0.0] the upper bound of the deactivated flux

setAllInactiveGPRBounds(*lower=0.0, upper=0.0*)

Set all reactions that are inactive (as determined by gene and gpr evaluation) to bounds:

Parameters

- **lower** – [default=0.0] the new lower bound
- **upper** – [default=0.0] the new upper bound

resetAllInactiveGPRBounds()

Resets all reaction bounds modified by the `cmod.setAllInactiveGeneReactionBounds()` method to their previous values.

changeAllFluxBoundsWithValue(*old, new*)

Replaces all flux bounds with value “old” with a new value “new”:

Parameters

- **old** – value

- **new** – value

deleteAllFluxBoundsWithValue(*value*)

Delete all flux bounds which have a specified value:

Parameters

value – the value of the flux bound(s) to delete

deleteCompartment(*sid*, *check_components=True*)

Deletes a compartment object with id. Returns True if the compartment is deleted, False if not. In addition if components were checked a list of id's that reference the compartment are also returned.

Parameters

- **sid** – the compartment id
- **check_components** – [default=True] if enabled check that no species or reactions makes

deleteSpecies(*sid*, *also_delete=None*)

Deletes a species object with id.

Parameters

- **sid** – the species id
- **also_delete** – [default=None] only delete the species

deleteGene(*gid*, *also_delete_gpr=True*)

Deletes the gene object with gid. Note if you want to delete a gene by label (locus tag etc) use the deleteGeneByLabel() function.

Parameters

- **gid** – the gene Id
- **also_delete_gpr** – [default=True] automatically delete GPR's that contain no gene references

deleteGeneByLabel(*label*, *also_delete_gpr=True*)

Deletes the gene object with label (b2003 etc).

Parameters

- **label** – the gene with label to be deleted
- **also_delete_gpr** – [default=True] automatically delete GPR's that contain no gene references

deleteGPRAssociation(*gprid*)

Delete a GPR association with id.

Parameters**gprid** – the GPR association id**deleteNonReactingSpecies** (*simulate=True*)

Deletes all species that are not reagents (do not to take part in a reaction). *Warning* this deletion is permanent and greedy (not selective). Returns a list of (would be) deleted species.

Parameters**simulate** – [default=True] only return a list of the speciesId's that would have been deleted if False**getCompartment** (*cid*)

Returns a compartment object with *cid*

Parameters**cid** – compartment ID**getReaction** (*rid*)

Returns a reaction object with *id*

Parameters**rid** – reaction ID**getSpecies** (*sid*)

Returns a species object with *sid*

Parameters**sid** – a species ID**getParameter** (*pid*)

Returns a parameter object with *pid*.

getReactionBounds (*rid*)

Get the bounds of a reaction, returns a tuple of *rid*, lowerbound value, upperbound value and equality value (None means bound does not exist).

Parameters**rid** – the reaction ID**getReactionLowerBound** (*rid*)

Returns the lower bound of a reaction (it it exists) or None.

Parameters**rid** – the reaction ID**getReactionUpperBound** (*rid*)

Returns the upper bound of a reaction (it it exists) or None.

Parameters**rid** – the reaction ID

getFluxBoundByID(*fid*)

Returns a FluxBound/Parameter with id.

Parameters

fid – the fluxBound ID

getFluxBoundByReactionID(*rid*, *bound*)

Returns a FluxBound/Parameter instance.

Parameters

- **rid** – the reaction ID
- **bound** – the bound: ‘upper’, ‘lower’, ‘equality’

getFluxBoundsByReactionID(*rid*)

Returns all FluxBound instances connected to a reactionId as a tuple of valid (lower, upper, None) or (None, None, equality) or alternatively invalid (lower, upper, equality).

Parameters

rid – the reaction ID

getCompartmentIds(*substring=None*)

Returns a list of compartment Ids, applies a substring search if substring is defined.

Parameters

substring – search for this pattern anywhere in the id

getCompartmentObjects(*substring=None*)

Returns a list of compartment objects, applies a substring search if substring is defined.

Parameters

substring – search for this pattern anywhere in the id

getSpeciesIds(*substring=None*)

Returns a list of species Ids, applies a substring search if substring is defined.

Parameters

substring – search for this pattern anywhere in the id

getSpeciesObjects(*substring=None*)

Returns a list of species objects, applies a substring search if substring is defined.

Parameters

substring – search for this pattern anywhere in the id

getReactionIds(*substring=None*)

Returns a list of reaction Ids, applies a substring search if substring is defined.

Parameters

substring – search for this pattern anywhere in the id

getReactionObjects(*substring=None*)

Returns a list of reaction objects, applies a substring search if substring is defined.

Parameters

substring – search for this pattern anywhere in the id

getReactionNames(*substring=None*)

Returns a list of reaction names, applies a substring search if substring is defined.

Parameters

substring – search for this pattern anywhere in the name

getFluxBoundIds(*substring=None*)

Returns a list of fluxbound Ids, applies a substring search if substring is defined.

Parameters

substring – search for this pattern anywhere in the id

getObjectiveIds(*substring=None*)

Returns a list of objective function Ids, applies a substring search if substring is defined.

Parameters

substring – search for this pattern anywhere in the id

getOptimalValue()

Returns the optimal value of the objective function.

getObjFuncValue()

Returns the objective function value.

getActiveObjective()

Returns the active objective object.

getActiveObjectiveStoichiometry()

Returns a list of (coefficient, flux_objective) tuples.

getActiveObjectiveReactionIds()

Returns the active objective flux objective reaction id's.

setActiveObjective(*objId*)

setBoundValueByName(*rid, value, bound*)

Deprecated use setReactionBound. Set a reaction bound

Parameters

- **rid** – the reactions id
- **value** – the new value
- **bound** – this is either 'lower' or 'upper'

getGeneIdFromLabel(*label*)

Given a gene label it returns the corresponding Gene id or None.

Parameters

label

getGeneByLabel(*label*)

Given a gene label return the corresponding Gene object.

Parameters

label

setGeneInactive(*g_id*, *update_reactions=False*, *lower=0.0*, *upper=0.0*)

Effectively deletes a gene by setting it's inactive flag while optionally updating the GPR associated reactions.

Parameters

- **g_id** – a gene ID
- **update_reactions** – [default=False] update the associated reactions fluxbounds
- **lower** – [default=0.0] the deactivated reaction lower bound
- **upper** – [default=0.0] the deactivated reaction upper bound

setGeneActive(*g_id*, *update_reactions=False*)

Effectively restores a gene by setting it's active flag.

Parameters

- **g_id** – a gene ID
- **update_reactions** – [default=False] update the associated reactions fluxbounds from the gene deletion bounds if they exist

updateNetwork(*lower=0.0*, *upper=0.0*, *silent=False*)

Update the reaction network based on gene activity. If reaction is deactivated then lower and upper bounds are used.

Parameters

- **lower** – [default=0.0] deactivated lower bound
- **upper** – [default=0.0] deactivated upper bound

resetAllGenes(*update_reactions=False*)

Resets all genes to their default activity state (normally on)

Parameters

- **update_reactions** – [default=False] update the associated reactions fluxbounds from the gene deletion bounds if they exist

setReactionBound(*rid, value, bound*)

Set a reaction bound.

Parameters

- **rid** – the reactions id
- **value** – the new value
- **bound** – this is either ‘lower’ or ‘upper’, or ‘equal’

setReactionBounds(*rid, lower, upper*)

Set both the upper and lower bound of a reaction:

Parameters

- **rid** – the good old reaction id
- **lower** – the lower bound value
- **upper** – the upper bound value

setReactionLowerBound(*rid, value*)

Set a reactions lower bound (if it exists)

Parameters

- **rid** – the reactions id
- **value** – the new value

setReactionUpperBound(*rid, value*)

Set a reactions upper bound (if it exists)

Parameters

- **rid** – the reaction id
- **value** – the new value

getAllFluxBounds()

DEPRECATED Returns a dictionary of all flux bounds [id:value]

setAllFluxBounds(*bounds*)

DEPRECATED! use setFluxBoundsFromDict() Sets all the fluxbounds present in bounds

Parameters

bounds – a dictionary of [fluxbound_id : value] pairs (not per reaction!!!)

setFluxBoundsFromDict(*bounds*)

DEPRECATED! This method will be modified to use reaction Idin CBMPy 0.9.0 Sets all the fluxbounds present in bounds.

Parameters

bounds – a dictionary of [fluxbound_id : value] pairs (not per reaction!!!)

renameObjectIds(*prefix=None, suffix=None, target='all', ignore=None*)

This method is designed for target="all" other use may result in inconsistent models. Update: "species" and "reactions" should also work as advertised, please check results.

Parameters

- **prefix** – [None] if supplied add as a prefix
- **suffix** – [None] if supplied add as a suffix
- **target** – ['all'] specify what class of objects to rename
- **ignore** – [default=None] a list of id's to ignore

setPrefix(*prefix, target*)

This is alpha stuff, target can be:

- 'species'
- 'reactions'
- 'constraints'
- 'objectives'
- 'all'

setSuffix(*suffix, target*)

This is alpha stuff, target can be:

- 'species'
- 'reactions'
- 'constraints'
- 'objectives'
- 'all'

setObjectiveFlux(*rid, coefficient=1, osense='maximize', delete_objflx=True*)

Set single target reaction flux for the current active objective function.

Parameters

- **rid** – a string containing a reaction id
- **coefficient** – [default=1] an objective flux coefficient
- **osense** – the optimization sense must be **maximize** or **minimize**
- **delete_objflx** – [default=True] delete all existing fluxObjectives in the active objective function

sortReactionsById()

Sorts the reactions by Reaction.id uses the python string sort.

sortSpeciesById()

Sorts the reaction list by Reaction.id uses the python string sort.

getExchangeReactions()

Returns reaction instances where the 'is_exchange' attribute set to True.

This is by default reactions that contain a boundary species.

getExchangeReactionIds()

Returns id's of reactions where the 'is_exchange' attribute set to True.

This is by default reactions that contain a boundary species.

getReactionValues(*only_exchange=False*)

Returns a dictionary of ReactionID : ReactionValue pairs:

Parameters

only_exchange – [default=False] only return the reactions labelled as exchange

getSolutionVector(*names=False*)

Return a vector of solution values.

Parameters

names – [default=False] if True return a solution vector and list of names

getReversibleReactionIds()

Return a list of reversible reaction Id's.

getIrreversibleReactionIds()

Return a list of irreversible reaction Id's.

getBoundarySpeciesIds(*rid=None*)

Return all boundary species associated with reaction.

- **rid** [default=None] by default return all boundary species in a model, alternatively a string containing a reaction id or list of reaction id's

findFluxesForConnectedSpecies(*metab*)

Returns a list of (reaction, flux value) pairs that this metabolite appears as a reagent of.

Parameters

metab – the metabolite name

getFluxesAssociatedWithSpecies(*metab*)

Returns a list of (reaction, flux value) pairs that this metabolite appears as a reagent in.

Parameters**metab** – the metabolite name**getReactionIdsAssociatedWithSpecies(*metab*)**

Returns a list of (reaction, flux value) pairs that this metabolite appears as a reagent in.

Parameters**metab** – the metabolite name**getFluxesAssociatedWithCompartments(*compartments*)**

Determines all reactions and flux values associated with a list of compartments. This function can be used to find all transport reactions between compartments, e.g. the cytosol and mitochondria. If the compartment IDs are 'cyt' and 'mit', respectively, you can call "your_model.getFluxesAssociatedWithCompartments(['cyt', 'mit'])" to get all fluxes between these compartments.

compartments: a list or set of compartment IDs. To check the existing compartment IDs in your model call "your_model.getCompartmentIds()"

:returns a dictionary with reaction IDs as keys and corresponding flux values as values

splitEqualityFluxBounds()

Splits any equalit flux bounds into lower and upper bounds.

buildStoichMatrix(*matrix_type*='numpy', *only_return*=False)

Build the stoichiometric matrix N and additional constraint matrix CN (if required)

Parameters

- **matrix_type** – [default='numpy'] the type of matrix to use to generate constraints
- **numpy** – a NumPy matrix default
- **sympy** – a SymPy symbolic matrix, if available note the denominator limit can be set in CBModel.
__CBCONFIG__['SYMPY_DENOM_LIMIT'] = 10**12
- **scipy_csr** – create using NumPy but store as SciPy csr_sparse
- **only_return** – [default=False] **IMPORTANT** only returns the stoichiometric matrix and constraint matrix (if required),

createSingleGeneEffectMap()

This takes a model and analyses the logical gene expression patterns. This only needs to be done once, the result is a dictionary that has boolean effect patterns as keys and the (list of) genes that give rise to those patterns as values. This map is used by the single gene deletion method for further analysis.

Note this dictionary can also be stored and retrieved separately as long as the model structure is not changed i.e. the gene associations themselves or order of reactions

(stored as the special entry ‘keyJ’).

Stored as self.__single_gene_effect_map__

testGeneProteinAssociations()

This method will test the GeneProtein associations and return a list of protein, association pairs.

exportFVAdata()

Export the fva data as an array and list of reaction id’s.

importUserConstraints(filename)

Exports user constraints in json.

exportUserConstraints(filename)

Exports user constraints in json.

createGroup(gid)

Create an empty group with.

Parameters

gid – the unique group id

addGroup(obj)

Add an instantiated group object to the model.

Parameters

obj – the Group instance

deleteGroup(gid)

Delete a group with.

Parameters

gid – the unique group id

getGroup(gid)

Return a group with.

Parameters

gid – the unique group id

getGroupIds()

Get all group ids.

getGroupNames()

Get all group names.

getGroupMembership()

Returns group membership of items in groups.

Returns {object_id: [‘group_id1’, ‘group_id2’]}

emptyUndelete()

Empties the undelete cache.

class cbmpy.CBModel.Objective(*pid, operation*)

Bases: *Fbase*

An objective function.

value = None

solution = None

__init__(*pid, operation*)

operation = None

flux_objectives = None

setOperation(*operation*)

Sets the objective operation (sense)

Parameters

operation – [default='maximize'] one of 'maximize', 'maximise', 'max', 'minimize', 'minimise', 'min'

getOperation()

Returns the operation or sense of the objective.

addFluxObjective(*fobj, override=False*)

Adds a FluxObjective instance to the Objective.

Parameters

- **fobj** – the FluxObjective object
- **override** – [default=False] override pushing the global id map, this should never be used

createFluxObjectives(*fluxlist*)

Create and add flux objective objects to this objective function.

Parameters

fluxlist – a list of one or more ('coefficient', 'rid', 'type') triples

createQuadraticFluxObjectives(*fluxlist*)

Create and add quadratic flux objective objects to this objective function.

Parameters

fluxlist – a list of one or more ('coefficient', 'rid', 'rid2', 'type') triples

deleteAllFluxObjectives()

Delete all flux objectives.

getFluxObjectiveIDs()

Returns a list of ObjectiveFlux ids, for the reaction id's use *getFluxObjectiveReactions()* or for coefficient, fluxobjective pairs use *getFluxObjectiveData()*

getFluxObjectiveForReaction(rid)

Returns the FluxObjective associated with the supplied rid. If there is more than fluxObjective associated with a reaction (illegal) then a list of fluxObjectives is returned.

rid a reaction id

getFluxObjectiveReactions()

Returns a list of reactions that are used as flux_objectives.

getFluxObjectiveData()

Returns a list of ObjectiveFunction components as (coefficient, flux, type) pairs.

getFluxObjective(foid)

Return the flux objective with id.

Parameters

foid – the flux objective id returns either an object or a list if there are multiply defined flux objectives

getFluxObjectives()

Returns the list of FluxObjective objects.

getValue()

Returns the current value of the attribute (input/solution)

setValue(value)

Sets the attribute "value".

getLinearFluxObjectives()

Returns a list of linear variable flux objective objects.

getQuadraticFluxObjectives()

Returns a list of quadratic variable flux objective objects.

getQuadraticBivariateFluxObjectives()

Returns a list of bivariate quadratic variable flux objective objects.

```
class cbmpy.CBModel.FluxObjectiveQuadratic(pid, reaction, reaction2, coefficient=1,  
                                           ctype='quadratic')
```

Bases: [Fbase](#)

A weighted quadratic flux that appears in an objective function, this fluxobjective contains two reaction terms to define “quadratic” fluxobjectives of the type $\text{coefficient} \times \text{variable1} \times \text{variable2}$. For example $2 \times R1 \times R2$.

NOTE: reaction is a string containing a reaction id

```
ctypes = 'quadratic'
```

```
__init__(pid, reaction, reaction2, coefficient=1, ctype='quadratic')
```

```
ctype = None
```

```
reaction = None
```

```
reaction2 = None
```

```
coefficient = None
```

```
getReactionIds()
```

```
getReactionId()
```

```
getReactionId2()
```

```
getCoefficient()
```

```
setReactionId(reaction)
```

```
setReactionId2(reaction2)
```

```
setReactionIds(reaction, reaction2)
```

```
setCoefficient(coefficient)
```

```
getType()
```

```
setType(ctype)
```

```
class cbmpy.CBModel.FluxObjective(pid, reaction, coefficient=1, ctype='linear')
```

Bases: [Fbase](#)

A weighted flux that appears in an objective function.

NOTE: reaction is a string containing a reaction id

```
ctypes = ('linear', 'quadratic')
```

```
__init__(pid, reaction, coefficient=1, ctype='linear')
```

```
ctype = None
```

```
reaction = None
```

```
coefficient = None
getReactionId()
getCoefficient()
setReactionId(reaction)
setCoefficient(coefficient)
getType()
setType(ctype)
```

```
class cbmpy.CBModel.UserDefinedConstraint(pid, lb, ub)
```

Bases: [*Fbase*](#)

This is an FBCv3 class that defines a set of user defined constraints, it is similar to an objective constraint except allows parameters as coefficients and values in the constraint components.

```
solution = None
ub = None
lb = None
__init__(pid, lb, ub)
constraint_components = None
setLowerBound(bnd)
setUppperBound(bnd)
getUpperBound()
getLowerBound()
getConstraintComponentIDs()
getConstraintComponentForVariable(rid)
    rid a component id.
getConstraintComponentVariables()
getConstraintComponentVariableTypes()
getConstraintComponentData()
```

getConstraintComponent(*cid*)

Parameters

cid – get a constraint component that matches cid

getConstraintComponents()

createConstraintComponent(*pid, coefficient, variable, ctype*)

addConstraintComponent(*cc*)

Parameters

cc – UserConstraintComponent

setObjRefOnComponents()

class cbmpy.CBModel.**ConstraintComponent**(*pid, coefficient, variable, ctype='linear'*)

Bases: [*Fbase*](#)

A weighted flux that appears in an user defined constraint.

ctypes = ('linear', 'quadratic')

__init__(*pid, coefficient, variable, ctype='linear'*)

variable = None

coefficient = None

ctype = None

getVariable()

getCoefficient()

setVariable(*variable*)

setCoefficient(*coefficient*)

setType(*ctype*)

getType()

getVariableType()

class cbmpy.CBModel.**Compartment**(*pid, name=None, size=1, dimensions=3,*
volume=None)

Bases: [*Fbase*](#)

A compartment.

```
__init__(pid, name=None, size=1, dimensions=3, volume=None)
```

Parameters

- **cid** – comapartment id
- **name** – optional compartment name
- **size** – compartment size, [default=1]
- **dimensions** – compartment spatial dimensions [default=3]

size = None

volume = None

dimensions = None

setId(fid)

Sets the object Id.

Parameters

fid – a valid c variable style id string

containsSpecies()

Lists the species contained in this compartment.

containsReactions()

Lists the species contained in this compartment.

getSize()

Get the compartment size.

getDimensions()

Get the compartment dimensions.

setSize(size)

Set the compartment size.

Parameters

size – the new compartment size

setDimensions(dimensions)

Get the compartment dimensions.

Parameters

dimensions – set the new compartment dimensions

class cbmpy.CBModel.GroupMemberAttributes

Bases: *Fbase*

Contains the shared attributes of the group members (equivalent to SBML annotation on ListOfMembers)

`__init__()`

class `cbmpy.CBModel.Group(pid)`

Bases: *Fbase*

Container for SBML groups.

kind = 'collection'

`__init__(pid)`

members = None

member_ids = None

clone()

Return a clone of this object.

Note the for Groups this is a shallow copy, in that the reference objects themselves are not cloned only the group (and attributes)

addMember(obj)

Add member CBMPy object(s) to the group.

Parameters

obj – either a single, tuple or list of CBMPy objects

deleteMember(oid)

Deletes a group member with group id.

Parameters

oid – group member id

hasMember(mid)

Returns a boolean indicating whether a member is in the group.

Parameters

mid – the id to check

getMembers(as_set=False)

Return the member objects of the group.

Parameters

as_set – return objects as a set rather than a list

getMember(mid)

Returns the group member object that corresponds to mid.

Parameters

mid – the id of the requested object

getMemberIDs(*as_set=False*)

Return the ids of the member objects.

Parameters

as_set – return id’s as a set rather than a list

getKind()

Return the group kind.

setKind(*kind*)

Sets the kind or type of the group, this must be one of: ‘collection’, ‘partonomy’, ‘classification’.

Parameters

kind – the kind

addSharedMIRIAMannotation(*qual, entity, mid*)

Add a qualified MIRIAM annotation or entity to the list of members (all) rather than the group itself:

Parameters

- **qual** – a Biomed models biological qualifier e.g. “is” “isEncodedBy”
- **entity** – a MIRIAM resource entity e.g. “ChEBI”
- **mid** – the entity id e.g. CHEBI:17158 or fully qualifies url (if only_qual_uri)

setSharedAnnotation(*key, value*)

Sets the list of members (all) annotation as a key : value pair.

Parameters

- **key** – the annotation key
- **value** – the annotation value

setSharedSBOTerm(*sbo*)

Set the SBO term for the the members of the group (all).

Parameters

sbo – the SBOTerm with format: “SBO:<7 digit integer>”

setSharedNotes(*notes*)

Sets the group of objects notes attribute (all):

Parameters

notes – the note string, should preferably be (X)HTML for SBML

getSharedMIRIAMannotations()

Return a dictionary of the shared member MIRIAM annotations (rather than the group attribute).

getSharedAnnotations()

Return a dictionary of the shared member annotations (rather than the group attribute).

getSharedSBOterm()

Return the shared member SBO term (rather than the group attribute).

getSharedNotes()

Return the shared member notes (rather than the group attribute).

assignSharedAnnotationToMembers()

This function merges or updates the group member objects annotations with the group shared annotation.

assignSharedMIRIAMannotationToMembers()

This function merges or updates the group member objects MIRIAM annotations with the group shared MIRIAM annotation.

assignSharedSBOtermsToMembers(*overwrite=False*)

Assigns the group shared member SBO term to the group members.

Parameters

overwrite – [default=False] overwrite the target SBO term if it is defined

assignSharedNotesToMembers(*overwrite=False*)

Assigns the group shared notes to the group members.

Parameters

overwrite – [default=False] overwrite the target notes if they are defined

assignAllSharedPropertiesToMembers(*overwrite=False*)

Assigns all group shared properties (notes, annotations, MIRIAM annotations, SBO) to the group members.

Parameters

overwrite – [default=False] overwrite the target notes if they are defined

serialize(*protocol=0*)

Serialize object, returns a string by default.

Parameters

protocol – [default=0] serialize to a string or binary if required,

serializeToDisk(*filename, protocol=2*)

Serialize to disk using pickle protocol:

Parameters

- **filename** – the name of the output file
- **protocol** – [default=2] serialize to a string or binary if required,

class cbmpy.CBModel.FluxBound(*pid, reaction, operation, value*)

Bases: *Fbase*

A reaction fluxbound.

value = None

__init__(*pid, reaction, operation, value*)

reaction = None

is_bound = None

operation = None

getType()

Returns the *type* of FluxBound: ‘lower’, ‘upper’, ‘equality’ or None.

getReactionId()

setReactionId(*react*)

Sets the reaction attribute of the FluxBound.

getValue()

Returns the current value of the attribute (input/solution)

setValue(*value*)

Sets the attribute ‘value’.

class cbmpy.CBModel.FluxBoundBase(*pid, operator, value, parent=None*)

Bases: *Fbase*

A refactored and streamlined FluxBound base class that can be a generic bound, superclass to FluxBoundUpper and FluxBoundLower.

__init__(*pid, operator, value, parent=None*)

Parameters

- **pid** – object id
- **operator** – <> GE or LE
- **value** – a float
- **parent** – [default=None] the parent reaction object

operator = None

getType()

Returns the *type* of FluxBound: 'lower', 'upper'.

getReactionId()

getValue()

Returns the current value of the attribute (input/solution)

setValue(value)

Sets the value attribute:

Parameters

value – a float

property value

Returns the current value of the attribute (input/solution)

class cbmpy.CBModel.FluxBoundUpper(reaction, value=inf)

Bases: [FluxBoundBase](#)

__init__(reaction, value=inf)

Upper Bound class, less flexible than generic superclass (no input checking) for model instantiation.

Parameters

- **value** – [default=inf] a float
- **reaction** – the parent Reaction

class cbmpy.CBModel.FluxBoundLower(reaction, value=-inf)

Bases: [FluxBoundBase](#)

__init__(reaction, value=-inf)

Lower Bound Class, less flexible than generic superclass (no input checking) for model instantiation.

Parameters

- **value** – [default=-inf] a float
- **reaction** – the parent Reaction

class cbmpy.CBModel.Parameter(pid, value, name=None, constant=True)

Bases: [Fbase](#)

Holds parameter information.

__init__(pid, value, name=None, constant=True)

Parameter definition class.

Parameters

- **pid** – the unique parameter pid
- **value** – the value
- **name** – [default=""] the parameter name
- **constant** – [default=True] is the parameter constant or can it be changed by a simulation/solver

constant = True

getValue()

Returns the current value of the attribute (input/solution)

setValue(value)

Sets the value attribute:

Parameters

value – a float

property value

Returns the current value of the attribute (input/solution)

getAssociations()

Return the Object ID's associated with this parameter.

addAssociation(assoc)

Add an object ID to associate with this object.

deleteAssociation(assoc)

Delete the object id associated with this object.

class cbmpy.CBModel.Reaction(pid, name=None, reversible=True)

Bases: *Fbase*

Holds reaction information.

is_exchange = False

value = None

reduced_cost = None

is_balanced = None

fva_min = None

fva_max = None

fva_status = None

__init__(*pid*, *name=None*, *reversible=True*)

reagents = None

reversible = None

addReagent(*reag*)

Adds an instantiated Reagent object to the reaction.

createReagent(*metabolite*, *coefficient*)

Create a new reagent and add it to the reaction:

- **metabolite** the metabolite name
- **coefficient** the

—negative coefficient is a substrate – positive coefficient is a product

Will fail if a species reference already exists

getReagentObjIds()

Returns a list of the reagent id's.

For the name of the reagents/metabolites use *<reaction>.getSpeciesIds()*

getReagentRefs()

Returns a list of the reagents/metabolites.

getSpeciesIds()

Returns a list of the reagents/metabolites.

getSpeciesObj()

Returns a list of the species objects that are reagents.

getFVAdata(*roundnum=None*, *silent=True*)

Returns the data generated by CBSolver.FluxVariabilityAnalysis() for this reaction as a tuple of (Flux, FVAmin, FVAmay, span) where span is abs(FVAmay - FVAmin). FVAmin or FVAmay is None this indicates no solution to that particular optimization (infeasible).

Parameters

- **roundnum** – [default=None] the integer number of roundoff decimals the default is no rounding
- **silent** – [default=True] suppress output to stdout

getReagent(*rid*)

Return the one or more reagent objects which have *rid*:

Parameters**rid** – a reagent *rid***changeId**(*pid*)

Changes the Id of the reaction and updates associated FluxBounds.

setId(*fid*)

Sets the object Id.

Parameters**fid** – a valid c variable style id string**getValue**()

Returns the current value of the flux.

setValue(*value*)Sets the attribute *value* in this case the flux.**getReagentWithSpeciesRef**(*sid*)Return the reagent object which refers to the *species* id. If there are multiple reagents that refer to the same species a list is returned.**Parameters****sid** – the species/metabolite id**setStoichCoefficient**(*sid*, *value*)Sets the stoichiometric coefficient of a reagent that refers to a metabolite. Note *negative coefficients* are *substrates* while *positive* ones are *products*. At this point zero coefficients are not allowed.**Parameters**

- **sid** – the species/metabolite id
- **value** – a floating point value != 0

getStoichiometry(*use_names=False*, *altout=False*)

Returns a list of (coefficient, species) pairs for this reaction.

Parameters

- **use_names** – [default = False] use species names rather than id's
- **altout** – [default = False] returns a dictionary [DEPRECATED]

getSubstrateIds(*use_names=False*)

Returns a list of the reaction substrates, species identifiers.

Parameters**use_names** – [default = False] use species names rather than id's

getProductIds(*use_names=False*)

Returns a list of the reaction products, species identifiers.

Parameters

use_names – [default = False] use species names rather than id's

getGPRassociationString(*use_labels=True*)

Return the GPR string associated with this reaction (assuming it exists) or None.

Parameters

use_labels – [default=True] return string with lab

deleteReagentWithSpeciesRef(*sid*)

Delete a reagent (or reagents) that refers to the species id:

Parameters

sid – a species/metabolite id

getLowerBound()

Get the value of the reactions lower bound.

getUpperBound()

Get the value of the reactions upper bound.

setLowerBound(*value*)

Set the value of the reactions lower bound.

Parameters

value – a floating point value

setUpperBound(*value*)

Set the value of the reactions upper bound.

Parameters

value – a floating point value

deactivateReaction(*lower=0.0, upper=0.0, silent=True*)

Deactivates a reaction by setting its bounds to lower and upper. Restore with `reactivateReaction()`

Parameters

- **lower** – [default=0.0] bound
- **upper** – [default=0.0] bound

reactivateReaction(*silent=True*)

Activates a reaction deactivated with `deactivateReaction`.

getEquation(*reverse_symb='=', irreversible_symb='>', use_names=False*)

Return a pretty printed string containing the reaction equation.

Parameters

- **reverse_symb** – [default = '='] the symbol to use for reversible reactions
- **irreverse_symb** – [default = '>'] the symbol to use for irreversible reactions
- **use_names** – [default = False] use species names rather than id's

class cbmpy.CBModel.ReactionNew(*pid, name=None, lb=-inf, ub=inf, reversible=True*)

Bases: [Reaction](#)

Extended reaction class with new upper/lower bound structure.

__init__(*pid, name=None, lb=-inf, ub=inf, reversible=True*)

ub = None

lb = None

setId(*fid*)

Sets the object Id.

Parameters

fid – a valid c variable style id string

getLowerBound()

Get the value of the reactions lower bound.

getUpperBound()

Get the value of the reactions upper bound.

setLowerBound(*value*)

Set the value of the reactions lower bound.

Parameters

value – a floating point value

setUpperBound(*value*)

Set the value of the reactions upper bound.

Parameters

value – a floating point value

deactivateReaction(*lower=0.0, upper=0.0, silent=True*)

Deactivates a reaction by setting its bounds to lower and upper. Restore with `reactivateReaction()`

Parameters

- **lower** – [default=0.0] bound

- **upper** – [default=0.0] bound

reactivateReaction(*silent=True*)

Activates a reaction deactivated with deactivateReaction.

setObjRefOnComponents()

class cbmpy.CBModel.**Species**(*pid, boundary=False, name=None, value=nan, compartment=None, charge=None, chemFormula=None*)

Bases: *Fbase*

Holds species/metabolite information.

shadow_price = None

__init__(*pid, boundary=False, name=None, value=nan, compartment=None, charge=None, chemFormula=None*)

Species/metabolite definition class.

- **pid** the unique species pid
- **boundary** [default=False] whether the species is a variable (False) or is a boundary parameter (fixed)
- **name** [default=""] the species name
- **value** [default=nan] the value *not currently used*
- **compartment** [default=None] the compartment the species is located in
- **charge** [default=None] the species charge, from v3 a float
- **chemFormula** [default=None] the chemical formula

value = None

is_boundary = False

charge = None

chemFormula = None

reagent_of = None

setId(*fid, allow_rename=False*)

Sets the object Id.

Parameters

fid – a valid c variable style id string

getValue()

Returns the current value of the attribute (input/solution)

setValue(*value*)

Sets the attribute “value”.

isReagentOf()

Returns a dynamically generated list of reactions that this species occurs as a reagent.

getReagentOf()

Returns a list of reaction id’s that this metabolite occurs in.

setReagentOf(*rid*)

Adds the supplied reaction id to the reagent_of list (if it isn’t one already)

Parameters

rid – a valid reaction id

setChemFormula(*cf*)

Sets the species chemical formula.

Parameters

cf – a chemical formula e.g. CH₃NO₂

getChemFormula()

Returns the species chemical formula.

setCharge(*charge*)

Sets the species charge:

Parameters

charge – a signed double but generally a signed int is used

getCharge()

Returns the species charge.

setBoundary()

Sets the species so it is a boundary metabolite or fixed which does not occur in the stoichiometric matrix N.

unsetBoundary()

Unsets the species boundary attribute so that the metabolite is free and therefore occurs in the stoichiometric matrix N.

class cbmpy.CBModel.Reagent(*pid*, *species_ref*, *coef*)

Bases: [Fbase](#)

Has a reactive species id and stoichiometric coefficient:

- negative = substrate
- positive = product
- species_ref a reference to a species obj

coefficient = None

role = None

__init__(*pid*, *species_ref*, *coef*)

Instantiates a reagent from a metabolite and coefficient, note that now the coefficient can be a Parameter object in which case a connection is made to the linked Parameter.

Parameters

- **pid** – a unique id
- **species_ref** – a reference to a species id
- **coefficient** – the stoichiometric coefficient, a non-zero integer or Parameter

species_ref = None

setCoefficient(*coef*)

Sets the reagent coefficient and role, negative coefficients are substrates and positive ones are products.

Parameters

coeff – the new coefficient

getCoefficient()

Returns the reagent coefficient.

setSpecies(*spe*)

Sets the metabolite/species that the reagent reference refers to.

getSpecies()

Returns the metabolite/species that the reagent reference refers to.

getRole()

Returns the reagents role, “substrate”, “product” or None.

class cbmpy.CBModel.**Gene**(*pid*, *label*=None, *active*=True)

Bases: *Fbase*

Contains all the information about a gene (or gene+protein construct depending on your philosophy)

label = None

__init__(*pid*, *label*=None, *active*=True)

A gene construct.

Parameters

- **pid** – the gene id

- **label** – the gene label this may or may not be a legal Sid
- **active** – is the gene is active or not (boolean)

active0 = False

active = False

setId(*fid*)

Sets the object Id.

Parameters

fid – a valid c variable style id string

getLabel()

Returns the gene label.

setLabel(*label*)

Sets the gene label, checks that the new label is unique.

setActive()

Set the gene to be active.

setInactive()

Set the gene to be inactive.

isActive()

Returns whether the gene is active or not.

resetActivity()

Reset the gene to its default activity state.

class cbmpy.CBModel.GeneProteinAssociation(*pid, protein, use_compiled=False*)

Bases: [*Fbase*](#)

This class associates genes to proteins.

assoc = None

tree = None

__init__(*pid, protein, use_compiled=False*)

Create a GeneProteinAssociation.

Parameters

- **pid** – a unique id
- **protein** – the protein the gene association refers to, in most cases this should be a reaction id

- **use_compiled** – [default=False] used compiled expressions for evaluation, potentially less portable

generefs = None

protein = None

use_compiled = False

evalAssociation()

Returns an integer value representing the logical associations or None.

buildEvalFunc()

addGeneref(*geneid*)

Add a gene reference to the list of gene references.

Parameters

geneid – a valid model Gene id

deleteGeneref(*gid*)

Deletes a gene reference.

Parameters

geneid – a valid model Gene id

addAssociation(*assoc*)

Add a gene/protein association expression.

createAssociationAndGeneRefsFromTree(*gprtree*, *altlabels*=None)

Evaluate the GPR tree and add the genes necessary to evaluate it Note that this GPR should be added to a model with `cmod.addGPRAssociation()` before calling this method.

Parameters

- **gprtree** – the CBMPy GPR tree data structure
- **altlabels** – [default=None] a dictionary containing a label<->id mapping

createAssociationAndGeneRefs()

createAssociationAndGeneRefsFromString(*assoc*, *altlabels*=None)

Evaluate the gene/protein association and add the genes necessary to evaluate it Note that this GPR should be added to a model with `cmod.addGPRAssociation()` before calling this method.

Parameters

- **assoc** – the COBRA style gene protein association

- **altlabels** – [default=None] a dictionary containing a label<->id mapping

getGenes()

Return a list of gene objects associated with this GPRass.

getGeneLabels()

Return a list of gene labels associated with this GPRass.

getGene(*gid*)

Return a gene object with id.

getAssociationStr(*use_labels=False*)

Return the gene association string, alternatively return string with labels.

Parameters

use_labels – [default=False] return the gene association string with labels rather than geneId's (FBCv2 issue)

getGeneIds()

Return a list of gene id's.

getActiveGenes()

Return a list of active gene objects.

getProtein()

Return the protein associated with this set of genes.

setProtein(*protein*)

Sets the protein associated with this set of genes.

setGeneInactive(*gid*)

Set a gene to be inactive.

setGeneActive(*gid*)

Set a gene to be inactive.

setAllGenesActive()

Activate all genes in association.

setAllGenesInactive()

Deactivates all genes in association.

isProteinActive()

This returns a boolean which indicates the result of evaluating the gene association.

If the result is positive then the protein is expressed and *True* is returned, otherwise if the expression evaluates to a value of 0 then the protein is not expressed and *False* is returned.

setTree(*tree*)

Add a GPR dictionary/tree representation to the GPR.

Parameters

tree – a dictionary representation of a GPR.

getTree()

Return the dictionary/tree representation of the GPR.

getTreeCopy()

Return a copy of the dictionary/tree representation of the GPR.

deleteGeneFromAssociation(*gid*)

Deletes a gene id from the gene association. *WARNING* this process is irreversible!!

Parameters

gid – a valid gene identifier (not label)

2.6.3 CBMPy: CBSolve module

PySCeS Constraint Based Modelling (<http://cbmpy.sourceforge.net>) Copyright (C) 2009-2024 Brett G. Olivier, VU University Amsterdam, Amsterdam, The Netherlands

This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program. If not, see <<http://www.gnu.org/licenses/>>

Author: Brett G. Olivier PhD Contact developers: <https://github.com/SystemsBioinformatics/cbmpy/issues> Last edit: \$Author: bgoli \$ (\$Id: CBModel.py 706 2020-03-23 21:31:49Z bgoli \$)

class cbmpy.CBSolve.LPproperties

Bases: `object`

LP property class.

property model

property id

property name

```
class cbmpy.CBSolve.LPbase(pid)
```

Bases: [LPproperties](#)

LP base class.

```
__init__(pid)
```

```
class cbmpy.CBSolve.LPcplex(pid)
```

Bases: [LPbase](#)

CPLEX interface.

```
__init__(pid)
```

```
class cbmpy.CBSolve.LPglpk(pid)
```

Bases: [LPbase](#)

GLPK interface.

```
__init__(pid)
```

2.6.4 CBMPy: CBRead module

PySCeS Constraint Based Modelling (<http://cbmpy.sourceforge.net>) Copyright (C) 2009-2024 Brett G. Olivier, VU University Amsterdam, Amsterdam, The Netherlands

This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program. If not, see <<http://www.gnu.org/licenses/>>

Author: Brett G. Olivier PhD Contact developers: <https://github.com/SystemsBioinformatics/cbmpy/issues> Last edit: \$Author: bgoli \$ (\$Id: CBRead.py 669 2019-02-18 22:58:19Z bgoli \$)

```
cbmpy.CBRead.loadModel(sbmlfile)
```

Loads any SBML model in COBRA, FAME (SBML2FBA), SBML3FBCv1, SBML3FBCv2 format.

Parameters

sbmlfile – an SBML model file

```
cbmpy.CBRead.readSBML3FBC(fname, work_dir=None, return_sbml_model=False,  
                           xoptions={'validate': False}, scan_notes_gpr=True)
```

Read in an SBML Level 3 file with FBC annotation where and return a CBM model object.

Parameters

- **fname** – is the filename
- **work_dir** – is the working directory
- **return_sbml_model** – deprecated and ignored please update code
- **xoptions** – special load options, enable with option=True except for nmatrix_type which has a type.
- **nogenes** – do not load/process genes
- **noannot** – do not load/process any annotations
- **validate** – validate model and display errors and warnings before loading
- **readcobra** – read the cobra annotation
- **read_model_string** – [default=False] read the model from a string (instead of a filename) containing an SBML document
- **nmatrix_type** – [default='normal'] define the type of stoichiometric matrix to be built
- **scan_notes_gpr** – [default=True] if the model is loaded and no genes are detected scan the <notes> field for GPR associations

```
cbmpy.CBRead.readCOBRASBML(fname, work_dir=None, return_sbml_model=False,  
                             delete_intermediate=False,  
                             fake_boundary_species_search=False, output_dir=None,  
                             skip_genes=False, scan_notes_gpr=True)
```

Read in a COBRA format SBML Level 2 file with FBA annotation where and return either a CBM model object or a (cbm_mod, sbml_mod) pair if return_sbml_model=True.

Parameters

- **fname** – is the filename
- **work_dir** – is the working directory
- **delete_intermediate** – [default=False] delete the intermediate SBML Level 3 FBC file
- **fake_boundary_species_search** – [default=False] after looking for the boundary_condition of a species search for overloaded id's <id>_b
- **output_dir** – [default=None] the directory to output the intermediate SBML L3 files (if generated) default to input directory
- **skip_genes** – [default=False] do not load GPR data

- **scan_notes_gpr** – [default=True] if the model is loaded and no genes are detected the scan the <notes> field for GPR associationa

`cbmpy.CBRead.readSBML2FBA(fname, work_dir=None, return_sbml_model=False, fake_boundary_species_search=False, scan_notes_gpr=True)`

Read in an SBML Level 2 file with FBA annotation where:

Parameters

- **fname** – is the filename
- **work_dir** – is the working directory if None then only fname is used
- **return_sbml_model** – [default=False] return a a (cbm_mod, sbml_mod) pair
- **fake_boundary_species_search** – [default=False] after looking for the boundary_condition of a species search for overloaded id's <id>_b
- **scan_notes_gpr** – [default=True] if the model is loaded and no genes are detected the scan the <notes> field for GPR associationa

`cbmpy.CBRead.readLPtoList(fname, work_dir)`

`cbmpy.CBRead.readSK_FVA(filename)`

Read Stevens FVA results (opt.fva) file and return a list of dictionaries.

`cbmpy.CBRead.readSK_vertexOld(fname, bigfile=False, fast_rational=False, nformat='%.14f', compresslevel=3)`

Reads in Stevens vertex analysis file and returns, even more optimized for large datasets than the original.

- a list of vertex vectors
- a list of ray vectors
- the basis of the lineality space as a list of vectors

all vectors in terms of the column space of N

`cbmpy.CBRead.readSK_vertex(fname, bigfile=True, fast_rational=False, nformat='%.14f', compression=None, hdf5file=None)`

Reads in Stevens vertex analysis file:

Parameters

- **fname** – the input filename (.all file that results from Stevens pipeline)
- **bigfile** – [default=True] this option is now always true and is left in for backwards compatability

- **fast_rational** – [default=False] by default off and uses SymPy for rational→float conversion, when on uses float decomposition with a slight (2th decimal) decrease in accuracy
- **nformat** – [default='%.14f'] the number format used in output files
- **compression** – [default=None] compression to be used in hdf5 files can be one of [None, 'lzf', 'gz?', 'gzip']
- **hdf5file** – [default=None] if None then generic filename '_vtx_.tmp.hdf5' is used otherwise <hdf5file>.hdf5

```
cbmpy.CBRead.readExcel97Model(xlname, write_sbml=True, sbml_level=3,
                             return_dictionaries=False)
```

Reads a model encoded as an Excel97 workbook and returns it as a CBMPy model object and SBML file. Note the workbook must be formatted exactly like those produced by `cbm.writeModelToExcel97()`. Note that reactions have to be defined in **both** the *reaction* and *network_react* sheets to be included in the model.

Parameters

- **xlpath** – the filename of the Excel workbook
- **return_model** – [default=True] construct and return the CBMPy model
- **write_sbml** – [default=True] write the SBML file to fname
- **return_dictionaries** – [default=False] return the dictionaries constructed when reading the Excel file (in place of the model)
- **sbml_level** – [default=3] write the SBML file as either SBML L2 FBA or SBML L3 FBC file.

2.6.5 CBMPy: CBWrite module

PySCeS Constraint Based Modelling (<http://cbmpy.sourceforge.net>) Copyright (C) 2009-2024 Brett G. Olivier, VU University Amsterdam, Amsterdam, The Netherlands

This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program. If not, see <<http://www.gnu.org/licenses/>>

Author: Brett G. Olivier PhD Contact developers: <https://github.com/SystemsBioinformatics/cbmpy/issues> Last edit: \$Author: bgoli \$ (\$Id: CBWrite.py 710 2020-04-27 14:22:34Z bgoli \$)

`cbmpy.CBWrite.saveModel(model, filename, compress=False)`

Saves the model to an SBML file using the latest SBML3 FBC version.

Parameters

- **model** – the CBMPy model
- **filename** – the filename to write

`cbmpy.CBWrite.writeSBML3FBC(fba, fname, directory=None, gpr_from_annot=False, add_groups=True, add_cbmpy_annot=True, add_cobra_annot=False, xoptions={'compress_bounds': True, 'fbc_version': 1, 'validate': False})`

Takes an FBA model object and writes it to file as SBML L3 FBC:

Parameters

- **fba** – an fba model object
- **fname** – the model will be written as XML to *fname*
- **directory** – [default=None] if defined it is prepended to *fname*
- **gpr_from_annot** – [default=True] if enabled will attempt to add the gene protein associations from the annotations
- **add_groups** – [default=True] add SBML3 groups (if supported by libSBML)
- **add_cbmpy_annot** – [default=True] add CBMPy KeyValueType annotation. Replaces <notes>
- **add_cobra_annot** – [default=True] add COBRA <notes> annotation
- **xoptions** – extended options
- **fbc_version** – [default=1] write SBML3FBC using version 1 (2013) or version 2 (2015)
- **validate** – [default=False] validate the output SBML file
- **compress_bounds** – [default=False] try compress output flux bound parameters
- **zip_model** – [default=False] compress the model using PKZIP encoding
- **return_model_string** – [default=False] return the SBML XML file as a string

`cbmpy.CBWrite.writeSBML3FBCV2(fba, fname, directory=None, gpr_from_annot=False, add_groups=True, add_cbmpy_annot=True, add_cobra_annot=False, validate=False, compress_bounds=False, zip_model=False, return_model_string=False)`

Takes an FBA model object and writes it to file as SBML L3 FBCv2 :

Parameters

- **fba** – an fba model object
- **fname** – the model will be written as XML to *fname*
- **directory** – [default=None] if defined it is prepended to *fname*
- **gpr_from_annot** – [default=False] if enabled will attempt to add the gene protein associations from the annotations
- **add_groups** – [default=True] add SBML3 groups (if supported by libSBML)
- **add_cbmpy_annot** – [default=True] add CBMPy KeyValueType annotation. Replaces <notes>
- **add_cobra_annot** – [default=False] add COBRA <notes> annotation
- **validate** – [default=False] validate the output SBML file
- **compress_bounds** – [default=True] try compress output flux bound parameters
- **zip_model** – [default=False] compress the model using ZIP encoding
- **return_model_string** – [default=False] return the SBML XML file as a string

```
cbmpy.CBWrite.writeSBML3FBCV3(fba, fname, directory=None, gpr_from_annot=False,  
                               add_groups=True, add_cbmpy_annot=True,  
                               add_cobra_annot=False, validate=False,  
                               compress_bounds=False, zip_model=False,  
                               return_model_string=False)
```

Takes an FBA model object and writes it to file as SBML L3 FBCv3 :

Parameters

- **fba** – an fba model object
- **fname** – the model will be written as XML to *fname*
- **directory** – [default=None] if defined it is prepended to *fname*
- **gpr_from_annot** – [default=False] if enabled will attempt to add the gene protein associations from the annotations
- **add_groups** – [default=True] add SBML3 groups (if supported by libSBML)

- **add_cbmpy_annot** – [default=True] add CBMPy KeyValueTypeData annotation. Replaces <notes>
- **add_cobra_annot** – [default=False] add COBRA <notes> annotation
- **validate** – [default=False] validate the output SBML file
- **compress_bounds** – [default=True] try compress output flux bound parameters
- **zip_model** – [default=False] compress the model using ZIP encoding
- **return_model_string** – [default=False] return the SBML XML file as a string

`cbmpy.CBWrite.writeCOBRASBML(fba, fname, directory=None)`

Takes an FBA model object and writes it to file as a COBRA compatible :

Parameters

- **fba** – an fba model object
- **fname** – the model will be written as XML to *fname*
- **directory** – [default=None] if defined it is prepended to *fname*

`cbmpy.CBWrite.writeSBML2FBA(fba, fname, directory=None, sbml_level_version=None)`

Takes an FBA model object and writes it to file as SBML L2 with FBA annotations. Note if you want to write BiGG/FAME style annotations then you must use *sbml_level_version=(2,1)*

Parameters

- **fba** – an fba model object
- **fname** – the model will be written as XML to *fname*
- **sbml_level_version** – [default=None] a tuple containing the SBML level and version e.g. (2,1)

`cbmpy.CBWrite.writeSensitivitiesToCSV(sensitivities, fname)`

Write out a sensitivity report using the objective sensitivities and bound sensitivity dictionaries created by e.g. `cplx_getSensitivities()`.

Parameters

- **sensitivity** – tuple containing
- **obj_sens** – dictionary of objective coefficient sensitivities (per flux)
- **rhs_sens** – dictionary of constraint rhs sensitivities (per constraint)
- **bound_sens** – dictionary of bound sensitivities (per flux)
- **fname** – output filename e.g. *fname.csv*

`cbmpy.CBWrite.WriteModelRaw(fba, work_dir=None)`

INFO: this method will be deprecated please update your scripts to use “writeModelRaw()”

`cbmpy.CBWrite.writeModelRaw(fba, work_dir=None)`

Writes a fba (actually just dumps it) to a text file.

Parameters

- **fba** – an instantiated FBAmodel instance
- **work_dir** – directory designated for output

`cbmpy.CBWrite.BuildLPFluxBounds(fba, use_rational=False)`

Build and return a csio that contains the flux bounds in LP format.

`cbmpy.CBWrite.BuildLPConstraints(fba, use_rational=False)`

Build and return a csio that contains constraint constructed from the StoichiometryLP object.

Parameters

- **fba** – an fba model object which has a stoichiometry
- **use_rational** – write rational number output [default=False]

`cbmpy.CBWrite.BuildLPUserConstraints(fba, use_rational=False)`

Build and return a csio that contains constraint constructed from the StoichiometryLP object.

Parameters

- **fba** – an fba model object which has a stoichiometry
- **use_rational** – write rational number output [default=False]

`cbmpy.CBWrite.BuildLPConstraintsRelaxed(fba)`

Build and return a csio that contains the constraints in LP format Relaxed refers to $dS/dt \geq 0$.

`cbmpy.CBWrite.BuildLPConstraintsStrict(fba, use_rational=False)`

Build and return a csio that contains the constraints in LP format Strict refers to $dS/dt = 0$.

`cbmpy.CBWrite.BuildLPConstraintsMath(fba, use_rational=False)`

Build and return a csio that contains the constraints in LP format Strict refers to $dS/dt \Rightarrow 0$ and $dS/dt \leq 0$.

`cbmpy.CBWrite.WriteModelLPold(fba, work_dir=None, multisymb=' ', lpt=True,
constraint_mode='strict', use_rational=False,
format='%s')`

INFO: this method will be deprecated please update your scripts to use “writeModelLPold()”

`cbmpy.CBWrite.writeModelLPold(fba, work_dir=None, multisymb=' ', lpt=True,
constraint_mode='strict', use_rational=False,
format='%s')`

Writes a fba as an LP/LPT.

Parameters

- **fba** – an instantiated FBAModel instance
- **work_dir** – directory designated for output
- **multisymb** – the multiplication symbol (default: <space>)
- **lpt** – the file format (default: True for lpt) or False for lp

```
cbmpy.CBWrite.WriteModelLP(fba, work_dir=None, fname=None, multisymb=' ',  
                             format='%s', use_rational=False, constraint_mode=None,  
                             quiet=False)
```

INFO: this method will be deprecated please update your scripts to use “writeModelLP()”

```
cbmpy.CBWrite.writeModelLP(fba, work_dir=None, fname=None, multisymb=' ',  
                             format='%s', use_rational=False, constraint_mode=None,  
                             quiet=False)
```

Writes an FBA object as an LP in CPLEX LP format.

Parameters

- **fba** – an instantiated FBAModel instance
- **work_dir** – directory designated for output
- **fname** – the file name [default=fba.getId()]
- **multisymb** – the multiplication symbol (default: <space>)
- **format** – the number format of the output
- **use_rational** – output rational numbers [default=False]
- **quiet** – [default=False] suppress information messages

```
cbmpy.CBWrite.BuildHformatFluxBounds(fba, infinity_replace=None, use_rational=False)
```

Build and return a csio that contains the flux bounds in H format.

Parameters

- **fba** – a PySCeS-CBM FBA object
- **infinity_replace** – [default=None] if defined this is the abs(value) of +-<infinity>

```
cbmpy.CBWrite.WriteModelHFormatFBA(fba, work_dir=None, use_rational=False,  
                                     fullLP=True, format='%s', infinity_replace=None)
```

INFO: this method will be deprecated please update your scripts to use “writeModelHFormatFBA2()”

```
cbmpy.CBWrite.writeModelHFormatFBA(fba, work_dir=None, use_rational=False,  
                                     fullLP=True, format='%s', infinity_replace=None)
```

Write an FBA-LP in polynomial H-Format file. This version has been replaced by *writeModelHFormatFBA2()* but is kept for backwards compatability.

Parameters

- **fba** – a PySCeS-CBM FBA object
- **Work_dir** – [default=None] the output directory
- **use_rational** – [default=false] use rational numbers in output (requires sympy)
- **fullLP** – [default=True] include the default objective function as a maximization target
- **format** – [default='%s'] the number format string
- **infinity_replace** – [default=None] if defined this is the abs(value) of $\pm<\text{infinity}>$

```
cbmpy.CBWrite.WriteModelHFormatFBA2(fba, fname=None, work_dir=None,  
                                     use_rational=False, fullLP=True, format='%s',  
                                     infinity_replace=None)
```

INFO: this method will be deprecated please update your scripts to use “*writeModelHFormatFBA2()*”

```
cbmpy.CBWrite.writeModelHFormatFBA2(fba, fname=None, work_dir=None,  
                                     use_rational=False, fullLP=True, format='%s',  
                                     infinity_replace=None)
```

Write an FBA-LP in polynomial H-Format file. This is an improved version of *WriteModelHFormatFBA()* which it replaces. Note that if a SymPy matrix is used as input then *use_rational* is automatically enabled.

Parameters

- **fba** – a PySCeS-CBM FBA object
- **fname** – [default=None] the output filename, *fba.getId()* if not defined
- **Work_dir** – [default=None] the output directory
- **use_rational** – [default=false] use rational numbers in output (requires sympy)
- **fullLP** – [default=True] include the default objective function as a maximization target
- **format** – [default='%s'] the number format string

- **infinity_replace** – [default=None] if defined this is the abs(value) of +-<infinity>

```
cbmpy.CBWrite.writeStoichiometricMatrix(fba, fname=None, work_dir=None,
                                         use_rational=False, fullLP=True,
                                         format='%s', infinity_replace=None)
```

Write an FBA-LP in polynomial H-Format file. This is an improved version of *WriteModel-HFormatFBA()* which it replaces but is kept for backwards compatability.

Parameters

- **fba** – a PySCeS-CBM FBA object
- **fname** – [default=None] the output filename, fba.getId() if not defined
- **Work_dir** – [default=None] the output directory
- **use_rational** – [default=false] use rational numbers in output (requires sympy)
- **fullLP** – [default=True] include the default objective function as a maximization target
- **format** – [default='%s'] the number format string
- **infinity_replace** – [default=None] if defined this is the abs(value) of +-<infinity>

```
cbmpy.CBWrite.writeListToLP(fname, obj=None, const=None, bnds=None,
                             work_dir=None, objtype='maximize')
```

```
cbmpy.CBWrite.writeMinDistanceLP(fname, fbas, work_dir=None, ignoreDistance=[],
                                  with_protein_cost=False, constraint_mode='strict',
                                  moma=False)
```

```
cbmpy.CBWrite.writeMinDistanceLP_absL1(fname, fbas, work_dir=None,
                                         ignoreDistance=[], bigM=500,
                                         with_protein_cost=False,
                                         constraint_mode='strict', moma=False)
```

```
cbmpy.CBWrite.writeMinDistanceLPwithCost(fname, fbas, work_dir=None,
                                           ignoreDistance=[],
                                           constraint_mode='strict')
```

For backwards compatability only.

```
cbmpy.CBWrite.writeOptimalSolution(fba, fname, Dir=None, separator=',',
                                   only_exchange=False)
```

This function writes the optimal solution to file.

Parameters

- **fba** – an instance of an PySCeSCBM model
- **fname** – the output filename
- **Dir** – [default=None] use current directory if not None
- **separator** – [default=','] the column separator
- **only_exchange** – [default=False] only output fluxes labelled as exchange reactions

```
cbmpy.CBWrite.writeModelInfoToFile(fba, fname, Dir=None, separator=',',  
                                   only_exchange=False, met_type='all')
```

This function writes a CBModel to file.

Parameters

- **fba** – an instance of an PySCeSCBM model
- **fname** – the output filename
- **Dir** – [default=None] use directory if not None
- **separator** – [default=','] the column separator
- **only_exchange** – [default=False] only output fluxes labelled as exchange reactions
- **type** – [default='all'] only output certain type of species: 'all','boundary' or 'variable'

```
cbmpy.CBWrite.writeReactionInfoToFile(fba, fname, Dir=None, separator=',',  
                                       only_exchange=False)
```

This function writes a CBModel to file.

Parameters

- **fba** – an instance of an PySCeSCBM model
- **fname** – the output filename
- **Dir** – [default=None] use directory if not None
- **separator** – [default=','] the column separator
- **only_exchange** – [default=False] only output fluxes labelled as exchange reactions

```
cbmpy.CBWrite.writeSpeciesInfoToFile(fba, fname, Dir=None, separator=',',  
                                      met_type='all')
```

This function writes a CBModel to file.

Parameters

- **fba** – an instance of an PySCeSCBM model

- **fname** – the output filename
- **Dir** – [default=None] use directory if not None
- **separator** – [default=','] the column separator
- **met_type** – [default='all'] only output certain type of species: 'all','boundary' or 'variable'

`cbmpy.CBWrite.printFBASolution(fba, include_all=False)`

Prints the FBA optimal solution to the screen.

Parameters

- **fba** – an FBA model object
- **include_all** – include all variables

`cbmpy.CBWrite.exportModel(fba, fname=None, fmt='lp', work_dir=None, use_rational='both')`

Export the FBA model in different formats:

Parameters

- **fba** – the FBA model
- **fname** – [default=None] the exported filename if None then `fba.getId()` is used
- **fmt** – [default='lp'] the export format can be one of: 'lp' (CPLEX), 'hformat' (Polyhedra), 'all' (both)
- **use_rational** – [default='both'] if *all* or *hformat* is specified should hformat files be written using rational math or not. The default *both* is the legacy behaviour and writes both.

`cbmpy.CBWrite.writeProteinCostToCSV(fba, fname)`

Writes the protein costs 'CBM_PEPTIDE_COST' annotation to a csv file.

Parameters

- **fba** – an instantiated FBA object
- **fname** – the exported file name

`cbmpy.CBWrite.WriteFVAtCSV(id, fva, names, Dir=None, fbaObj=None)`

INFO: this method will be deprecated please update your scripts to use "writeFVAtCSV()"

`cbmpy.CBWrite.writeFVAtCSV(fvadata, names, fname, Dir=None, fbaObj=None)`

Takes the results of a FluxVariabilityAnalysis method and writes it to a nice csv file. Note this method replaces the `glpk/cplx_WriteFVAtCSV` methods.

Parameters

- **fva**data – FluxVariabilityAnalysis() OUTPUT_ARRAY
- **names** – FluxVariabilityAnalysis() OUTPUT_NAMES
- **fname** – filename_base for the CSV output
- **Dir** – [default=None] if set the output directory for the csv files
- **fbaObj** – [default=None] if supplied adds extra model information into the output tables

```
cbmpy.CBWrite.WriteFVAdata(fva, names, fname, work_dir=None, roundec=None,  
                             scale_min=False, appendfile=False, info=None)
```

INFO: this method will be deprecated please update your scripts to use “writeFVAdata()”

```
cbmpy.CBWrite.writeFVAdata(fva, names, fname, work_dir=None, roundec=None,  
                             scale_min=False, appendfile=False, info=None)
```

Takes the results of a FluxVariabilityAnalysis method and writes it to a nice csv file. Note this method replaces the glpk/cplx_WriteFVAtoCSV methods. Data is output as a csv file with columns: FluxName, FVA_MIN, FVA_MAX, OPT_VAL, SPAN

Parameters

- **fva**data – FluxVariabilityAnalysis() FVA OUTPUT_ARRAY
- **names** – FluxVariabilityAnalysis() FVA OUTPUT_NAMES
- **fname** – filename_base for the CSV output
- **work_dir** – [default=None] if set the output directory for the csv files
- **roundec** – [default=None] an integer indicating at which decimal to round off output. Default is no rounding.
- **scale_min** – [default=False] normalise each flux such that that FVA_MIN = 0.0
- **appendfile** – [default=False] instead of opening a new file try and append the data
- **info** – [default=None] a string added to the results as an extra column, useful with *appendfile*

```
cbmpy.CBWrite.writeSolutions(fname, sols=[], sep=',', extra_output=None, fba=None)
```

Write 2 or more solutions where a solution is a dictionary of flux:value pairs:

Parameters

- **fname** – the export filename
- **sols** – a list of dictionaries containing flux:value pairs (e.g. output by `cmod.getReactionValues()`)
- **sep** – [default=','] the column separator

- **extra_output** – [default=None] add detailed information to output e.g. reaction names by giving a CBModel object as an argument to *extra_output*.
- **fba** – an fba model that can be used for extra_output

`cbmpy.CBWrite.generateBGID(num, prefix)`

Create a BGID generator, which is <prefix><num> where prefix is two letters num is padded to 6 figures.

Parameters

- **num** – the starting number
- **prefix** – the two letter prefix

`cbmpy.CBWrite.convertFloatToExcel(num, roundoff)`

Converts a float to Excel compatible “number”.

Parameters

- **num** – a number
- **roundoff** – the number of roundoff digits for round()

`cbmpy.CBWrite.convertExcelToFloat(num)`

Converts an Excel “number” to a float.

Parameters

num – a number

`cbmpy.CBWrite.writeModelToExcel97(fba, filename, roundoff=6)`

Exports the model as an Excel 97 spreadsheet.

Parameters

- **fba** – a CBMPy model instance
- **filename** – the filename of the workbook
- **roundoff** – [default=6] the number of digits to round off to

`cbmpy.CBWrite.writeModelToCOMBINEarchive(mod, fname=None, directory=None, sbmlname=None, withExcel=True, vc_given='CBMPy', vc_family='Software', vc_email='None', vc_org='cbmpy.sourceforge.net', add_cbmpy_annot=True, add_cobra_annot=True)`

Write a model in SBML and Excel format to a COMBINE archive using the following information:

Parameters

- **mod** – a model object
- **fname** – the output base filename, archive will be <fname>.zip
- **directory** – [default=None] created the combine archive ‘directory’
- **sbmlname** – [default=None] If *sbmlname* is defined then SBML file is <sbmlname>.xml otherwise sbml will be <fname>.xml.
- **withExcel** – [default=True] include a human readable Excel spreadsheet version of the model
- **vc_given** – [default='CBMPy'] first name
- **vc_family** – [default='Software'] family name
- **vc_email** – [default=None] email
- **vc_org** – [default=None] organisation
- **add_cbmpy_annot** – [default=True] add CBMPy KeyValueType annotation. Replaces <notes>
- **add_cobra_annot** – [default=True] add COBRA <notes> annotation

2.6.6 CBMPy: CBConfig module

PySCeS Constraint Based Modelling (<http://cbmpy.sourceforge.net>) Copyright (C) 2009-2024 Brett G. Olivier, VU University Amsterdam, Amsterdam, The Netherlands

This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program. If not, see <<http://www.gnu.org/licenses/>>

Author: Brett G. Olivier PhD Contact developers: <https://github.com/SystemsBioinformatics/cbmpy/issues> Last edit: \$Author: bgoli \$ (\$Id: CBConfig.py 711 2020-04-27 14:22:34Z bgoli \$)

`cbmpy.CBConfig.current_version()`

Return the current CBMPy version as a string.

`cbmpy.CBConfig.current_version_tuple()`

Return the current CBMPy version as a tuple (x, y, z)

2.6.7 CBMPy: CBCommon module

PySCeS Constraint Based Modelling (<http://cbmpy.sourceforge.net>) Copyright (C) 2009-2024 Brett G. Olivier, VU University Amsterdam, Amsterdam, The Netherlands

This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program. If not, see <<http://www.gnu.org/licenses/>>

Author: Brett G. Olivier PhD Contact developers: <https://github.com/SystemsBioinformatics/cbmpy/issues> Last edit: \$Author: bgoli \$ (\$Id: CBCommon.py 710 2020-04-27 14:22:34Z bgoli \$)

`cbmpy.CBCommon.parseGeneAssociation(gs)`

Parse a COBRA style gene association into a nested list.

Parameters

gs (*str*) – A string containing a gene association.

Returns

list – A nested list representation of the gene association.

`cbmpy.CBCommon.checkChemFormula(cf, quiet=False)`

Check if a string conforms to a Chemical Formula.

Please see the SBML Level 3 specification and http://wikipedia.org/wiki/Hill_system for more information.

Parameters

- **cf** (*str*) – A string that contains a formula to check.
- **quiet** (*bool*, *optional*) – Do not print error messages. Default is False.

Returns

bool – True if valid chemical formula, False otherwise.

`cbmpy.CBCommon.extractGeneIdsFromString(g, return_clean_gpr=False)`

Extract and return a list of gene names from a gene association string formulation.

Parameters

- **g** (*str*) – A COBRA style gene association string.

- **return_clean_gpr** (*bool*, *optional*) – In addition to the list returns the “cleaned” GPR string. Default is False.

Returns

list or tuple – A list of gene names, or a tuple of (names, cleaned_gpr) if return_clean_gpr=True.

`cbmpy.CBCommon.createAssociationDictFromNode(node, out, model, useweakref=True, cnt=0)`

Convert a GPR string ‘((g1 and g2) or g3)’ to a dictionary via a Python AST.

Parameters

- **node** (*ast.AST*) – A Python AST node (e.g. body).
- **out** (*dict*) – A GPR dictionary (modified in place).
- **model** (*Fbase*) – A CBMPy model instance.
- **useweakref** (*bool*, *optional*) – Use a weakref as the gene object or alternatively the label. Default is True.
- **cnt** (*int*, *optional*) – Counter for unique key generation. Default is 0.

Returns

None

`cbmpy.CBCommon.func_getAssociationStrFromGprDict(gprd, out, parent='')`

Get an old school GPR association string from a CBMPy gprDict.

Parameters

- **gprd** (*dict*) – The GPR dictionary.
- **out** (*str*) – The output string accumulator.
- **parent** (*str*, *optional*) – The string representing the current node’s parent relationship, used for recursion. Default is ‘’.

Returns

str – The GPR association string.

`cbmpy.CBCommon.getGPRasDictFromString(node, out)`

Convert a GPR string ‘((g1 and g2) or g3)’ to a gprDict.

Parameters

- **node** (*ast.AST*) – A Python AST node (e.g. `ast.parse(gprstring).body[0]`).
- **out** (*dict*) – A new dictionary that will be created in place.

Returns

dict – The populated gpr dictionary.

class cbmpy.CBCommon.ComboGen

Bases: `object`

Generate sets of unique combinations.

This class generates and manages combinations of elements.

combo

List of stored combinations.

Type

`list`

combo_int

Integer representation of combinations.

Type

`list`

combo_int = `None`

__init__()

Initialize the ComboGen instance.

Returns

None

combo = `None`

addCombination(*data*)

Add a combination to the internal list.

Parameters

data (`list`) – A list of elements to join into a string and add to combo.

Returns

None

uniqueCombinations(*data*, *number*, *temp*=*None*)

Generate unique combinations of the given data.

Parameters

- **data** (`list`) – A list of elements to combine.
- **number** (`int`) – The number of elements per combination.
- **temp** (`list`, *optional*) – Temporary list used internally. Default is `None`.

Returns*None***numberifyComb2Int()**

Convert string combinations to integer tuples.

Returns*None*

`cbmpy.CBCommon.processSpeciesChargeChemFormulaAnnot(s, getFromName=False, overwriteChemFormula=False, overwriteCharge=False)`

Disambiguate the chemical formula and charge from species annotations.

Parameters

- **s** (*object*) – A species object.
- **getFromName** (*bool*, *optional*) – Whether to try stripping the chemical formula from the name (old COBRA style). Default is False.
- **overwriteChemFormula** (*bool*, *optional*) – Whether to overwrite existing chemical formula. Default is False.
- **overwriteCharge** (*bool*, *optional*) – Whether to overwrite existing charge. Default is False.

Returns*None*

`cbmpy.CBCommon.binHash(keys, d)`

Create a hash tuple for binning by checking keys against a dictionary.

Parameters

- **keys** (*list*) – A list of keys to check.
- **d** (*dict*) – A dictionary.

Returns

tuple – A tuple of values from d for keys present, or True for missing keys.

`cbmpy.CBCommon.fixId(s, replace=None)`

Fix a string to be a valid C style variable name.

The first letter must be an underscore or letter, and the rest should be alphanumeric or underscore.

Parameters

- **s** (*str*) – The string to test.

- **replace** (*str* or *None*, *optional*) – Character to use for illegal characters, or None to remove them. Default is None.

Returns

str – A fixed string conforming to C variable naming rules.

`cbmpy.CBCommon.checkId(s)`

Check if a string conforms to a C variable naming convention.

Parameters

s (*str*) – A string.

Returns

bool – True if the string is a valid C variable name, False otherwise.

class `cbmpy.CBCommon.StructMatrix(array, ridx, cidx, row=None, col=None)`

Bases: `object`

Store structural matrix information and generate row/col labels.

This class stores structural matrix data and uses row/column index permutations to generate labels from a source.

array

The structural matrix array.

Type

`ndarray`

ridx

Row indexes.

Type

`list`

cidx

Column indexes.

Type

`list`

row

Row labels.

Type

`list`

col

Column labels.

Type

`list`

shape

Shape of the matrix.

Type

`tuple`

__init__(*array*, *ridx*, *cidx*, *row=None*, *col=None*)

Instantiate a StructMatrix with array and matching row/col index arrays.

Parameters

- **array** (*ndarray*) – The structural matrix array.
- **ridx** (*list*) – Array of row indexes.
- **cidx** (*list*) – Array of column indexes.
- **row** (*list* or *None*, *optional*) – Optional array of row labels. Default is *None*.
- **col** (*list* or *None*, *optional*) – Optional array of column labels. Default is *None*.

Returns

None

array = *None*

ridx = *None*

cidx = *None*

row = *None*

col = *None*

shape = *None*

getRowsByIdx(**args*)

Return the rows referenced by index.

Parameters

***args** (*int*) – Indices of rows to retrieve (e.g. 1, 3, 5).

Returns

ndarray – A subset of rows from the array.

getColsByIdx(**args*)

Return the columns referenced by index.

Parameters

***args** (*int*) – Indices of columns to retrieve (e.g. 1, 3, 5).

Returns

ndarray – A subset of columns from the array.

setRow(*src*)

Map row labels to ridx using a source label array.

Assuming that the row index array is a permutation of a source label array, this method maps the row labels to ridx and creates self.row (row label list).

Parameters

src (*list*) – A source array of row labels.

Returns

None

setCol(*src*)

Map column labels to cidx using a source label array.

Assuming that the col index array is a permutation of a source label array, this method maps the row labels to cidx and creates self.col (col label list).

Parameters

src (*list*) – A source array of column labels.

Returns

None

getRowsByName(*args)

Return the rows referenced by label.

Parameters

***args** (*str*) – Row labels to retrieve (e.g. 's', 'x', 'd').

Returns

ndarray or None – A subset of rows from the array, or None if row labels are not set.

getColsByName(*args)

Return the columns referenced by label.

Parameters

***args** (*str*) – Column labels to retrieve (e.g. 's', 'x', 'd').

Returns

ndarray or None – A subset of columns from the array, or None if column labels are not set.

getLabels(*axis*='all')

Return the matrix labels.

Parameters

axis (*str*, *optional*) – ‘row’/‘col’/‘all’ to specify which labels to return. Default is ‘all’.

Returns

list or tuple – A single label list or a tuple of (row_labels, col_labels).

getIndexes(*axis='all'*)

Return the matrix indexes.

Parameters

axis (*str*, *optional*) – ‘row’/‘col’/‘all’ to specify which indexes to return. Default is ‘all’.

Returns

list or tuple – A single index list or a tuple of (row_indexes, col_indexes).

getByIdx(*row, col*)

Return the array value at the given row and column index.

Parameters

- **row** (*int*) – A row index.
- **col** (*int*) – A column index.

Returns

float – The value at self.array[row, col].

getByName(*row, col*)

Return the array value at the given row and column name.

Parameters

- **row** (*str*) – A row name.
- **col** (*str*) – A column name.

Returns

float – The value at self.array[row, col].

setByIdx(*row, col, val*)

Set the array value at the given row and column index.

Parameters

- **row** (*int*) – A row index.
- **col** (*int*) – A column index.
- **val** (*float*) – The value to set.

Returns

None

setByName(*row*, *col*, *val*)

Set the array value at the given row and column name.

Parameters

- **row** (*str*) – A row name.
- **col** (*str*) – A column name.
- **val** (*float*) – The value to set.

Returns

None

class cbmpy.CBCommon.**StructMatrixLP**(*array*, *ridx*, *cidx*, *row=None*, *col=None*,
rhs=None, *operators=None*)

Bases: *StructMatrix*

Adds LP-friendly attributes to StructMatrix.

This class extends StructMatrix to add LP (linear programming) support including RHS values, constraint operators, and stoichiometric constraints.

RHS

Right-hand side values for constraints.

Type

ndarray

operators

List of constraint senses ('G', 'L', 'E').

Type

list

__array_type__

Type of the underlying array.

Type

type

__init__(*array*, *ridx*, *cidx*, *row=None*, *col=None*, *rhs=None*, *operators=None*)

Initialize a StructMatrixLP with LP-friendly attributes.

Object that holds an LP (linear programming) model with stoichiometric constraints, operators, RHS values, and names.

Parameters

- **array** (*ndarray*) – Stoichiometric matrix (linear constraints).
- **ridx** (*list*) – A list of indexes typically range(shape[0]).

- **cidx** (*list*) – A list of indexes typically range(shape[1]).
- **row** (*list or None, optional*) – A list of row names. Default is None.
- **col** (*list or None, optional*) – A list of col names. Default is None.
- **rhs** (*array-like or None, optional*) – A list of RHS values (defaults to 0). Default is None.
- **operators** (*list or None, optional*) – A list of constraint senses (defaults to 'E'). Default is None.

Returns

None

RHS = None

operators = None

getCopy(*attr_str, deep=False*)

Return a copy of the specified attribute.

Uses the copy module `copy.copy` or `copy.deepcopy` depending on the deep flag.

Parameters

- **attr_str** (*str*) – A string of the attribute name (e.g. 'row', 'col').
- **deep** (*bool, optional*) – If True, uses `deepcopy`; otherwise uses `copy`. Default is False.

Returns

ndarray or list – A copy of the attribute.

getLandRHS()

Return the stoichiometric matrix augmented with RHS column.

Returns

ndarray – A numpy array of [self.array, RHS].

setRHSbyName(*name, value*)

Set the RHS value by row name.

Parameters

- **name** (*str*) – A row name.
- **value** (*float*) – The RHS value to set.

Returns

None

setRHSbyIdx(*idx*, *val*)

Set the RHS value by row index.

Parameters

- **idx** (*int*) – A row index.
- **val** (*float*) – The RHS value to set.

Returns

None

setOperatorbyName(*name*, *value*)

Set the constraint operator by row name.

Parameters

- **name** (*str*) – A row name.
- **value** (*str*) – A constraint operator ('G', 'L', 'E').

Returns

None

setOperatorbyIdx(*idx*, *value*)

Set the constraint operator by row index.

Parameters

- **idx** (*int*) – A row index.
- **value** (*str*) – A constraint operator ('G', 'L', 'E').

Returns

None

getRHSbyName(*name*)

Get the RHS value by row name.

Parameters

name (*str*) – A row name.

Returns

float – The RHS value for the given row.

getRHSbyIdx(*idx*)

Get the RHS value by row index.

Parameters

idx (*int*) – A row index.

Returns

float – The RHS value for the given index.

getOperatorbyName(*name*)

Get the constraint operator by row name.

Parameters

name (*str*) – A row name.

Returns

str – The operator ('G', 'L', or 'E') for the given row.

getOperatorbyIdx(*idx*)

Get the constraint operator by row index.

Parameters

idx (*int*) – A row index.

Returns

str – The operator ('G', 'L', or 'E') for the given index.

class cbmpy.CBCommon.MIRIAMannotation

Bases: *object*

MIRIAM annotations for Biological Qualifiers.

This class handles MIRIAM (Mapping Identification Resource) annotations, which provide standardized identifiers for biological entities.

MIRIAM

MIRIAM resource identifiers.

Type

dict

MIDS

MIRIAM keys as tuple.

Type

tuple

MIDS1c

Lowercase MIRIAM keys as tuple.

Type

tuple

QUALIFIERS

Biomodels qualifier names.

Type

tuple

isA

‘is’ qualifier entries.

Type

[list](#) or None

isEncodedBy

‘isEncodedBy’ qualifier entries.

Type

[list](#) or None

encodes

‘encodes’ qualifier entries.

Type

[list](#) or None

hasPart

‘hasPart’ qualifier entries.

Type

[list](#) or None

hasProperty

‘hasProperty’ qualifier entries.

Type

[list](#) or None

hasTaxon

‘hasTaxon’ qualifier entries.

Type

[list](#) or None

hasVersion

‘hasVersion’ qualifier entries.

Type

[list](#) or None

isDescribedBy

‘isDescribedBy’ qualifier entries.

Type

[list](#) or None

isHomologTo

‘isHomologTo’ qualifier entries.

Type*list* or None**isPartOf**

‘isPartOf’ qualifier entries.

Type*list* or None**isPropertyOf**

‘isPropertyOf’ qualifier entries.

Type*list* or None**isVersionOf**

‘isVersionOf’ qualifier entries.

Type*list* or None**occursIn**

‘occursIn’ qualifier entries.

Type*list* or None**isDerivedFrom**

‘isDerivedFrom’ qualifier entries.

Type*list* or None**isA = None****isEncodedBy = None****encodes = None****hasPart = None****hasProperty = None****hasTaxon = None****hasVersion = None****isDescribedBy = None****isHomologTo = None**

isPartOf = None

isPropertyOf = None

isVersionOf = None

occursIn = None

isDerivedFrom = None

__init__()

Initialize the MIRIAMannotation instance.

Returns

None

MIRIAM = None

MIDS = None

MIDS1c = None

QUALIFIERS = None

addIDorgURI(*qual, uri*)

Add a URI directly into a qualifier collection.

Parameters

- **qual** (*str*) – A Biomodels biological qualifier (e.g. ‘is’, ‘isEncodedBy’).
- **uri** (*str*) – The complete identifiers.org URI (e.g. ‘<http://identifiers.org/chebi/CHEBI:58088>’).

Returns

None

checkEntityPattern(*entity*)

Compile the regex pattern for a MIRIAM resource entity.

For an entity key, this compiles the pattern to a regex if necessary.

Parameters

entity (*str*) – A MIRIAM resource entity.

Returns

bool – True if the pattern is valid/compiled, False otherwise.

checkId(*entity, mid*)

Check that a MIRIAM entity ID is valid.

Parameters

- **entity** (*str*) – A MIRIAM resource entity (e.g. ‘CHEBI’).
- **mid** (*str*) – The entity ID (e.g. ‘CHEBI:17158’).

Returns

bool – True if the ID matches the entity pattern, False otherwise.

checkEntity(*entity*)

Check if a MIRIAM resource entity name is valid.

This checks a MIRIAM resource name (e.g., ‘chEBI’). The test is case insensitive and automatically corrects wrongly capitalized entities. If the entity is not recognized, a list of possible candidates based on the first letters of the input is displayed.

Parameters

entity (*str*) – A MIRIAM resource entity (e.g. ‘ChEBI’).

Returns

str or None – The normalized entity name if valid, None otherwise.

addMIRIAMannotation(*qual, entity, mid*)

Add a qualified MIRIAM annotation.

Parameters

- **qual** (*str*) – A Biomodels biological qualifier (e.g. ‘is’, ‘isEncodedBy’).
- **entity** (*str*) – A MIRIAM resource entity (e.g. ‘ChEBI’).
- **mid** (*str*) – The entity ID (e.g. ‘CHEBI:17158’).

Returns

None

deleteMIRIAMannotation(*qual, entity, mid*)

Delete a qualified MIRIAM annotation.

Parameters

- **qual** (*str*) – A Biomodels biological qualifier (e.g. ‘is’, ‘isEncodedBy’).
- **entity** (*str*) – A MIRIAM resource entity (e.g. ‘ChEBI’).
- **mid** (*str*) – The entity ID (e.g. ‘CHEBI:17158’).

Returns

None

getMIRIAMUrisForQualifier(*qual*)

Return all URLs associated with a qualifier.

Parameters

qual (*str*) – The qualifier (e.g. ‘is’ or ‘isEncoded’).

Returns

tuple – A tuple of URLs for the given qualifier, or () if not found.

getAndViewUrisForQualifier(qual)

Retrieve all URLs associated with a qualifier and open them in browser tabs.

Parameters

qual (*str*) – The qualifier (e.g. ‘is’ or ‘isEncoded’).

Returns

None

getAllMIRIAMUris()

Return a dictionary of qualifiers that contain ID.org URLs.

Returns

dict – A dictionary mapping qualifier names to tuples of their associated ID.org URLs.

viewURL(url)

Open a URL in a new browser tab.

Parameters

url (*str*) – The URL to open.

Returns

None

2.6.8 CBMPy: CBTools module

PySCeS Constraint Based Modelling (<http://cbmpy.sourceforge.net>) Copyright (C) 2009-2024 Brett G. Olivier, VU University Amsterdam, Amsterdam, The Netherlands

This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program. If not, see <<http://www.gnu.org/licenses/>>

Author: Brett G. Olivier PhD Contact developers: <https://github.com/SystemsBioinformatics/cbmpy/issues> Last edit: \$Author: bgoli \$ (\$Id: CBTools.py 710 2020-04-27 14:22:34Z bgoli \$)

`cbmpy.CBTools.createTempFileName()`

Create a temporary filename based on the current timestamp.

Returns

str – A string representing a temporary filename.

`cbmpy.CBTools.storeObj(obj, filename, compress=False)`

Store a Python object as a serialized binary object.

Stores a Python *obj* as a serialized binary object in *filename.dat*.

Parameters

- **obj** (*object*) – a Python object
- **filename** (*str*) – the base filename (will append .dat if not present)
- **compress** (*bool, optional*) – use gzip compression (not implemented) [default=False]

Returns

None – writes object to disk

`cbmpy.CBTools.loadObj(filename)`

Load a serialized Python pickle from disk.

Loads a serialized Python pickle from *filename.dat* and returns the Python object.

Parameters

filename (*str*) – the base filename (will append .dat if not present)

Returns

object – the loaded Python object

`cbmpy.CBTools.deSerialize(s)`

Deserialize a serialized object from a string.

Deserializes a serialized Python object that was previously stored in a string.

Parameters

s (*str*) – a string containing a serialized object

Returns

object – the deserialized Python object

`cbmpy.CBTools.deSerializeFromDisk(filename)`

Deserialize a Python pickle from a file on disk.

Loads a serialized Python pickle from *filename* and returns the Python object.

Parameters

filename (*str*) – the path to the serialized file

Returns

object – the deserialized Python object

`cbmpy.CBTools.addStoichToFBAModel(fm)`

Build stoichiometry for an FBA model.

Parameters

fm (*FBA model*) – an FBA model object

Note

This method has been refactored into the model class. Use `cmod.buildStoichMatrix()` instead.

Returns

None – calls `fm.buildStoichMatrix()`

`cbmpy.CBTools.addSinkReaction(fbam, species, lb=0.0, ub=1000.0)`

Add a sink reaction that consumes a model species.

Adds a sink reaction that consumes a model *species* ($X \rightarrow \text{sink}$).

Parameters

- **fbam** (*FBA model*) – an FBA model object
- **species** (*str*) – a valid species name
- **lb** (*float, optional*) – lower flux bound [default=0.0]
- **ub** (*float, optional*) – upper flux bound [default=1000.0]

Returns

None – adds the sink reaction to the model

`cbmpy.CBTools.addSourceReaction(fbam, species, lb=0.0, ub=1000.0)`

Add a source reaction that produces a model species.

Adds a source reaction that produces a model *species* (source $\rightarrow X$).

Parameters

- **fbam** (*FBA model*) – an FBA model object
- **species** (*str*) – a valid species name
- **lb** (*float, optional*) – lower flux bound [default=0.0]
- **ub** (*float, optional*) – upper flux bound [default=1000.0]

Note

Reversibility is determined by the lower bound (default 0 = irreversible). If negative, the reaction is reversible.

Returns

None – adds the source reaction to the model

`cbmpy.CBTools.findDeadEndMetabolites(fbam)`

Find dead-end metabolites (those connected to a single reaction).

Finds metabolites that are only involved in a single reaction (orphaned metabolites).

Parameters

fbam (*FBA model*) – an FBA model instance

Returns

list – a list of tuples (metabolite_id, reaction_id) for dead-end metabolites

`cbmpy.CBTools.findDeadEndReactions(fbam)`

Find dead-end reactions (those with a single substrate/product).

Finds reactions that have only a single metabolite (orphaned reactions).

Parameters

fbam (*FBA model*) – an FBA model instance

Returns

list – a list of tuples (metabolite_id, reaction_id) for dead-end reactions

`cbmpy.CBTools.setSpeciesPropertiesFromAnnotations(fbam, overwriteCharge=False,
overwriteChemFormula=False)`

Set model species properties from annotations.

With default options, this will only replace missing data. For ChemicalFormula this is easy to detect, however charge may have an “unknown value” of 0. Setting the optional values to True will replace any existing value with any valid annotation.

Parameters

- **fbam** (*FBA model*) – an FBA model instance
- **overwriteChemFormula** (*bool*, *optional*) – whether to overwrite existing chemical formula [default=False]
- **overwriteCharge** (*bool*, *optional*) – whether to overwrite existing charge [default=False]

Returns

None – modifies species properties in place

`cbmpy.CBTools.fixReversibility(fbam, auto_correct=False)`

Fix reaction reversibility by setting appropriate flux bounds.

Set flux bound lower bounds from reactions' reversibility information.

Parameters

- **fbam** (*FBA model*) – an FBA model instance
- **auto_correct** (*bool, optional*) – if True, automatically sets lower bound to zero if required. Otherwise, prints a warning if inconsistency is found [default=False]

Returns

None – modifies model in place

`cbmpy.CBTools.splitReversibleReactions(fba, selected_reactions=None)`

Split reversible reactions into forward and reverse reactions.

Splits a set of reversible reactions into irreversible forward and reverse reactions (e.g., R1: A = B becomes R1f: A -> B and R1r: B -> A).

Parameters

- **fba** (*CBMPy model*) – an instantiated CBMPy model object
- **selected_reactions** (*list, optional*) – if provided, only split these reactions; otherwise splits all reversible reactions [default=None]

Returns

CBMPy model – a clone of the original model with reversible reactions split

`cbmpy.CBTools.splitSingleReversibleReaction(fba, rid, fwd_id=None, rev_id=None)`

Split a single reversible reaction into two irreversible reactions.

Splits a reversible reaction (A = B) into two irreversible reactions: - Forward: A -> B - Reverse: B -> A

The original reversible reaction and its bounds are deleted from the model.

Parameters

- **fba** (*CBMPy model*) – an instantiated CBMPy model object
- **rid** (*str*) – a valid reaction ID
- **fwd_id** (*str, optional*) – the new forward reaction ID (defaults to rid + '_fwd') [default=None]
- **rev_id** (*str, optional*) – the new reverse reaction ID (defaults to rid + '_rev') [default=None]

Returns

tuple – (original_reaction, lower_bound, upper_bound, equality_bound)

`cbmpy.CBTools.exportLabelledArray(arr, fname, names=None, sep=',', fmt='%f')`

Export a 2D array to a file with optional row labels.

Write a 2D array-like object to file with optional row labels and column separator.

Parameters

- **arr** (*array-like*) – a 2D array-like object
- **fname** (*str*) – the output filename
- **names** (*list, optional*) – the list of row names [default=None]
- **sep** (*str, optional*) – the column separator character [default=',']
- **fmt** (*str, optional*) – the output number format [default='%f']

Returns

None – writes array to file

`cbmpy.CBTools.exportLabelledArrayWithHeader(arr, fname, names=None, header=None, sep=',', fmt='%f')`

Export an array to file with row names and header.

Export a 2D array to file with row labels, column header, and optional append mode.

Parameters

- **arr** (*array-like*) – a 2D array-like object
- **fname** (*str*) – the output filename
- **names** (*list, optional*) – the list of row names [default=None]
- **header** (*list, optional*) – the list of column names [default=None]
- **sep** (*str, optional*) – the column separator character [default=',']
- **fmt** (*str, optional*) – the output number format [default='%f']

Returns

None – writes array to file

`cbmpy.CBTools.exportLabelledLinkedList(arr, fname, names=None, sep=',', fmt='%s', appendlist=False)`

Export a 2D linked list to file with optional row labels.

Write a 2D linked list `[[...],[...],[...]]` and optionally a list of row labels to file.

Parameters

- **arr** (*list*) – a 2D linked list (list of lists)

- **fname** (*str*) – the output filename
- **names** (*list*, *optional*) – the list of row names [default=None]
- **sep** (*str*, *optional*) – the column separator character [default=',']
- **fmt** (*str*, *optional*) – the output format string [default='%s']
- **appendlist** (*bool*, *optional*) – if True, append to file; otherwise create a new file [default=False]

Returns

None – writes array to file

`cbmpy.CBTools.exportLabelledArrayWithHeader2CSV(arr, fname, names=None, header=None)`

Export an array with row names and header to a CSV file.

Parameters

- **arr** (*array-like*) – a 2D array-like object
- **fname** (*str*) – the output filename (will append .csv)
- **names** (*list*, *optional*) – the list of row names [default=None]
- **header** (*list*, *optional*) – the list of column names [default=None]

Returns

None – creates a CSV file

`cbmpy.CBTools.exportLabelledArray2CSV(arr, fname, names=None)`

Export an array with row names to a CSV file.

Parameters

- **arr** (*array-like*) – a 2D array-like object
- **fname** (*str*) – the output filename (will append .csv)
- **names** (*list*, *optional*) – the list of row names [default=None]

Returns

None – creates a CSV file

`cbmpy.CBTools.exportArray2CSV(arr, fname)`

Export an array to a CSV file.

Parameters

- **arr** (*array-like*) – a 2D array-like object
- **fname** (*str*) – the output filename (will append .csv)

Returns

None – creates a CSV file

`cbmpy.CBTools.exportLabelledArrayWithHeader2TXT(arr, fname, names=None, header=None)`

Export an array with row names and header to a text file.

Parameters

- **arr** (*array-like*) – a 2D array-like object
- **fname** (*str*) – the output filename (will append .txt)
- **names** (*list*) – the list of row names
- **header** (*list*) – the list of column names

Returns

None – creates a text file

`cbmpy.CBTools.exportLabelledArray2TXT(arr, fname, names=None)`

Export an array with row names to a text file.

Parameters

- **arr** (*array-like*) – a 2D array-like object
- **fname** (*str*) – the output filename (will append .txt)
- **names** (*list, optional*) – the list of row names [default=None]

Returns

None – creates a text file

`cbmpy.CBTools.exportArray2TXT(arr, fname)`

Export an array to a text file.

Parameters

- **arr** (*array-like*) – a 2D array-like object
- **fname** (*str*) – the output filename (will append .txt)

Returns

None – creates a text file

`cbmpy.CBTools.stringReplace(fbamod, old, new, target)`

Replace strings in model components.

Replace occurrences of *old* with *new* in model identifiers. Target can be:

- ‘species’
- ‘reactions’

- ‘constraints’
- ‘objectives’
- ‘all’ (applies to all of the above)

Parameters

- **fbamod** (*FBA model*) – the model to modify
- **old** (*str*) – the string to replace
- **new** (*str*) – the replacement string
- **target** (*str*) – where to apply the replacement: ‘species’, ‘reactions’, ‘constraints’, ‘objectives’, or ‘all’

Returns

FBA model – the modified model (fbamod)

`cbmpy.CBTools.getBoundsDict(fbamod, substring=None)`

Get a dictionary of reactions and their bounds.

Returns a dictionary mapping reaction names to their lower bound, upper bound, and equality constraints.

Parameters

- **fbamod** (*FBA model*) – an FBA model instance
- **substring** (*str, optional*) – filter reactions by ID substring [default=None]

Returns

dict – a dictionary {reaction_name: {‘lb’: lower_bound, ‘ub’: upper_bound, ‘eq’: equality_bound}}

`cbmpy.CBTools.getExchBoundsDict(fbamod)`

Get bounds dictionary for all exchange reactions.

Returns a dictionary of all exchange reactions (as determined by the `is_exchange` attribute of Reaction) after removing non-exchange reactions.

Parameters

fbamod (*CBMPy model*) – a CBMPy model

Returns

dict – a dictionary mapping exchange reaction names to their bounds

`cbmpy.CBTools.processBiGGchemFormula(fba)`

Disambiguate overloaded BiGG species names (NAME_CHEMFORMULA).

Parse BiGG-style overloaded names and separate them into species name and chemical formula components.

Parameters

fba (*FBA model*) – an FBA model instance

Note

Modifies species.name and species.chemFormula in place.

`cbmpy.CBTools.processBiGGannotationNote(fba, annotation_key='note')`

Parse HTML formatted reaction information from BiGG notes field.

This function is being deprecated in favor of `processSBMLAnnotationNotes()`.

Parameters

- **fba** (*FBA model*) – an FBA model instance
- **annotation_key** (*str, optional*) – the annotation key containing BiGG HTML [default='note']

Warning

This function is deprecated. Use `CBTools.processSBMLAnnotationNotes()` instead.

Note

Modifies r.annotation in place.

`cbmpy.CBTools.processSBMLAnnotationNotes(fba, annotation_key='note', level=3)`

Parse HTML formatted reaction information from SBML notes field.

Parse HTML formatted reaction information stored in the SBML notes field. Processes BiGG and PySCeSCBM style annotations, looking for entries indexed with the *annotation_key*.

Parameters

- **fba** (*FBA model*) – an FBA model instance
- **annotation_key** (*str, optional*) – the annotation key containing HTML/XHTML fragment (ignored in SBML Level 3) [default='note']
- **level** (*int, optional*) – SBML level (only level >= 3 uses SBML notes) [default=3]

Note

Modifies `r.annotation` in place.

`cbmpy.CBTools.processExchangeReactions(fba, key)`

Extract exchange reactions from model and return dictionaries.

Extract exchange reactions from model using *key* and return dictionaries of exchange and medium reactions.

Parameters

- **fba** (*FBA model*) – an FBA model instance
- **key** (*str, optional*) – substring to filter reactions by name [default=None]

Returns

tuple – a dictionary of all exchange reactions (without medium reactions) a dictionary of medium exchange reactions (those with negative lower bound)

`cbmpy.CBTools.generateInputScanReports(fba, exDict, mediumDict, optimal_growth_rates, wDir, tag="")`

Generate scan reports for optimal growth rates.

Generates CSV reports for optimal growth rates, exchange reactions, and medium composition.

Parameters

- **fba** (*FBA model*) – an FBA model instance
- **exDict** (*dict*) – dictionary of exchange reaction bounds
- **mediumDict** (*dict*) – dictionary of medium reaction bounds
- **optimal_growth_rates** (*dict*) – dictionary mapping reaction IDs to optimal rates
- **wDir** (*str*) – working directory for output files
- **tag** (*str, optional*) – tag appended to model name [default=""]

Returns

None – writes three CSV files: `.medium.csv`, `.exchange.csv`, `.optima.csv`

`cbmpy.CBTools.getAllReactionsAssociatedWithGene(fba, gene, gene_annotation_key='GENE ASSOCIATION')`

Get all reactions associated with a gene via annotations.

Parameters

- **fba** (*FBA model*) – an FBA model instance
- **gene** (*str*) – a gene ID to search for
- **gene_annotation_key** (*str, optional*) – the annotation key containing gene associations [default='GENE ASSOCIATION']

Returns

list – a list of reaction IDs associated with the given gene

`cbmpy.CBTools.scanForReactionDuplicates(f, ignore_coefficients=False)`

Scan for duplicate reactions with matching stoichiometry.

This method uses a brute force approach to finding reactions with matching stoichiometry.

Parameters

- **f** (*FBA model*) – an FBA model instance
- **ignore_coefficients** (*bool, optional*) – if True, ignore coefficients when comparing [default=False]

Returns

list – a list of duplicate reaction pairs, where each pair contains: [reaction_id_1, reaction_id_2, refspecies, reaction_name_1, reaction_name_2]

`cbmpy.CBTools.countedPause(Tsec)`

Print a countdown timer.

Parameters

Tsec (*int*) – number of seconds to count down

Returns

None – prints a countdown timer, then returns

`cbmpy.CBTools.addGenesFromAnnotations(fba, annotation_key='GENE ASSOCIATION',
gene_pattern=None)`

Deprecated method for adding genes from annotations.

Add genes to the model using the definitions stored in the annotation key.

Parameters

- **fba** (*CBMPy model*) – an FBA model object
- **annotation_key** (*str, optional*) – the annotation dictionary key that holds gene associations [default='GENE ASSOCIATION']
- **gene_pattern** (*str, optional*) – deprecated, no longer needed [default=None]

Warning

This method is deprecated. Use `cmod.createGeneAssociationsFromAnnotations()` instead.

Note

Modifies the model in place.

```
cbmpy.CBTools.getModelGenesPerReaction(fba, gene_pattern=None,  
                                         gene_annotation_key='GENE  
                                         ASSOCIATION')
```

Get the genes per reaction for a model.

Parse BiGG-style gene annotation strings to extract gene IDs per reaction.

Parameters

- **fba** (*FBA model*) – an FBA model instance
- **gene_pattern** (*str*, *optional*) – deprecated, use model methods instead [default=None]
- **gene_annotation_key** (*str*, *optional*) – the annotation key [default='GENE ASSOCIATION']

Note

It is advisable to use the model methods directly rather than this function.

Returns

dict – a dictionary mapping reaction IDs to lists of gene IDs (or None if no genes)

```
cbmpy.CBTools.getReactionsPerGene(react_gene)
```

Get the reactions per gene for a gene-reaction dictionary.

Convert a dictionary of gene-to-reaction mappings into a reverse mapping.

Parameters

react_gene (*dict*) – dictionary mapping gene IDs to lists of reaction IDs

Returns

tuple – (gene_react, genes, no_gene) where: - gene_react: dictionary mapping gene IDs to lists of reaction IDs - genes: list of gene IDs with associated reactions - no_gene: list of reaction IDs with no gene associated

`cbmpy.CBTools.removeFixedSpeciesReactions(f)`

Remove fixed-species reactions from a model.

This function removes reactions that only have boundary species as reactants and products. These are typically gene associations encoded in Manchester style.

Parameters

f (*FBA model*) – an FBA model object

Returns

None – removes reactions in place

`cbmpy.CBTools.addFluxAsActiveObjective(f, reaction_id, osense, coefficient=1)`

Add a flux as an active objective function.

Parameters

- **f** (*FBA model*) – an FBA model instance
- **reaction_id** (*str*) – a reaction ID string
- **osense** (*str*) – objective sense (must be ‘maximize’ or ‘minimize’)
- **coefficient** (*int, optional*) – the objective function coefficient [default=1]

Returns

None – modifies model in place

`cbmpy.CBTools.checkReactionBalanceElemental(f, Rid=None, zero_tol=1e-12)`

Check if reactions are balanced using chemical formulas.

Check if reactions are balanced (elementally and by charge) using chemical formulas.

Parameters

- **f** (*FBA model*) – an FBA model object
- **Rid** (*str or list, optional*) – specific reaction(s) to check, or None for all [default=None]
- **zero_tol** (*float, optional*) – floating point tolerance for zero check [default=1.0e-12]
- **installed.** (*Requires PyParsing to be*)

Returns

dict – a dictionary mapping reaction IDs to their balance status: {‘id’: reaction_id, ‘charge_balanced’: bool, ‘element_balanced’: bool, ‘elements’: dict, ‘charge’: float, ‘stuff’: list}

`cbmpy.CBTools.scanForUnbalancedReactions(f, output='all')`

Scan a model for unbalanced reactions.

Scan for unbalanced reactions and return dictionaries of balanced and unbalanced reactions by charge and element balance.

Parameters

- **f** (*FBA model*) – an FBA model instance
- **output** (*str, optional*) – one of ['all', 'charge', 'element'] [default='all'] - 'charge': return all charge-unbalanced reactions - 'element': return all element-unbalanced reactions - 'all': return all unbalanced reactions

Returns

dict – a dictionary of unbalanced reactions

`cbmpy.CBTools.createZipArchive(zipname, files, move=False, compression='normal')`

Create a zip archive containing one or more files.

Parameters

- **zipname** (*str*) – the fully qualified name of the zip archive to create
- **files** (*str or list*) – either a filename or list of filenames (fully qualified)
- **move** (*bool, optional*) – if True, delete input files after creating archive [default=False]
- **compression** (*str or None, optional*) – compression mode (None for no compression) [default='normal']
- **files.** (*Creates a zip archive containing one or more*)

Returns

None – writes archive to zipname

`cbmpy.CBTools.checkExchangeReactions(fba, autocorrect=True)`

Scan all reactions for exchange reactions and correct inconsistencies.

Scan all reactions for exchange reactions (those containing a boundary species). Returns a list of inconsistent reactions or correct them automatically.

Parameters

- **fba** (*CBMPy model*) – a CBMPy model
- **autocorrect** (*bool, optional*) – if True, automatically correct the “is_exchange” attribute on inconsistent reactions [default=True]

Returns

list – a list of reaction IDs that were inconsistent (and possibly corrected)

`cbmpy.CBTools.checkIds(fba, items='all')`

Check model attribute IDs for validity and duplicates.

Check the IDs of specified model attributes for legal naming and duplicates.

Parameters

- **fba** (*CBMPy model*) – a CBMPy model instance
- **items** (*str or list, optional*) – what to check: ‘all’ means ‘species,reactions,flux_bounds,objectives’ [default=‘all’]

Returns

dict – a dictionary of errors found for each item type

`cbmpy.CBTools.checkFluxBoundConsistency(fba)`

Check flux bound consistency for the model.

Check for multiply defined bounds, bounds without a reaction, inconsistent bounds with respect to each other, and reaction reversibility.

Parameters

fba (*CBMPy model*) – a CBMPy model instance

Returns

dict – a dictionary of errors found: - ‘eq+lb’: equality + lower bound conflicts - ‘eq+ub’: equality + upper bound conflicts - ‘duplicate_ids’: duplicate flux bound IDs - ‘multiple_defines’: multiple bound definitions - ‘lb>ub’: lower bound > upper bound - ‘undefined’: bounds without values - ‘rev_contradict’: reversibility contradictions - ‘no_reaction’: bounds without a reaction

`cbmpy.CBTools.roundOffWithSense(val, osense='max', tol=1e-08)`

Round a value with sense-aware rounding.

Round a value in a way that takes into consideration the sense of the operation that generated it (maximize rounds up, minimize rounds down).

Parameters

- **val** (*float*) – the value to round
- **osense** (*str, optional*) – the operation sense (‘maximize’ or ‘minimize’) [default=‘max’]
- **tol** (*float, optional*) – the tolerance for roundoff [default=1e-8]

Returns

float – the rounded value

`cbmpy.CBTools.mergeGroups(m, groups, new_id, new_name="", auto_delete=False)`

Merge a list of groups into a new group.

Merge a list of groups into a new group. Note: annotations are not merged.

Parameters

- **m** (*CBMPy model*) – a model containing the source groups
- **groups** (*list*) – a list of group IDs to merge
- **new_id** (*str*) – the new merged group ID
- **new_name** (*str*, *optional*) – the new group name (default merges old names) [default=""]
- **auto_delete** (*bool*, *optional*) – if True, delete source groups after merge [default=False]

Returns

bool – True if merge succeeded, False otherwise

`cbmpy.CBTools.merge2Models(m1, m2, ignore=None, ignore_duplicate_ids=False)`

Merge two models (m2 into m1) without model checking.

Merge 2 models, this method does a raw merge of model 2 into model 1 without any model checking. Component IDs in ignore are ignored in both models and the first objective of model 1 is arbitrarily set as active. Compartments are also merged and a new “OuterMerge” compartment is created.

In all cases, duplicate IDs are tracked and ignored, essentially using the object ID encountered first (usually that of model 1). Duplicate checking can be disabled by setting the *ignore_duplicate_ids* flag.

Parameters

- **m1** (*CBMPy model*) – model 1 (target)
- **m2** (*CBMPy model*) – model 2 (source)
- **ignore** (*list*, *optional*) – list of IDs to skip during merge [default=None]
- **ignore_duplicate_ids** (*bool*, *optional*) – disable duplicate ID checking [default=False]

Note

In development - merging genes and GPRs.

Returns

CBMPy model – the merged model

2.6.9 CBMPy: CBDataStruct module

PySCeS Constraint Based Modelling (<http://cbmpy.sourceforge.net>) Copyright (C) 2009-2024 Brett G. Olivier, VU University Amsterdam, Amsterdam, The Netherlands

This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program. If not, see <<http://www.gnu.org/licenses/>>

Author: Brett G. Olivier PhD Contact developers: <https://github.com/SystemsBioinformatics/cbmpy/issues> Last edit: \$Author: bgoli \$ (\$Id: CBDataStruct.py 710 2020-04-27 14:22:34Z bgoli \$)

2.7 Solver Implementations

2.7.1 CBMPy: CBSolver module

PySCeS Constraint Based Modelling (<http://cbmpy.sourceforge.net>) Copyright (C) 2009-2024 Brett G. Olivier, VU University Amsterdam, Amsterdam, The Netherlands

This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program. If not, see <<http://www.gnu.org/licenses/>>

Author: Brett G. Olivier PhD Contact developers: <https://github.com/SystemsBioinformatics/cbmpy/issues> Last edit: \$Author: bgoli \$ (\$Id: CBSolver.py 710 2020-04-27 14:22:34Z bgoli \$)

2.7.2 CBMPy: CBSolver2 module

PySCeS Constraint Based Modelling (<http://cbmpy.sourceforge.net>) Copyright (C) 2009-2024 Brett G. Olivier, VU University Amsterdam, Amsterdam, The Netherlands

This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program. If not, see <<http://www.gnu.org/licenses/>>

Author: Brett G. Olivier PhD Contact developers: <https://github.com/SystemsBioinformatics/cbmpy/issues> Last edit: \$Author: bgoli \$ (\$Id: CBSolver.py 305 2024-04-23 15:18:31Z bgoli \$)

class cbmpy.CBSolverX.LPBase

Bases: `object`

The base of all LP's.

obj = None

id = None

name = None

objective = None

objective_sense = None

linear_constraints = None

addObjective(*arr*)

Add objective as a list of (coefficient, variable) tuples.

- *arr* a list of one or more (coefficient, variable) tuples

setObjectiveSense(*sense*)

Set the objective sense.

- *sense* is the string 'MAX' or 'MIN'

addLinearConstraint(*cid*, *arr*, *sense*, *rhs*, *overwrite=False*)

Add linear constraint as a list of (coefficient, variable) tuples.

- *cid* the constraint id
- *arr* a list of (coefficient, variable) tuples
- *sense* GE, LE, EQ ($\geq$, $\leq$, $=$)
- *rhs* the floating point value of the rhs
- *overwrite* [default=False] if a cid already exists overwrite it

class cbmpy.CBSolverX.MILPBase

Bases: `object`

The base of all MILP's.

class cbmpy.CBSolverX.QPbase

Bases: `object`

The base of all QP's.

class cbmpy.CBSolverX.LPCPLEX

Bases: `LPBase`

CPLEX LP implimentation.

class cbmpy.CBSolverX.LPECYGLPKI

Bases: `LPBase`

ECYGLPKI LP implimentation.

2.7.3 CBMPy: CBGLPK module

PySCeS Constraint Based Modelling (<http://cbmpy.sourceforge.net>) Copyright (C) 2009-2024 Brett G. Olivier, VU University Amsterdam, Amsterdam, The Netherlands

This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program. If not, see <<http://www.gnu.org/licenses/>>

Author: Brett G. Olivier PhD Contact developers: <https://github.com/SystemsBioinformatics/cbmpy/issues> Last edit: \$Author: bgoli \$ (\$Id: CBGLPK.py 713 2020-07-29 22:28:43Z bgoli \$)

`cbmpy.CBGLPK.glpk_constructLPfromFBA(fba, fname=None)`

Create a GLPK LP in memory.

Parameters

- **fba** – an FBA object
- **fname** – optional filename if defined writes out the constructed lp

`cbmpy.CBGLPK.glpk_Solve(lp, method='s')`

Solve the LP and create a status attribute with the solution status.

Parameters

method – [default='s'] 's' = simplex, 'i' = interior, 'e' = exact

`cbmpy.CBGLPK.glpk_getSolutionStatus(lp)`

Returns one of:

Parameters

- **LPS_OPT** – : solution is optimal;
- **LPS_FEAS** – : solution is feasible;
- **LPS_INFEAS** – : solution is infeasible;
- **LPS_NOFEAS** – : problem has no feasible solution;
- **LPS_UNBND** – : problem has unbounded solution;
- **LPS_UNDEF** – : solution is undefined.

`cbmpy.CBGLPK.glpk_analyzeModel(f, lpFname=None, return_lp_obj=False, with_reduced_costs='scaled', with_sensitivity=False, del_intermediate=False, build_n=True, quiet=False, oldlpgen=False, method='s')`

Optimize a model and add the result of the optimization to the model object (e.g. *reaction.value*, *objectiveFunction.value*). The stoichiometric matrix is automatically generated. This is a common function available in all solver interfaces. By default returns the objective function value.

Parameters

- **f** – an instantiated PySCeSCBM model object
- **lpFname** – [default=None] the name of the intermediate LP file saved when this has a string value.
- **return_lp_obj** – [default=False] off by default when enabled it returns the PyGLPK LP object
- **with_reduced_costs** – [default='scaled'] calculate and add reduced cost information to mode this can be: 'unscaled' or 'scaled'
- **with_sensitivity** – [default=False] add solution sensitivity information (not yet implemented)
- **del_intermediate** – [default=False] delete the intermediary files after updating model object, useful for server applications
- **build_n** – [default=True] generate stoichiometry from the reaction network (reactions/reagents/species)
- **quiet** – [default=False] suppress glpk output
- **oldlpgen** – [default=False] not used anymore
- **method** – [default='s'] select the GLPK solver method, see the GLPK documentation for details

`cbmpy.CBGLPK.glpk_setSolutionStatusToModel(m, lp)`

Sets the lp solutions status to the CBMPy model.

`cbmpy.CBGLPK.glpk_setFBAsolutionToModel(fba, lp, with_reduced_costs='scaled')`

Sets the FBA solution from a CPLEX solution to an FBA object.

Parameters

- **fba** – an fba object
- **lp** – a CPLEX LP object
- **with_reduced_costs** – [default='unscaled'] calculate and add reduced cost information to mode this can be: 'unscaled' or 'scaled'

`cbmpy.CBGLPK.glpk_getOptimalSolution(c)`

From a GLPK model extract a tuple of solution, ObjFuncName and ObjFuncVal.

Parameters

c – a GLPK object

`cbmpy.CBGLPK.glpk_getReducedCosts(c, scaled=False)`

Extract ReducedCosts from LP and return as a dictionary 'Rid' : reduced cost.

Parameters

- **c** – a GLPK LP object
- **scaled** – scale the reduced cost by the optimal flux value

`cbmpy.CBGLPK.getReducedCosts(fba)`

Get a dictionary of reduced costs for each reaction/flux.

`cbmpy.CBGLPK.setReducedCosts(fba, reduced_costs)`

For each reaction/flux, sets the attribute "reduced_cost" from a dictionary of reduced costs.

Parameters

- **fba** – an fba object
- **reduced_costs** – a dictionary of {reaction : value} pairs

`cbmpy.CBGLPK.glpk_FluxVariabilityAnalysis(fba, selected_reactions=None,
pre_opt=True, tol=None,
objF2constr=True, rhs_sense='lower',
optPercentage=100.0, work_dir=None,
quiet=True, debug=False,
oldlpgen=False, markupmodel=True,
default_on_fail=False,
roundoff_span=10, method='s')`

Perform a flux variability analysis on an fba model:

Parameters

- **fba** – an FBA model object
- **pre_opt** – [default=True] attempt to presolve the FBA and report its results in the output, if this is disabled and *objF2constr* is True then the vid/value of the current active objective is used
- **tol** – [default=None] do not floor/ceiling the objective function constraint, otherwise round of to *tol*
- **rhs_sense** – [default='lower'] means $\text{objC} \geq \text{objVal}$ the inequality to use for the objective constraint can also be *upper* or *equal*
- **optPercentage** – [default=100.0] means the percentage optimal value to use for the RHS of the objective constraint: $\text{optimal_value} * (\text{optPercentage} / 100.0)$
- **work_dir** – [default=None] the FVA working directory for temporary files default = `cwd+fva`
- **debug** – [default=False] if True write out all the intermediate FVA LP's into `work_dir`
- **quiet** – [default=False] if enabled suppress CPLEX output
- **objF2constr** – [default=True] add the model objective function as a constraint using *rhs_sense* etc. If
- **markupmodel** – [default=True] add the values returned by the fva to the `reaction.fva_min` and `reaction.fva_max`
- **default_on_fail** – [default=False] if *pre_opt* is enabled replace a failed minimum/maximum with the solution value
- **roundoff_span** – [default=10] number of digits is round off (not individual min/max values)
- **method** – [default='s'] select the GLPK solver method, see the GLPK documentation for details

Returns

an array with columns Reaction, Reduced Costs, Variability Min, Variability Max, abs(Max-Min), MinStatus, MaxStatus and a list containing the row names.

```
cbmpy.CBGLPK.glpk_func_GetCPXandPresolve(fba, pre_opt, objF2constr, quiet=False,
                                          oldlpgen=True,
                                          with_reduced_costs='unscaled',
                                          method='s')
```

This is a utility function that does a presolve for FVA, MSAF etc. Generates properly formatted empty objects if `pre_opt == False`.

Parameters

- **pre_opt** – a boolean
- **fba** – a CBModel object
- **objF2constr** – add objective function as constraint
- **quiet** – [default=False] suppress cplex output
- **with_reduced_costs** – [default='unscaled'] can be 'scaled' or 'unscaled'
- **method** – [default='s'] select the GLPK solver method, see the GLPK documentation for details

`cbmpy.CBGLPK.glpk_func_SetObjectiveFunctionAsConstraint(cpx, rhs_sense, oval, tol, optPercentage)`

Take the objective function and “optimum” value and add it as a constraint.

Parameters

- **cpx** – a cplex object
- **oval** – the objective value
- **tol** – [default=None] do not floor/ceiling the objective function constraint, otherwise round of to *tol*
- **rhs_sense** – [default='lower'] means $\text{objC} \geq \text{objVal}$ the inequality to use for the objective constraint can also be *upper* or *equal*
- **optPercentage** – [default=100.0] means the percentage optimal value to use for the RHS of the objective constraint: $\text{optimal_value} * (\text{optPercentage} / 100.0)$

`cbmpy.CBGLPK.glpk_setSingleConstraint(c, cid, expr=[], sense='E', rhs=0.0)`

Sets a new single constraint to a GLPK model.

Parameters

- **c** – a GLPK instance
- **cid** – the constraint id
- **expr** – a list of (coefficient, name) pairs
- **sense** – [default='G'] LGE
- **rhs** – [default=0.0] the right hand side

`cbmpy.CBGLPK.glpk_setObjective(c, oid, expr=None, sense='maximize', reset=True)`

Set a new objective function note that there is a major memory leak in `c.variables.get_names()` which is used when `reset=True`. If this is a problem use `cplx_setObjective2` which takes *names* as an input:

Parameters

- **c** – a GLPK LP object
- **oid** – the `r_id` of the flux to be optimized
- **expr** – a list of (coefficient, flux) pairs
- **sense** – ‘maximize’/‘minimize’
- **reset** – [default=True] reset all objective function coefficients to zero

Optional Solvers ==—

CBMPy supports additional solvers that require optional dependencies.

CBCPLEX - IBM CPLEX solver wrapper. Requires the IBM CPLEX Optimization Studio package. Source: `cbmpy/CBCPLEX.py`.

CBGLPKOLD - Legacy GLPK wrapper. Requires an older version of the `glpk` package. Source: `cbmpy/CBGLPKOLD.py`.

File I/O Modules —==—==—==—==

2.8 CBMPy: CBRead module

PySCeS Constraint Based Modelling (<http://cbmpy.sourceforge.net>) Copyright (C) 2009-2024 Brett G. Olivier, VU University Amsterdam, Amsterdam, The Netherlands

This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program. If not, see <<http://www.gnu.org/licenses/>>

Author: Brett G. Olivier PhD Contact developers: <https://github.com/SystemsBioinformatics/cbmpy/issues> Last edit: \$Author: bgoli \$ (\$Id: CBRead.py 669 2019-02-18 22:58:19Z bgoli \$)

`cbmpy.CBRead.loadModel(sbmlfile)`

Loads any SBML model in COBRA, FAME (SBML2FBA), SBML3FBCv1, SBML3FBCv2 format.

Parameters

sbmlfile – an SBML model file

`cbmpy.CBRead.readSBML3FBC(fname, work_dir=None, return_sbml_model=False, xoptions={'validate': False}, scan_notes_gpr=True)`

Read in an SBML Level 3 file with FBC annotation where and return a CBM model object.

Parameters

- **fname** – is the filename
- **work_dir** – is the working directory
- **return_sbml_model** – deprecated and ignored please update code
- **xoptions** – special load options, enable with option=True except for nmatrix_type which has a type.
- **nogenes** – do not load/process genes
- **noannot** – do not load/process any annotations
- **validate** – validate model and display errors and warnings before loading
- **readcobra** – read the cobra annotation
- **read_model_string** – [default=False] read the model from a string (instead of a filename) containing an SBML document
- **nmatrix_type** – [default='normal'] define the type of stoichiometric matrix to be built
- **scan_notes_gpr** – [default=True] if the model is loaded and no genes are detected scan the <notes> field for GPR association

```
cbmpy.CBRead.readCOBRASBML(fname, work_dir=None, return_sbml_model=False,  
                             delete_intermediate=False,  
                             fake_boundary_species_search=False, output_dir=None,  
                             skip_genes=False, scan_notes_gpr=True)
```

Read in a COBRA format SBML Level 2 file with FBA annotation where and return either a CBM model object or a (cbm_mod, sbml_mod) pair if return_sbml_model=True.

Parameters

- **fname** – is the filename
- **work_dir** – is the working directory
- **delete_intermediate** – [default=False] delete the intermediate SBML Level 3 FBC file
- **fake_boundary_species_search** – [default=False] after looking for the boundary_condition of a species search for overloaded id's <id>_b
- **output_dir** – [default=None] the directory to output the intermediate SBML L3 files (if generated) default to input directory

- **skip_genes** – [default=False] do not load GPR data
- **scan_notes_gpr** – [default=True] if the model is loaded and no genes are detected the scan the <notes> field for GPR associationa

```
cbmpy.CBRead.readSBML2FBA(fname, work_dir=None, return_sbml_model=False,  
                           fake_boundary_species_search=False, scan_notes_gpr=True)
```

Read in an SBML Level 2 file with FBA annotation where:

Parameters

- **fname** – is the filename
- **work_dir** – is the working directory if None then only fname is used
- **return_sbml_model** – [default=False] return a a (cbm_mod, sbml_mod) pair
- **fake_boundary_species_search** – [default=False] after looking for the boundary_condition of a species search for overloaded id's <id>_b
- **scan_notes_gpr** – [default=True] if the model is loaded and no genes are detected the scan the <notes> field for GPR associationa

```
cbmpy.CBRead.readLPtoList(fname, work_dir)
```

```
cbmpy.CBRead.readSK_FVA(filename)
```

Read Stevens FVA results (opt.fva) file and return a list of dictionaries.

```
cbmpy.CBRead.readSK_vertexOld(fname, bigfile=False, fast_rational=False,  
                              nformat='%%.14f', compresslevel=3)
```

Reads in Stevens vertex analysis file and returns, even more optimized for large datasets than the original.

- a list of vertex vectors
- a list of ray vectors
- the basis of the lineality space as a list of vectors

all vectors in terms of the column space of N

```
cbmpy.CBRead.readSK_vertex(fname, bigfile=True, fast_rational=False, nformat='%%.14f',  
                           compression=None, hdf5file=None)
```

Reads in Stevens vertex analysis file:

Parameters

- **fname** – the input filename (.all file that results from Stevens pipeline)
- **bigfile** – [default=True] this option is now always true and is left in for backwards compatability

- **fast_rational** – [default=False] by default off and uses SymPy for rational→float conversion, when on uses float decomposition with a slight (2th decimal) decrease in accuracy
- **nformat** – [default='%.14f'] the number format used in output files
- **compression** – [default=None] compression to be used in hdf5 files can be one of [None, 'lzf', 'gz?', 'gzip']
- **hdf5file** – [default=None] if None then generic filename '_vtx_.tmp.hdf5' is used otherwise <hdf5file>.hdf5

```
cbmpy.CBRead.readExcel97Model(xlname, write_sbml=True, sbml_level=3,
                              return_dictionaries=False)
```

Reads a model encoded as an Excel97 workbook and returns it as a CBMPy model object and SBML file. Note the workbook must be formatted exactly like those produced by `cbm.writeModelToExcel97()`. Note that reactions have to be defined in **both** the *reaction* and *network_react* sheets to be included in the model.

Parameters

- **xlpath** – the filename of the Excel workbook
- **return_model** – [default=True] construct and return the CBMPy model
- **write_sbml** – [default=True] write the SBML file to fname
- **return_dictionaries** – [default=False] return the dictionaries constructed when reading the Excel file (in place of the model)
- **sbml_level** – [default=3] write the SBML file as either SBML L2 FBA or SBML L3 FBC file.

2.9 CBMPy: CBReadtxt module

PySCeS Constraint Based Modelling (<http://cbmpy.sourceforge.net>) Copyright (C) 2009-2024 Brett G. Olivier, VU University Amsterdam, Amsterdam, The Netherlands

This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program. If not, see <<http://www.gnu.org/licenses/>>

Author: Brett G. Olivier PhD Contact developers: <https://github.com/SystemsBioinformatics/cbmpy/issues> Last edit: \$Author: bgoli \$ (\$Id: CBReadtxt.py 710 2020-04-27 14:22:34Z bgoli \$)

```
cbmpy.CBReadtxt.getBounds(fname, reaction_prefix='R_', has_header=False)

cbmpy.CBReadtxt.getReactions(fname, reaction_prefix='R_', has_header=False,
                             save_rpt=False, ignore_duplicates=False)

cbmpy.CBReadtxt.getReactions_old_format(fname, reaction_prefix='R_',
                                         has_header=False, save_rpt=False,
                                         ignore_duplicates=False)

cbmpy.CBReadtxt.parseReactions(sRxns)

cbmpy.CBReadtxt.addBoundsToReactions(Reactions, Bounds, default=1111.0)

cbmpy.CBReadtxt.dumpReactionsToTxt(Reactions, fname)

cbmpy.CBReadtxt.getSpecies(reactions)

cbmpy.CBReadtxt.dumpSpeciesToTxt(species, fname)

cbmpy.CBReadtxt.buildFBAobj(reactions, species, objfname, modname='model')

cbmpy.CBReadtxt.readCSV(model_file, bounds_file=None, biomass_flux=None,
                        model_id='FBAModel', reaction_prefix='R_', has_header=False)
```

This function loads a CSV file and translates it into a Python object:

```
- *model_file* the name of the CSV file that contains the model
- *bounds_file* the name of the CSV file that contains the flux.
  ↳ bounds
- *biomass_flux* the name of the reaction that is the objective.
  ↳ function
- *reaction_prefix* [default='R _'] the prefix to add to input.
  ↳ reaction ID's
- *has_header* [default=False] if there is a header row in the csv.
  ↳ file
```

2.10 CBMPy: CBWrite module

PySCeS Constraint Based Modelling (<http://cbmpy.sourceforge.net>) Copyright (C) 2009-2024 Brett G. Olivier, VU University Amsterdam, Amsterdam, The Netherlands

This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program. If not, see <<http://www.gnu.org/licenses/>>

Author: Brett G. Olivier PhD Contact developers: <https://github.com/SystemsBioinformatics/cbmpy/issues> Last edit: \$Author: bgoli \$ (\$Id: CBWrite.py 710 2020-04-27 14:22:34Z bgoli \$)

`cbmpy.CBWrite.saveModel(model, filename, compress=False)`

Saves the model to an SBML file using the latest SBML3 FBC version.

Parameters

- **model** – the CBMPy model
- **filename** – the filename to write

`cbmpy.CBWrite.writeSBML3FBC(fba, fname, directory=None, gpr_from_annot=False, add_groups=True, add_cbmpy_annot=True, add_cobra_annot=False, xoptions={'compress_bounds': True, 'fbc_version': 1, 'validate': False})`

Takes an FBA model object and writes it to file as SBML L3 FBC:

Parameters

- **fba** – an fba model object
- **fname** – the model will be written as XML to *fname*
- **directory** – [default=None] if defined it is prepended to *fname*
- **gpr_from_annot** – [default=True] if enabled will attempt to add the gene protein associations from the annotations
- **add_groups** – [default=True] add SBML3 groups (if supported by libSBML)
- **add_cbmpy_annot** – [default=True] add CBMPy KeyValueType annotation. Replaces <notes>
- **add_cobra_annot** – [default=True] add COBRA <notes> annotation
- **xoptions** – extended options
- **fbc_version** – [default=1] write SBML3FBC using version 1 (2013) or version 2 (2015)
- **validate** – [default=False] validate the output SBML file
- **compress_bounds** – [default=False] try compress output flux bound parameters

- **zip_model** – [default=False] compress the model using PKZIP encoding
- **return_model_string** – [default=False] return the SBML XML file as a string

```
cbmpy.CBWrite.writeSBML3FBCV2(fba, fname, directory=None, gpr_from_annot=False,
                               add_groups=True, add_cbmpy_annot=True,
                               add_cobra_annot=False, validate=False,
                               compress_bounds=False, zip_model=False,
                               return_model_string=False)
```

Takes an FBA model object and writes it to file as SBML L3 FBCv2 :

Parameters

- **fba** – an fba model object
- **fname** – the model will be written as XML to *fname*
- **directory** – [default=None] if defined it is prepended to *fname*
- **gpr_from_annot** – [default=False] if enabled will attempt to add the gene protein associations from the annotations
- **add_groups** – [default=True] add SBML3 groups (if supported by libSBML)
- **add_cbmpy_annot** – [default=True] add CBMPy KeyValueData annotation. Replaces <notes>
- **add_cobra_annot** – [default=False] add COBRA <notes> annotation
- **validate** – [default=False] validate the output SBML file
- **compress_bounds** – [default=True] try compress output flux bound parameters
- **zip_model** – [default=False] compress the model using ZIP encoding
- **return_model_string** – [default=False] return the SBML XML file as a string

```
cbmpy.CBWrite.writeSBML3FBCV3(fba, fname, directory=None, gpr_from_annot=False,
                               add_groups=True, add_cbmpy_annot=True,
                               add_cobra_annot=False, validate=False,
                               compress_bounds=False, zip_model=False,
                               return_model_string=False)
```

Takes an FBA model object and writes it to file as SBML L3 FBCv3 :

Parameters

- **fba** – an fba model object
- **fname** – the model will be written as XML to *fname*

- **directory** – [default=None] if defined it is prepended to *fname*
- **gpr_from_annot** – [default=False] if enabled will attempt to add the gene protein associations from the annotations
- **add_groups** – [default=True] add SBML3 groups (if supported by libSBML)
- **add_cbmpy_annot** – [default=True] add CBMPy KeyValueType annotation. Replaces <notes>
- **add_cobra_annot** – [default=False] add COBRA <notes> annotation
- **validate** – [default=False] validate the output SBML file
- **compress_bounds** – [default=True] try compress output flux bound parameters
- **zip_model** – [default=False] compress the model using ZIP encoding
- **return_model_string** – [default=False] return the SBML XML file as a string

`cbmpy.CBWrite.writeCOBRASBML(fba, fname, directory=None)`

Takes an FBA model object and writes it to file as a COBRA compatible :

Parameters

- **fba** – an fba model object
- **fname** – the model will be written as XML to *fname*
- **directory** – [default=None] if defined it is prepended to *fname*

`cbmpy.CBWrite.writeSBML2FBA(fba, fname, directory=None, sbml_level_version=None)`

Takes an FBA model object and writes it to file as SBML L2 with FBA annotations. Note if you want to write BiGG/FAME style annotations then you must use *sbml_level_version*=(2,1)

Parameters

- **fba** – an fba model object
- **fname** – the model will be written as XML to *fname*
- **sbml_level_version** – [default=None] a tuple containing the SBML level and version e.g. (2,1)

`cbmpy.CBWrite.writeSensitivitiesToCSV(sensitivities, fname)`

Write out a sensitivity report using the objective sensitivities and bound sensitivity dictionaries created by e.g. `cplx_getSensitivities()`.

Parameters

- **sensitivity** – tuple containing

- **obj_sens** – dictionary of objective coefficient sensitivities (per flux)
- **rhs_sens** – dictionary of constraint rhs sensitivities (per constraint)
- **bound_sens** – dictionary of bound sensitivities (per flux)
- **fname** – output filename e.g. fname.csv

`cbmpy.CBWrite.WriteModelRaw(fba, work_dir=None)`

INFO: this method will be deprecated please update your scripts to use “writeModelRaw()”

`cbmpy.CBWrite.writeModelRaw(fba, work_dir=None)`

Writes a fba (actually just dumps it) to a text file.

Parameters

- **fba** – an instantiated FBAmodel instance
- **work_dir** – directory designated for output

`cbmpy.CBWrite.BuildLPFluxBounds(fba, use_rational=False)`

Build and return a csio that contains the flux bounds in LP format.

`cbmpy.CBWrite.BuildLPConstraints(fba, use_rational=False)`

Build and return a csio that contains constraint constructed from the StoichiometryLP object.

Parameters

- **fba** – an fba model object which has a stoichiometry
- **use_rational** – write rational number output [default=False]

`cbmpy.CBWrite.BuildLPUserConstraints(fba, use_rational=False)`

Build and return a csio that contains constraint constructed from the StoichiometryLP object.

Parameters

- **fba** – an fba model object which has a stoichiometry
- **use_rational** – write rational number output [default=False]

`cbmpy.CBWrite.BuildLPConstraintsRelaxed(fba)`

Build and return a csio that contains the constraints in LP format Relaxed refers to $dS/dt \geq 0$.

`cbmpy.CBWrite.BuildLPConstraintsStrict(fba, use_rational=False)`

Build and return a csio that contains the constraints in LP format Strict refers to $dS/dt = 0$.

`cbmpy.CBWrite.BuildLPConstraintsMath(fba, use_rational=False)`

Build and return a csio that contains the constraints in LP format Strict refers to $dS/dt \Rightarrow 0$ and $dS/dt \leq 0$.

```
cbmpy.CBWrite.WriteModelLPOld(fba, work_dir=None, multisymb=' ', lpt=True,  
                                constraint_mode='strict', use_rational=False,  
                                format='%s')
```

INFO: this method will be deprecated please update your scripts to use “writeModelLPOld()”

```
cbmpy.CBWrite.writeModelLPOld(fba, work_dir=None, multisymb=' ', lpt=True,  
                                constraint_mode='strict', use_rational=False,  
                                format='%s')
```

Writes a fba as an LP/LPT.

Parameters

- **fba** – an instantiated FBAModel instance
- **work_dir** – directory designated for output
- **multisymb** – the multiplication symbol (default: <space>)
- **lpt** – the file format (default: True for lpt) or False for lp

```
cbmpy.CBWrite.WriteModelLP(fba, work_dir=None, fname=None, multisymb=' ',  
                             format='%s', use_rational=False, constraint_mode=None,  
                             quiet=False)
```

INFO: this method will be deprecated please update your scripts to use “writeModelLP()”

```
cbmpy.CBWrite.writeModelLP(fba, work_dir=None, fname=None, multisymb=' ',  
                             format='%s', use_rational=False, constraint_mode=None,  
                             quiet=False)
```

Writes an FBA object as an LP in CPLEX LP format.

Parameters

- **fba** – an instantiated FBAModel instance
- **work_dir** – directory designated for output
- **fname** – the file name [default=fba.getId()]
- **multisymb** – the multiplication symbol (default: <space>)
- **format** – the number format of the output
- **use_rational** – output rational numbers [default=False]
- **quiet** – [default=False] suppress information messages

```
cbmpy.CBWrite.BuildHformatFluxBounds(fba, infinity_replace=None, use_rational=False)
```

Build and return a csio that contains the flux bounds in H format.

Parameters

- **fba** – a PySCeS-CBM FBA object

- **infinity_replace** – [default=None] if defined this is the abs(value) of $\pm<\text{infinity}>$

```
cbmpy.CBWrite.WriteModelHFormatFBA(fba, work_dir=None, use_rational=False,  
                                     fullLP=True, format='%s', infinity_replace=None)
```

INFO: this method will be deprecated please update your scripts to use “writeModelHFormatFBA2()”

```
cbmpy.CBWrite.writeModelHFormatFBA(fba, work_dir=None, use_rational=False,  
                                     fullLP=True, format='%s', infinity_replace=None)
```

Write an FBA-LP in polynomial H-Format file. This version has been replaced by *writeModelHFormatFBA2()* but is kept for backwards compatability.

Parameters

- **fba** – a PySCeS-CBM FBA object
- **Work_dir** – [default=None] the output directory
- **use_rational** – [default=false] use rational numbers in output (requires sympy)
- **fullLP** – [default=True] include the default objective function as a maximization target
- **format** – [default='%s'] the number format string
- **infinity_replace** – [default=None] if defined this is the abs(value) of $\pm<\text{infinity}>$

```
cbmpy.CBWrite.WriteModelHFormatFBA2(fba, fname=None, work_dir=None,  
                                     use_rational=False, fullLP=True, format='%s',  
                                     infinity_replace=None)
```

INFO: this method will be deprecated please update your scripts to use “writeModelHFormatFBA2()”

```
cbmpy.CBWrite.writeModelHFormatFBA2(fba, fname=None, work_dir=None,  
                                     use_rational=False, fullLP=True, format='%s',  
                                     infinity_replace=None)
```

Write an FBA-LP in polynomial H-Format file. This is an improved version of *WriteModelHFormatFBA()* which it replaces. Note that if a SymPy matrix is used as input then *use_rational* is automatically enabled.

Parameters

- **fba** – a PySCeS-CBM FBA object
- **fname** – [default=None] the output filename, *fba.getId()* if not defined
- **Work_dir** – [default=None] the output directory

- **use_rational** – [default=false] use rational numbers in output (requires sympy)
- **fullLP** – [default=True] include the default objective function as a maximization target
- **format** – [default='%s'] the number format string
- **infinity_replace** – [default=None] if defined this is the abs(value) of $\pm<\text{infinity}>$

```
cbmpy.CBWrite.writeStoichiometricMatrix(fba, fname=None, work_dir=None,  
                                         use_rational=False, fullLP=True,  
                                         format='%s', infinity_replace=None)
```

Write an FBA-LP in polynomial H-Format file. This is an improved version of *WriteModel-HFormatFBA()* which it replaces but is kept for backwards compatability.

Parameters

- **fba** – a PySCeS-CBM FBA object
- **fname** – [default=None] the output filename, *fba.getId()* if not defined
- **Work_dir** – [default=None] the output directory
- **use_rational** – [default=false] use rational numbers in output (requires sympy)
- **fullLP** – [default=True] include the default objective function as a maximization target
- **format** – [default='%s'] the number format string
- **infinity_replace** – [default=None] if defined this is the abs(value) of $\pm<\text{infinity}>$

```
cbmpy.CBWrite.writeListToLP(fname, obj=None, const=None, bnds=None,  
                           work_dir=None, objtype='maximize')
```

```
cbmpy.CBWrite.writeMinDistanceLP(fname, fbas, work_dir=None, ignoreDistance=[],  
                                with_protein_cost=False, constraint_mode='strict',  
                                moma=False)
```

```
cbmpy.CBWrite.writeMinDistanceLP_absL1(fname, fbas, work_dir=None,  
                                       ignoreDistance=[], bigM=500,  
                                       with_protein_cost=False,  
                                       constraint_mode='strict', moma=False)
```

```
cbmpy.CBWrite.writeMinDistanceLPwithCost(fname, fbas, work_dir=None,  
                                         ignoreDistance=[],  
                                         constraint_mode='strict')
```

For backwards compatability only.

```
cbmpy.CBWrite.writeOptimalSolution(fba, fname, Dir=None, separator=',',  
                                   only_exchange=False)
```

This function writes the optimal solution to file.

Parameters

- **fba** – an instance of an PySCeSCBM model
- **fname** – the output filename
- **Dir** – [default=None] use current directory if not None
- **separator** – [default=','] the column separator
- **only_exchange** – [default=False] only output fluxes labelled as exchange reactions

```
cbmpy.CBWrite.writeModelInfoToFile(fba, fname, Dir=None, separator=',',  
                                   only_exchange=False, met_type='all')
```

This function writes a CBModel to file.

Parameters

- **fba** – an instance of an PySCeSCBM model
- **fname** – the output filename
- **Dir** – [default=None] use directory if not None
- **separator** – [default=','] the column separator
- **only_exchange** – [default=False] only output fluxes labelled as exchange reactions
- **type** – [default='all'] only output certain type of species: 'all', 'boundary' or 'variable'

```
cbmpy.CBWrite.writeReactionInfoToFile(fba, fname, Dir=None, separator=',',  
                                      only_exchange=False)
```

This function writes a CBModel to file.

Parameters

- **fba** – an instance of an PySCeSCBM model
- **fname** – the output filename
- **Dir** – [default=None] use directory if not None
- **separator** – [default=','] the column separator

- **only_exchange** – [default=False] only output fluxes labelled as exchange reactions

```
cbmpy.CBWrite.writeSpeciesInfoToFile(fba, fname, Dir=None, separator='',  
                                     met_type='all')
```

This function writes a CBModel to file.

Parameters

- **fba** – an instance of an PySCeSCBM model
- **fname** – the output filename
- **Dir** – [default=None] use directory if not None
- **separator** – [default=','] the column separator
- **met_type** – [default='all'] only output certain type of species: 'all', 'boundary' or 'variable'

```
cbmpy.CBWrite.printFBASolution(fba, include_all=False)
```

Prints the FBA optimal solution to the screen.

Parameters

- **fba** – an FBA model object
- **include_all** – include all variables

```
cbmpy.CBWrite.exportModel(fba, fname=None, fmt='lp', work_dir=None,  
                          use_rational='both')
```

Export the FBA model in different formats:

Parameters

- **fba** – the FBA model
- **fname** – [default=None] the exported filename if None then *fba.getId()* is used
- **fmt** – [default='lp'] the export format can be one of: 'lp' (CPLEX), 'hformat' (Polyhedra), 'all' (both)
- **use_rational** – [default='both'] if *all* or *hformat* is specified should hformat files be written using rational math or not. The default *both* is the legacy behaviour and writes both.

```
cbmpy.CBWrite.writeProteinCostToCSV(fba, fname)
```

Writes the protein costs 'CBM_PEPTIDE_COST' annotation to a csv file.

Parameters

- **fba** – an instantiated FBA object

- **fname** – the exported file name

`cbmpy.CBWrite.WriteFVAtCSV(id, fva, names, Dir=None, fbaObj=None)`

INFO: this method will be deprecated please update your scripts to use “`writeFVAtCSV()`”

`cbmpy.CBWrite.writeFVAtCSV(fvadata, names, fname, Dir=None, fbaObj=None)`

Takes the results of a FluxVariabilityAnalysis method and writes it to a nice csv file. Note this method replaces the `glpk/cplx_WriteFVAtCSV` methods.

Parameters

- **fvadata** – FluxVariabilityAnalysis() OUTPUT_ARRAY
- **names** – FluxVariabilityAnalysis() OUTPUT_NAMES
- **fname** – filename_base for the CSV output
- **Dir** – [default=None] if set the output directory for the csv files
- **fbaObj** – [default=None] if supplied adds extra model information into the output tables

`cbmpy.CBWrite.WriteFVAdata(fva, names, fname, work_dir=None, roundec=None, scale_min=False, appendfile=False, info=None)`

INFO: this method will be deprecated please update your scripts to use “`writeFVAdata()`”

`cbmpy.CBWrite.writeFVAdata(fvadata, names, fname, work_dir=None, roundec=None, scale_min=False, appendfile=False, info=None)`

Takes the results of a FluxVariabilityAnalysis method and writes it to a nice csv file. Note this method replaces the `glpk/cplx_WriteFVAtCSV` methods. Data is output as a csv file with columns: FluxName, FVA_MIN, FVA_MAX, OPT_VAL, SPAN

Parameters

- **fvadata** – FluxVariabilityAnalysis() FVA OUTPUT_ARRAY
- **names** – FluxVariabilityAnalysis() FVA OUTPUT_NAMES
- **fname** – filename_base for the CSV output
- **work_dir** – [default=None] if set the output directory for the csv files
- **roundec** – [default=None] an integer indicating at which decimal to round off output. Default is no rounding.
- **scale_min** – [default=False] normalise each flux such that that FVA_MIN = 0.0
- **appendfile** – [default=False] instead of opening a new file try and append the data
- **info** – [default=None] a string added to the results as an extra column, useful with *appendfile*

`cbmpy.CBWrite.writeSolutions(fname, sols=[], sep=',', extra_output=None, fba=None)`

Write 2 or more solutions where a solution is a dictionary of flux:value pairs:

Parameters

- **fname** – the export filename
- **sols** – a list of dictionaries containing flux:value pairs (e.g. output by `cmod.getReactionValues()`)
- **sep** – [default=','] the column separator
- **extra_output** – [default=None] add detailed information to output e.g. reaction names by giving a `CBModel` object as an argument to *extra_output*.
- **fba** – an fba model that can be used for *extra_output*

`cbmpy.CBWrite.generateBGID(num, prefix)`

Create a BGID generator, which is <prefix><num> where prefix is two letters num is padded to 6 figures.

Parameters

- **num** – the starting number
- **prefix** – the two letter prefix

`cbmpy.CBWrite.convertFloatToExcel(num, roundoff)`

Converts a float to Excel compatible “number”.

Parameters

- **num** – a number
- **roundoff** – the number of roundoff digits for `round()`

`cbmpy.CBWrite.convertExcelToFloat(num)`

Converts an Excel “number” to a float.

Parameters

num – a number

`cbmpy.CBWrite.writeModelToExcel97(fba, filename, roundoff=6)`

Exports the model as an Excel 97 spreadsheet.

Parameters

- **fba** – a CBMPy model instance
- **filename** – the filename of the workbook
- **roundoff** – [default=6] the number of digits to round off to

```
cbmpy.CBWrite.writeModelToCOMBINEarchive(mod, fname=None, directory=None,
                                           sbmlname=None, withExcel=True,
                                           vc_given='CBMPy', vc_family='Software',
                                           vc_email=None,
                                           vc_org='cbmpy.sourceforge.net',
                                           add_cbmpy_annot=True,
                                           add_cobra_annot=True)
```

Write a model in SBML and Excel format to a COMBINE archive using the following information:

Parameters

- **mod** – a model object
- **fname** – the output base filename, archive will be <fname>.zip
- **directory** – [default=None] created the combine archive ‘directory’
- **sbmlname** – [default=None] If *sbmlname* is defined then SBML file is <sbmlname>.xml otherwise sbml will be <fname>.xml.
- **withExcel** – [default=True] include a human readable Excel spreadsheet version of the model
- **vc_given** – [default='CBMPy'] first name
- **vc_family** – [default='Software'] family name
- **vc_email** – [default=None] email
- **vc_org** – [default=None] organisation
- **add_cbmpy_annot** – [default=True] add CBMPy KeyValueType annotation. Replaces <notes>
- **add_cobra_annot** – [default=True] add COBRA <notes> annotation

2.11 CBMPy: CBXML module

PySCeS Constraint Based Modelling (<http://cbmpy.sourceforge.net>) Copyright (C) 2009-2024 Brett G. Olivier, VU University Amsterdam, Amsterdam, The Netherlands

This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program. If not, see <<http://www.gnu.org/licenses/>>

Author: Brett G. Olivier PhD Contact developers: <https://github.com/SystemsBioinformatics/cbmpy/issues> Last edit: \$Author: bgoli \$ (\$Id: CBXML.py 710 2020-04-27 14:22:34Z bgoli \$)

```
cbmpy.CBXML.SBML_NS =
[('http://www.sbml.org/sbml/level3/version1/fbc/version3', 'L3V1FBC3'),
('http://www.sbml.org/sbml/level3/version1/fbc/version2', 'L3V1FBC2'),
('http://www.sbml.org/sbml/level3/version1/fbc/version1', 'L3V1FBC1'),
('http://www.sbml.org/sbml/level3/version2/fbc/version2', 'L3V2FBC2'),
('http://www.sbml.org/sbml/level3/version2/fbc/version1', 'L3V2FBC1'),
('http://www.sbml.org/sbml/level3/version2/core', 'L3V2core'),
('http://www.sbml.org/sbml/level3/version1/core', 'L3V1core'),
('http://www.sbml.org/sbml/level2/version4', 'L2'),
('http://www.sbml.org/sbml/level2', 'L2')]

print libsbml.BQB_ENCODES , 8 # “encodes”, print libsbml.BQB_HAS_PART
, 1 # “hasPart”, print libsbml.BQB_HAS_PROPERTY , 10 # “hasProperty”,
print libsbml.BQB_HAS_VERSION , 4 # “hasVersion”, print libsbml.BQB_IS
, 0 # “isA”, print libsbml.BQB_IS_DESCRIBED_BY , 6 # “isDescribedBy”,
print libsbml.BQB_IS_ENCODED_BY , 7 # “isEncodedBy”, print libsbml.BQB_IS_HOMOLOG_TO , 5 # “isHomologTo”, print libsbml.BQB_IS_PART_OF
, 2 # “isPartOf”, print libsbml.BQB_IS_PROPERTY_OF , 11 # “isPropertyOf”, print
libsbml.BQB_IS_VERSION_OF , 3 # “isVersionOf”, print libsbml.BQB_OCCURS_IN , 9
# “occursIn”, print libsbml.BQB_UNKNOWN , 12 # None

print libsbml.BQM_IS , 0 # None print libsbml.BQM_IS_DERIVED_FROM , 2 # None print
libsbml.BQM_IS_DESCRIBED_BY , 1 # None print libsbml.BQM_UNKNOWN , 3 # None
```

class cbmpy.CBXML.MLStripper

Bases: [HTMLParser](#)

Class for stripping a string of HTML/XML used from:

<http://stackoverflow.com/questions/753052/strip-html-from-strings-in-python>

__init__()

Strip a string of HTML/XML tags.

handle_data(*d*)

get_data()

cbmpy.CBXML.xml_stripTags(*html*)

Strip HTML/XML tags from a string.

Parameters

html – the string containing HTML

Returns

a string with all HTML/XML tags removed.

`cbmpy.CBXML.formatSbmlId(s)`

Format a string to be a valid SBML identifier. Convert a string to a valid SBML identifier by replacing non-alphanumeric characters with underscores and prefixing with underscore if it doesn't start with a letter or underscore.

Parameters

s – the string to format

Returns

a valid SBML identifier.

`cbmpy.CBXML.sbml_readSBML2FBA(fname, work_dir=None, return_sbml_model=False, fake_boundary_species_search=False)`

Read in an SBML Level 2 file with FBA annotation where and return either a CBM model object or a (cbm_mod, sbml_mod) pair if return_sbml_model=True.

Parameters

- **fname** – is the filename
- **work_dir** – is the working directory (only used if not None)
- **return_sbml_model** – [default=False] return a (cbm_mod, sbml_mod) pair
- **fake_boundary_species_search** – [default=False] after looking for the boundary_condition of a species search for overloaded id's <id>_b

`cbmpy.CBXML.xml_createSBML2FBADoc()`

Create a 'document' to store the SBML2FBA annotation, returns:

Parameters

DOC – a minidom document

`cbmpy.CBXML.xml_viewSBML2FBAXML(document, fname=None)`

Print a minidom XML document to screen or file, arguments:

Parameters

- **document** – a minidom XML document
- **fname** – [default=None] by default print to screen or write to file fname

`cbmpy.CBXML.xml_addSBML2FBAFluxBound(document, rid, operator, value, fbid=None)`

Adds an SBML3FBA flux bound to the document:

Parameters

- **document** – a minidom XML document created by `xml_createSBML2FBADoc`

- **rid** – the reaction id
- **operator** – one of ['greater', 'greaterEqual', 'less', 'lessEqual', 'equal', '>', '>=', '<', '<=', '=']
- **value** – a float which will be cast to a string using str(value)
- **fbid** – the flux bound id, autogenerated by default

`cbmpy.CBXML.xml_createListOfFluxObjectives(document, l)`

Create a list of fluxObjectives to add to an Objective.

Parameters

- **document** – a minidom XML document created by `xml_createSBML2FBADoc`
- **l** – a list of (reaction_id, coefficient) tuples to add as flux objectives

Returns

the listOfFluxes element.

`cbmpy.CBXML.xml_createSBML2FBAObjective(document, oid, sense, fluxObjectives)`

Create an SBML objective element with fluxObjectives. Create an objective element with a list of fluxObjectives added to it.

Parameters

- **document** – a minidom XML document created by `xml_createSBML2FBADoc`
- **oid** – the objective ID
- **sense** – a string containing the objective sense ('maximize' or 'minimize')
- **fluxObjectives** – a list of (reaction_id, coefficient) tuples

Returns

the objective element.

`cbmpy.CBXML.xml_addSBML2FBAObjective(document, objective, active=True)`

Adds an objective element to the documents listOfObjectives and sets the active attribute:

Parameters

- **document** – a minidom XML document created by `xml_createSBML2FBADoc`
- **objective** – a minidom XML objective element created with `xml_createSBML2FBAObjective`
- **active** – [default=True] a boolean flag specifying whether this objective is active

`cbmpy.CBXML.xml_getSBML2FBAannotation(fba, fname=None)`

Takes an FBA model object and returns the SBML3FBA annotation as an XML string:

Parameters

- **fba** – an fba model object
- **fname** – [default=None] if supplied the XML will be written to file *fname*

`cbmpy.CBXML.sbml_createModelL2(fba, level=2, version=1)`

Create an SBML model and document:

Parameters

- **fba** – a PySCeSCBM model instance
- **level** – always 2
- **version** – always 1
- **model** – an SBML model

`cbmpy.CBXML.sbml_setCompartmentsL2(model, compartments)`

`cbmpy.CBXML.sbml_setCompartmentsL3(model, fba)`

Sets the model compartments.

Parameters

- **model** – a libSBML model instance
- **fba** – a PySCeSCBM model instance

`cbmpy.CBXML.sbml_setParametersL3Fbc(fbcmo, add_cbmpy_anno=True, fbc_version=2)`

Add non fluxbound related parameters to the model.

Parameters

- **fbcmo** – a CBM2SBML instance
- **add_cbmpy_anno** – [default=True] add CBMPy KeyValueTypeData annotation.

`cbmpy.CBXML.sbml_setAnnotationsL3Fbc(cbmo, sbmlo)`

Add CBMPy Fbase annotations to an SBML object, MIRIAM, SBO, Notes. Should be called last when creating SBML objects.

Parameters

- **cbmo** – the CBMPy object
- **sbmlo** – SBML object

`cbmpy.CBXML.sbml_setDescription(model, fba)`

Set the model description as an HTML paragraph in a <note> element. Sets the model description as a <note> containing HTML in a paragraph element on the model object.

Parameters

- **model** – a libSBML model instance
- **fba** – a PySCeSCBM model instance

Returns

None (modifies model in place).

`cbmpy.CBXML.sbml_setNotes3(obj, s)`

Set notes for an SBML object. Formats the CBMPy notes as an SBML note and adds it to the SBML object.

Parameters

- **obj** – an SBML object
- **s** – a string that should be added as a note

Returns

True if notes were set successfully, False otherwise.

`cbmpy.CBXML.sbml_getNotes(obj)`

Get the notes from an SBML object.

Parameters

obj – an SBML object

Returns

the notes string, or an empty string if no notes exist.

`cbmpy.CBXML.sbml_setUnits(model, units=None, give_default=False, L3=True)`

Set units for the model. Adds units to the model or returns the default unit dictionary.

Parameters

- **model** – a libSBML model instance
- **units** – [default=None] a dictionary of units (if None, default units are used)
- **give_default** – [default=False] if True, returns the default unit dictionary
- **L3** – [default=True] use SBML Level 3 defaults

Returns

the units dictionary if give_default is True, otherwise None.

`cbmpy.CBXML.sbml_writeAnnotationsAsCOBRANote(annotations)`

Write a COBRA-compatible SBML <note> element from an annotations dictionary.

Parameters

annotations – a dictionary of annotation keys to values

Returns

the formatted annotation string.

`cbmpy.CBXML.sbml_setSpeciesL2(model, fba, return_dicts=False)`

Add species definitions to an SBML Level 2 model.

Parameters

- **model** – a libSBML model instance (or None if `return_dicts=True`)
- **fba** – a PySCeSCBM model instance
- **return_dicts** – [default=False] if True, only return compartments and species dicts

Returns

the compartments dictionary (or (compartments, species) tuple if `return_dicts=True`).

`cbmpy.CBXML.sbml_setReactionsL2(model, fba, return_dict=False)`

Add FBA instance reactions to an SBML Level 2 model.

Parameters

- **model** – an SBML model instance
- **fba** – a PySCeSCBM model instance
- **return_dict** – [default=False] if True, return reaction dictionaries without adding to SBML

Returns

reaction dictionaries if `return_dict` is True, otherwise None.

`cbmpy.CBXML.sbml_exportSBML2FBAModel(document, filename, directory=None, return_doc=False, remove_note_body=False)`

Writes an SBML model object to file. Note this is an internal SBML method use `sbml_writeSBML2FBA()` to write an FBA model:

Parameters

- **model** – a libSBML model instance
- **filename** – the output filename
- **directory** – [default=None] by default use filename otherwise join, <dir><filename>

- **return_doc** – [default=False] return the SBML document used to write the XML

`cbmpy.CBXML.sbml_writeSBML2FBA(fba, fname, directory=None, sbml_level_version=None)`

Takes an FBA model object and writes it to file as SBML L3 FBA:

Parameters

- **fba** – an fba model object
- **fname** – the model will be written as XML to *fname*
- **directory** – [default=None] if defined it is prepended to *fname*
- **sbml_level_version** – [default=None] a tuple containing the SBML level and version e.g. (2,4) (ignored)

`class cbmpy.CBXML.FBCconnect(fbc_version=2, fbc_strict=True, enable_groups=False)`

Bases: `object`

`metaprefix = 'meta_'`

`sbmllevel = 3`

`sbmlversion = 1`

`groupsversion = 1`

`maxobjname = ('maximize', 'maximise', 'max')`

`minobjname = ('minimize', 'minimise', 'min')`

`groupList = None`

`__init__(fbc_version=2, fbc_strict=True, enable_groups=False)`

Connect SBML Packages.

Parameters

- **fbc_version** – [default=2] fbc version
- **fbc_strict** – [default=True] if using FBC V2 set the strict flag
- **enable_groups** – [default=False] use the Groups package if possible

`sbml = None`

`fbcversion = 2`

`sbmlns = None`

`doc = None`

model = None

fbc = None

groups = None

GROUPS_AVAILABLE = False

fbcstrict = True

createGroup(*grp*)

Adds a group to the SBML model.

Parameters

grp – the cbmpy group object

createGroupLegacy(*gid*, *group*, *name=None*)

Tries to add a group to the model if (assuming the groups package is available).

Parameters

- **gid** – the group name
- **groups** – a list of strings that refer to model SId's

createFluxBoundV1(*fid*, *reaction*, *operation*, *value*)

createObjective(*oid*, *osense*, *flux_objs*, *name=None*, *active=True*)

Create and add the FBC Objective function. Returns the SBML objective object.

Parameters

- **oid** – objective id
- **osense** – objective sense
- **flux_objs** – [(reaction, coefficient, type)]
- **name**
- **active**

createGeneAssociationV1(*rid*, *assoc*, *gprid=None*)

Create a gene association for a specified reaction and an association string.

Parameters

- **rid** – a reaction id
- **assoc** – the association string e.g.(b0698 and b0697) or (b0696)

writeToFile(*filename*)

```
class cbmpy.CBXML.CBMtoSBML3(fba, fbc_version=2, fbc_strict=True,  
                             enable_groups=False)
```

Bases: [FBCconnect](#)

bound_registry = None

parameter_cntr = 0

```
__init__(fba, fbc_version=2, fbc_strict=True, enable_groups=False)
```

Convert a CBM model to SBML level 3 with FBC.

fba = None

parameter_map = None

```
addModelAnnotation(fba)
```

```
addModelHistory()
```

If they exist add CBM model history information to SBML model.

```
addBoundsV2(compress_bounds=False, sig_dig=20)
```

Add FBC V2 style fluxbounds to model.

Parameters

- **compress_bounds** – [default=False] enable parameter compression
- **sig_dig** – [default=20] round off to significant digits

```
addBoundsV1(autofix=False)
```

Add the CBM fluxbounds to SBML.

Parameters

autofix – convert '< >' to '<= >=' default = False

```
addObjectives(add_cbmpy_anno=True)
```

Add the CBM objective function to SBML.

```
addGenesV2(parse_from_annotation=False, annotation_key='GENE ASSOCIATION',  
          add_cbmpy_anno=True)
```

Create genes and add to SBML models (FBC V2)

Parameters

- **parse_from_annotation** – [default=False] if the gpr_assoc list is empty e.g. the GPR associations have
- **annotation_key** – [default='GENE ASSOCIATION'] the key to use for the gene association if parse_from_annotation is True
- **add_cbmpy_anno** – [default=True] add PySCeS CBMPy annotation

```
addGeneProteinAssociationsV1(parse_from_annotation=False,  
                               annotation_key='GENE ASSOCIATION',  
                               add_cbmpy_anno=True)
```

Add the CBM geneProtein associations to SBML from the gpr_assoc list (FBC V1)

Parameters

- **parse_from_annotation** – [default=False] if the gpr_assoc list is empty e.g. the GPR associations have
- **annotation_key** – [default='GENE ASSOCIATION'] the key to use for the gene association if parse_from_annotation is True
- **add_cbmpy_anno** – add PySCeS CBMPy annotation (still to be implemented)

```
addUserDefinedConstraintsV3(add_cbmpy_anno=True)
```

Add FBC V3 user defined constraints.

```
createParParameter(param, add_cbmpy_anno=True)
```

Create a generic SBML parameter from a CBMPy parameter.

Parameters

- **param** – a CBMPy parameter object
- **add_cbmpy_anno** – [default=True] add annotation to SBML object

```
createFbParameterV2(bnd, add_cbmpy_anno=True)
```

Create SBML V2 flux bound parameters for reaction.

Parameters

bnd – object

```
createFbParameterSharedV2(pid, value)
```

Create SBML V2 flux bound parameters for reaction.

Parameters

- **value** – parameter value
- **pid** – optional parameter id

```
cbmpy.CBXML.sbml_writeKeyValueDataAnnotation(annotations)
```

Writes the key:value annotations as a KeyValueData annotation (<http://pysces.sourceforge.net/KeyValueData>)

```
cbmpy.CBXML.sbml_readKeyValueDataAnnotation(annotations)
```

Reads KeyValueData annotation (<http://pysces.sourceforge.net/KeyValueData>) and returns a dictionary of key:value pairs

`cbmpy.CBXML.sbml_readFBCv3KeyValuePairs(fbcv)`

Reads FBCv3 Key Value pair annotation and returns a dictionary of key:value pairs.

Parameters

fbcv – an FBC plugin

`cbmpy.CBXML.sbml_setSpeciesL3(model, fba, return_dicts=False, add_cobra_anno=False, add_cbmpy_anno=True, substance_units=True, fbc_version=2)`

Add species definitions to an SBML Level 3 FBC model.

Parameters

- **model** – an SBML model instance (or None if `return_dicts=True`)
- **fba** – a PySCeSCBM model instance
- **return_dicts** – [default=False] if True, only return compartments and species dicts
- **add_cbmpy_anno** – [default=True] add CBMPy Key Value Data annotation
- **add_cobra_anno** – [default=False] add COBRA <notes> annotation
- **substance_units** – [default=True] use amounts instead of concentrations
- **fbc_version** – [default=2] the FBC version to use

Returns

the compartments dictionary (or (compartments, species) tuple if `return_dicts=True`).

`cbmpy.CBXML.sbml_setReactionsL3Fbc(fbcmod, return_dict=False, add_cobra_anno=False, add_cbmpy_anno=True, fbc_version=2)`

Add the FBA instance reactions to the SBML model.

Parameters

- **fbcmod** – a CBM2SBML instance
- **return_dict** – [default=False] if True do not add reactions to SBML document instead return a dictionary description of the reactions
- **add_cbmpy_anno** – [default=True] add CBMPy Key Value Data annotation. Replaces <notes>
- **add_cobra_anno** – [default=False] add COBRA <notes> annotation
- **fbc_version** – [default=2] writes either FBC v1 (2013) or v2 (2015) or v3 (2023)

`cbmpy.CBXML.sbml_setFBCv3KeyValuePairs(fbcv, kv_pairs)`

Adds key value pairs to any FBCv3 SBase derived object.

Parameters

- **fbcv** – an FBCv3 plugin
- **kv_pairs** – a dictionary of CBMPy Key Value pairs (format will be upgraded over next few versions)

`cbmpy.CBXML.sbml_setGroupsL3(cs, fba)`

Add groups to the SBML model.

Parameters

- **cs** – a CBMLtoSBML instance
- **fba** – a CBMPy model instance

`cbmpy.CBXML.sbml_writeCOBRASBML(fba, fname, directory=None)`

Write an FBA model as COBRA-compatible SBML. Takes an FBA model object and writes it to a file as COBRA-compatible SBML by first writing as SBML L3 FBC, then converting to COBRA format.

Parameters

- **fba** – an FBA model object
- **fname** – the output filename
- **directory** – [default=None] if provided, prepended to fname

Returns

None (writes to file).

`cbmpy.CBXML.sbml_getGeneRefs(association, out)`

Walk through a gene association and extract GeneRefs inspired by Frank.

`cbmpy.CBXML.sbml_getGPRasDictFBCv1(node, out)`

Converts a GPR string ‘((g1 and g2) or g3)’ to a gprDict which is returned.

Parameters

- **node** – a Python AST note (e.g. *ast.parse(gprstring).body[0]*)
- **out** – a new dictionary that will be created in place

`cbmpy.CBXML.sbml_getGPRasDictFBCv2(association, out, cntr)`

Walk through an SBML L3FBCV2 gene protein association and return a dictionary/tree representation.

`cbmpy.CBXML.sbml_createAssociationFromAST(node, out)`

Converts a GPR string ‘((g1 and g2) or g3)’ to an association via a Python AST. In future I will get rid of all the string elements and work only with associations and AST’s.

Parameters

- **node** – a Python AST note (e.g. body)
- **out** – a new shiny FBC V2 GeneProductAssociation

`cbmpy.CBXML.sbml_createAssociationFromTreeV2(tree, out)`

Converts a GPR tree to an association.

Parameters

- **tree** – a GPR dict tree
- **out** – a new shiny FBC V2 GeneProductAssociation

`cbmpy.CBXML.sbml_writeSBML3FBC(fba, fname, directory=None, sbml_level_version=(3, 1),
 autofix=True, return_fbc=False, gpr_from_annot=False,
 add_groups=False, add_cbmpy_annot=True,
 add_cobra_annot=False, xoptions={})`

Takes an FBA model object and writes it to file as SBML L3 FBC:

Parameters

- **fba** – an fba model object
- **fname** – the model will be written as XML to *fname*
- **directory** – [default=None] if defined it is prepended to *fname*
- **sbml_level_version** – [default=(3,1)] a tuple containing the SBML level and version e.g. (3,1)
- **autofix** – convert <> to <=>=
- **return_fbc** – return the FBC converter instance
- **gpr_from_annot** – [default=False] if enabled will attempt to add the gene protein associations from the annotations
- **add_cbmpy_annot** – [default=True] add CBMPy KeyValueData annotation. Replaces <notes>
- **add_cobra_annot** – [default=True] add COBRA <notes> annotation
- **xoptions** – extended options
- **fbc_version** – [default=2] write SBML3FBC using version 1 (2013) or version 2 (2015) or version (2023)
- **validate** – [default=False] validate the output SBML file

- **compress_bounds** – [default=False] try compress output flux bound parameters
- **zip_model** – [default=False] compress the model using ZIP encoding
- **return_model_string** – [default=False] return the SBML XML file as a string

`cbmpy.CBXML.sbml_setValidationOptions(D, level)`

Set the validation level of an SBML document.

Parameters

- **D** – an SBML document
- **level** – the level of consistency check can be either one of:

`cbmpy.CBXML.sbml_readCOBRASBML(fname, work_dir=None, return_sbml_model=False, delete_intermediate=False, fake_boundary_species_search=False, output_dir=None, speciesAnnotationFix=True, skip_genes=False)`

Read in a COBRA format SBML Level 2 file with FBA annotation where and return either a CBM model object or a (cbm_mod, sbml_mod) pair if return_sbml_model=True.

Parameters

- **fname** – is the filename
- **work_dir** – is the working directory
- **return_sbml_model** – [default=False] return a (cbm_mod, sbml_mod) pair
- **delete_intermediate** – [default=False] delete the intermediate SBML Level 3 FBC file
- **fake_boundary_species_search** – [default=False] after looking for the boundary_condition of a species search for overloaded id's <id>_b
- **output_dir** – [default=None] the directory to output the intermediate SBML L3 files (if generated) default to input directory
- **speciesAnnotationFix** – [default=True]
- **skip_genes** – [default=False] convert GPR associations

`cbmpy.CBXML.sbml_convertCOBRASBMLtoFBC(fname, outname=None, work_dir=None, output_dir=None)`

Read in a COBRA SBML Level 2 file and return the name of the created SBML Level 3 with FBC file that is created in the output directory.

Parameters

- **fname** – is the filename
- **outname** – the name of the output file. If not specified then <file-name>.l3fbc.xml is used as default
- **work_dir** – [default=None] is the working directory
- **output_dir** – [default=None] is the output directory (default is work_dir)

`cbmpy.CBXML.sbml_convertSBML3FBCToCOBRA(fname, outname=None, work_dir=None, output_dir=None)`

Read in a SBML Level 3 file and return the name of the created COBRA file that is created in the output directory.

Parameters

- **fname** – is the filename
- **outname** – the name of the output file. If not specified then <file-name>.cobra.xml is used as default
- **work_dir** – [default=None] is the working directory
- **output_dir** – [default=None] is the output directory (default is work_dir)

`cbmpy.CBXML.sbml_validateDocument(D, fullmsg=False, docread=False)`

Validates and SBML document returns three dictionaries, errors, warnings, other and a boolean indicating an invalid document:

Parameters

- **D** – and SBML document
- **fullmsg** – [default=False] optionally display the full error message

`cbmpy.CBXML.sbml_fileValidate(f, level='normal')`

Validate an SBML file and model.

Parameters

- **f** – the SBML file
- **level** – [default='normal'] the level of validation “normal” or “full”

`cbmpy.CBXML.sbml_fileFindVersion(f)`

Try and find the SBML version and FBC support.

Parameters

- f** – the SBML file

`cbmpy.CBXML.setCBSB0term(sbo, obj)`

Given an SBOterm from libSBML, add it to a CBMPy object.

Parameters

- **sbo** – the sbo term string
- **obj** – the CBMPy Fbase derived object

`cbmpy.CBXML.with_metaclass(meta, *bases)`

Create a base class with a metaclass.

Usage is: `NewClass(with_metaclass(MetaClass, BaseClass*))`

`cbmpy.CBXML.sbml_readSBML3FBC(fname, work_dir=None, return_sbml_model=False, xoptions={})`

Read in an SBML Level 3 file with FBC annotation where and return either a CBM model object or a (cbm_mod, sbml_mod) pair if `return_sbml_model=True`.

Parameters

- **fname** – is the filename
- **work_dir** – is the working directory
- **return_sbml_model** – [default=False] return a (cbm_mod, sbml_mod) pair
- **xoptions** – special load options enable with option = True
- **nogenes** – do not load/process genes
- **noannot** – do not load/process any annotations
- **validate** – validate model and display errors and warnings before loading
- **readcobra** – read the cobra annotation
- **read_model_string** – [default=False] read the model from a string (instead of a filename) containing an SBML document
- **nmatrix_type** – [default='normal'] define the type of stoichiometric matrix to be built
- **model_extension_class** – extend CBModel class with new class (experimental, Python 3 only)
- **model_metaclass** – add a custom metaclass to CBModel (experimental, Python 3 only)

`cbmpy.CBXML.sbml_readCOBRANote(s)`

Parses a COBRA style note from a XML string.

Parameters**s** – an XML string`cbmpy.CBXML.sbml_getCVterms(sb, model=False)`

Get the MIRIAM compliant CV terms and return a MIRIAMAnnotation or None.

Parameters

- **sb** – a libSBML SBase derived object
- **model** – is this a BQmodel term

`cbmpy.CBXML.sbml_setCVterms(sb, uridict, model=False)`

Add MIRIAM compliant CV terms to a sbml object from a CBM object.

Parameters

- **sb** – a libSBML SBase derived object
- **uridict** – a dictionary of uri's as produced by `getAllMIRIAMUris()`
- **model** – is this a BQmodel term [deprecated attribute, ignored and autodetected]

Network Analysis —-=-=-=-=—

2.12 CBMPy: CNetDB module

PySCeS Constraint Based Modelling (<http://cbmpy.sourceforge.net>) Copyright (C) 2009-2024 Brett G. Olivier, VU University Amsterdam, Amsterdam, The Netherlands

This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program. If not, see <<http://www.gnu.org/licenses/>>

Author: Brett G. Olivier PhD Contact developers: <https://github.com/SystemsBioinformatics/cbmpy/issues> Last edit: \$Author: bgoli \$ (\$Id: CNetDB.py 710 2020-04-27 14:22:34Z bgoli \$)

class `cbmpy.CNetDB.NetDBbase`Bases: `object``text_encoding = 'utf8'`

URLEncode(*txt*)

URL encodes a string.

URLDecode(*txt*)

Decodes a URL encoded string.

class cbmpy.CBNetDB.DBTools

Bases: *NetDBbase*

Tools to work with SQLite DB's (optimized, no SQL required).

db_conn = None

db_cursor = None

__init__()

sqlite = None

sqlite_version = None

db_tables = None

connectSQLiteDB(*db_name*, *work_dir=None*)

Connect to a sqlite database.

Parameters

- **db_name** – the name of the sqlite database
- **work_dir** – the optional database path

commitDB()

Commits all curent changes to DB, returns a boolean.

createDBTable(*table*, *sqlcols*)

Create a database table if it does not exist:

Parameters

- **table** – the table name
- **sqlcols** – a list containing the SQL definitions of the table columns:
<id> <type> for example [*'gene TEXT PRIMARY KEY'*, *'aa_seq TEXT'*,
'nuc_seq TEXT', *'aa_len INT'*, *'nuc_len INT'*]

insertData(*table*, *data*, *commit=True*)

Insert data into a table: "INSERT INTO %s (?, ?, ?, ?, ?) VALUES (?, ?, ?, ?, ?)" %
table, (?, ?, ?, ?, ?)))

Parameters

- **table** – the DB table name

- **data** – a dictionary of {id:value} pairs
- **commit** – whether to commit the data insertions

updateData(*table, col, rid, data, commit=True*)

Update already defined data.

Parameters

- **table** – the table name
- **col** – the column name
- **rid** – the row id to update
- **data** – a dictionary of {id:value} pairs
- **commit** – whether to commit the data updates

checkEntryInColumn(*table, col, rid*)

Check if an entry exists in a table.

Parameters

- **table** – the table name
- **col** – the column name
- **rid** – the row to search for

executeSQL(*sql*)

Execute a SQL command:

Parameters

- sql** – a string containing a SQL command

getColumns(*table, cols*)

Fetch the contents of one or more columns of data in a table.

Parameters

- **table** – the database table
- **cols** – a list of one or more column id's

getRow(*table, col, rid*)

Get the table row(s) which correspond to rid in column. Returns the row(s) as a list, if the column is the primary key this is always a single entry.

Parameters

- **table** – the database table
- **col** – the column id
- **rid** – the row index id

getCell(*table, col, rid, cell*)

Get the table cell which correspond to rid in column. Returns the value or None.

Parameters

- **table** – the database table
- **col** – the column id
- **rid** – the row index id
- **cell** – the column of the cell you want tp extract

getTable(*table, colOut=False*)

Returns an entire database table.

Parameters

- **table** – the table name
- **colOut** – optionally return a tuple of (data,ColNames)

dumpTableToTxt(*table, filename*)

Save a table as tab separated txt file.

Parameters

- **table** – the table to export
- **filename** – the filename of the table dump

dumpTableToCSV(*table, filename*)

Save a table as tab separated txt file.

Parameters

- **table** – the table to export
- **filename** – the filename of the table dump

fetchAll(*sql*)

Raw SQL query e.g. ‘SELECT id FROM gene WHERE gene=”G”’.

closeDB()

Close the DB connection and reset the DBTools instance (can be reconnected)

class cbmpy.CBNetDB.**KeGGTools**(*url*)

Bases: `object`

Class that holds useful methods for querying KeGG via a SUDS provided soap client.

__init__(*url*)

Kclient = None

fetchSeqfromKeGG(*k_gene*)

Given a gene name try and retrieve the gene and amino acid sequence.

class cbmpy.CBNetDB.**KeGGSequenceTools**(*url, db_name, work_dir*)

Bases: [object](#)

Using the KeGG connector this class provides tools to construct an organisms specific sequence database.

__init__(*url, db_name, work_dir*)

DB = None

KEGG = None

buildGeneDatabase(*genes, tablename, UPDATE_IF_EXISTS=False, default_length=0*)

getPeptideLengthsFromDB(*genes, keg_prefix*)

class cbmpy.CBNetDB.**RESTClient**

Bases: [NetDBbase](#)

Class that provides the basis for application specific connectors to REST web services.

site_root = None

conn = None

history = ''

CONNECTED = False

USER_AGENT = 'Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:13.0)
Gecko/20100101 Firefox/13.0'

__init__()

Log(*txt*)

Add txt to logfile history.

Parameters

txt – a string

GetLog()

Return the logged history.

Connect(*root*)

Establish HTTP connection to.

Parameters

root – the site root “www.google.com”

Get(*query*)

Perform an http GET using:

Parameters

- **query** – e.g.
- **reply_mode** – [default=""] this is the reply mode

Close()

Close the currently active connection.

class cbmpy.CBNetDB.**MIRIAMTools**

Bases: `object`

Tools dealing with MIRIAM annotations.

MiriamURN2IdentifiersURL(*urn*)

class cbmpy.CBNetDB.**SemanticSBML**

Bases: `RESTClient`, `MIRIAMTools`

REST client for connecting to SemanticSBML services.

data = `None`

item_re = `re.compile('<item>.+?</item>')`

__init__()

quickLookup(*txt*)

Do a quick lookup for *txt* using SemanticSBML (connectic if required) and return results. Returns a list of identifiers.org id's in descending priority (as return)

Parameters

txt – the string to lookup

viewDataInWebbrowser(*maxres=10*)

Attempt to view #maxres results returned by SemanticSBML in the default browser.

Parameters

maxres – default maximum number of results to display.

parseXMLtoText(*xml*)

Parse the xml output by quickLookup() into a list of URL.

Parameters

xml – XML returns from SemanticSBML

2.13 Visualization

2.13.1 CBMPy: CBPlot module

PySCeS Constraint Based Modelling (<http://cbmpy.sourceforge.net>) Copyright (C) 2009-2024 Brett G. Olivier, VU University Amsterdam, Amsterdam, The Netherlands

This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program. If not, see <<http://www.gnu.org/licenses/>>

Author: Brett G. Olivier PhD Contact developers: <https://github.com/SystemsBioinformatics/cbmpy/issues> Last edit: \$Author: bgoli \$ (\$Id: CBPlot.py 710 2020-04-27 14:22:34Z bgoli \$)

```
cbmpy.CBPlot.plotFluxVariability(fva_data, fva_names, fname, work_dir=None,
                                title=None, ySlice=None, minHeight=None,
                                maxHeight=None, roundec=None, autoclose=True,
                                fluxval=True, type='png')
```

Plots and saves as an image the flux variability results as generated by CB-Solver.FluxVariabilityAnalysis.

- *fva_data* FluxVariabilityAnalysis() FVA OUTPUT_ARRAY
- *fva_names* FluxVariabilityAnalysis() FVA OUTPUT_NAMES
- *fname* filename_base for the CSV output
- *work_dir* [default=None] if set the output directory for the csv files
- *title* [default=None] the user defined title for the graph
- *ySlice* [default=None] this sets an absolute (fixed) limit on the Y-axis (+- ySlice)
- *minHeight* [default=None] the minimum length that defined a span
- *maxHeight* [default=None] the maximum length a span can obtain, bar will be limited to maxHeight and coloured yellow
- *roundec* [default=None] an integer indicating at which decimal to round off output. Default is no rounding.
- *autoclose* [default=True] autoclose plot after save
- *fluxval* [default=True] plot the flux value

- `type` [default='png'] the output format, depends on matplotlib backend e.g. 'png', 'pdf', 'eps'

2.14 GUI Frontends

2.14.1 CBMPy: CBGUI module

PySCeS Constraint Based Modelling (<http://cbmpy.sourceforge.net>) Copyright (C) 2009-2024 Brett G. Olivier, VU University Amsterdam, Amsterdam, The Netherlands

This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program. If not, see <<http://www.gnu.org/licenses/>>

Author: Brett G. Olivier PhD Contact developers: <https://github.com/SystemsBioinformatics/cbmpy/issues> Last edit: \$Author: bgoli \$ (\$Id: CBGUI.py 710 2020-04-27 14:22:34Z bgoli \$)

`cbmpy.CBGUI.loadCBGUI(mod, version=2)`

Load an FBA model instance into the quick editor to view or change basic model properties.

- `mod` a PySCeS CBMPy model instance

2.14.2 CBMPy: CBWx module

PySCeS Constraint Based Modelling (<http://cbmpy.sourceforge.net>) Copyright (C) 2009-2024 Brett G. Olivier, VU University Amsterdam, Amsterdam, The Netherlands

This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program. If not, see <<http://www.gnu.org/licenses/>>

Author: Brett G. Olivier PhD Contact developers: <https://github.com/SystemsBioinformatics/cbmpy/issues> Last edit: \$Author: bgoli \$ (\$Id: CBWx.py 710 2020-04-27 14:22:34Z bgoli \$)

Note

CBQt4 requires PyQT4 and may not be available in all environments.

2.15 Multi-processing

2.15.1 CBMPy: CBMultiCore module

PySCeS Constraint Based Modelling (<http://cbmpy.sourceforge.net>) Copyright (C) 2009-2024 Brett G. Olivier, VU University Amsterdam, Amsterdam, The Netherlands

This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program. If not, see <<http://www.gnu.org/licenses/>>

Author: Brett G. Olivier PhD Contact developers: <https://github.com/SystemsBioinformatics/cbmpy/issues> Last edit: \$Author: bgoli \$ (\$Id: CBMultiCore.py 710 2020-04-27 14:22:34Z bgoli \$)

```
cbmpy.CBMultiCore.grouper(3, 'abcdefg', 'x') --> ('a', 'b', 'c'), ('d', 'e', 'f'), ('g', 'x', 'x')
```

```
cbmpy.CBMultiCore.runMultiCoreFVA(fba, selected_reactions=None, pre_opt=True,
                                   tol=None, objF2constr=True, rhs_sense='lower',
                                   optPercentage=100.0, work_dir=None, quiet=True,
                                   debug=False, oldlpgen=False, markupmodel=True,
                                   procs=2, override_bin=None)
```

Run a multicore FVA where:

- *fba* is an fba model instance
- *procs* [default=2] number of processing threads (optimum seems to be about the number of physical cores)
- *python_override_bin* allows customization of the Python bin used for the multicore process

2.15.2 CBMPy: CBMultiModel module

PySCeS Constraint Based Modelling (<http://cbmpy.sourceforge.net>) Copyright (C) 2009-2024 Brett G. Olivier, VU University Amsterdam, Amsterdam, The Netherlands

This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program. If not, see <<http://www.gnu.org/licenses/>>

Author: Brett G. Olivier PhD Contact developers: <https://github.com/SystemsBioinformatics/cbmpy/issues> Last edit: \$Author: bgoli \$ (\$Id: CBTools.py 544 2024-01-12 16:31:50Z bgoli \$)

`cbmpy.CBMultiModel.copySpecies(m_src, m_targ, sid, is_boundary=False)`

Copy a species from a source model to a target model.

- *m_src* the source model
- *m_targ* the target model
- *sid* the species to copy
- *is_boundary* [default] add the species as a variable metabolite or boundary if True

`cbmpy.CBMultiModel.copyReaction(m_src, m_targ, rid, altrid=None)`

Copy a reaction from a source model to a target model, if the required species exist in the target then they are mapped as reagents, otherwise new metabolites are added as boundary species.

- *m_src* the source model
- *m_targ* the target model
- *rid* the reaction id to copy
- *altrid* if the reaction name exists in the target, try use this one instead

2.15.3 CBMPy: CBMultiEnv module

PySCeS Constraint Based Modelling (<http://cbmpy.sourceforge.net>) Copyright (C) 2009-2024 Brett G. Olivier, VU University Amsterdam, Amsterdam, The Netherlands

This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program. If not, see <<http://www.gnu.org/licenses/>>

Author: Brett G. Olivier PhD Contact developers: <https://github.com/SystemsBioinformatics/cbmpy/issues> Last edit: \$Author: bgoli \$ (\$Id: CBMultiEnv.py 710 2020-04-27 14:22:34Z bgoli \$)

```
cbmpy.CBMultiEnv.addProteinCostAnnotation(F, reaction_gene_map,  
                                           gene_peptide_map,  
                                           gene_annotation_key='GENE  
ASSOCIATION')
```

```
cbmpy.CBMultiEnv.scaleProteinCostAnnotation(fba)
```

```
cbmpy.CBMultiEnv.runMultiStateFBA(input_list, dev_factor, options, dofva=False,  
                                  fva_selected_reactions=None)
```

2.16 Model Utilities

2.16.1 CBMPy: CBModelTools module

PySCeS Constraint Based Modelling (<http://cbmpy.sourceforge.net>) Copyright (C) 2009-2024 Brett G. Olivier, VU University Amsterdam, Amsterdam, The Netherlands

This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A particular PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program. If not, see <<http://www.gnu.org/licenses/>>

Author: Brett G. Olivier PhD Contact developers: <https://github.com/SystemsBioinformatics/cbmpy/issues> Last edit: \$Author: bgoli \$ (\$Id: CBModelTools.py 710 2020-04-27 14:22:34Z bgoli \$)

```
cbmpy.CBModelTools.addReversibilityBounds(reactions, bounds, infinity=inf)
```

Add reversibility bounds to the bounds dictionary.

Parameters

- **reactions** (*dict*) – Dictionary of reactions.

- **bounds** (*dict*) – Dictionary of bounds.
- **infinity** (*float, optional*) – The value for infinity, by default `numpy.inf`.

Returns

dict – The updated bounds dictionary.

Return type

dict

`cbmpy.CBModelTools.addReversibilityBoundsIgnoreReversible(reactions, bounds)`

Add reversibility bounds to the bounds dictionary, ignoring reversible reactions.

Parameters

- **reactions** (*dict*) – Dictionary of reactions.
- **bounds** (*dict*) – Dictionary of bounds.

Returns

dict – The updated bounds dictionary.

Return type

dict

`cbmpy.CBModelTools.addSpecies(model, species)`

Add species to the model.

Parameters

- **model** (*object*) – The model object.
- **species** (*dict*) – Dictionary of species information.

Returns

None

Return type

None

`cbmpy.CBModelTools.addBounds(model, bounds)`

Add flux bounds to the model.

Parameters

- **model** (*object*) – The model object.
- **bounds** (*dict*) – Dictionary of bounds.

Returns

None

Return type

None

`cbmpy.CBModelTools.addReactions(model, reactions)`

Add reactions to the model.

Parameters

- **model** (*object*) – The model object.
- **reactions** (*dict*) – Dictionary of reactions information.
- **Returns**
- **else**
- **None**

`cbmpy.CBModelTools.addObjectiveFunction(model, objective_function)`

Add objective function to the model.

Parameters

- **model** (*object*) – The model object.
- **objective_function** (*dict*) – Dictionary of objective function information.

Returns

None

Return type

None

`cbmpy.CBModelTools.quickDefaultBuild(model_name, Reactions, Species, Bounds, Objective_function, infinity=inf)`

Quickly build a model.

Parameters

- **model_name** (*str*) – Name of the model.
- **Reactions** (*dict*) – Dictionary of reactions.
- **Species** (*dict*) – Dictionary of species.
- **Bounds** (*dict*) – Dictionary of bounds.
- **Objective_function** (*dict*) – Dictionary of objective functions.
- **infinity** (*float, optional*) – The value for infinity, by default `numpy.inf`.

Returns

object – The built model.

Return type

object

`cbmpy.CBModelTools.printSolution(fba, wait=False)`

Print the solution to the console.

Parameters

- **fba** (*object*) – The model object containing the solution.
- **wait** (*bool*, *optional*) – Whether to wait for user input, by default False.

Returns

None

Return type

None

2.16.2 CBMPy: default model module

PySCeS Constraint Based Modelling (<http://cbmpy.sourceforge.net>) Copyright (C) 2009-2024 Brett G. Olivier, VU University Amsterdam, Amsterdam, The Netherlands

This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program. If not, see <<http://www.gnu.org/licenses/>>

Author: Brett G. Olivier PhD Contact developers: <https://github.com/SystemsBioinformatics/cbmpy/issues> Last edit: \$Author: bgoli \$ (\$Id: CBCommon.py 660 2024-09-24 14:57:04Z bgoli \$)

2.17 Release Notes

2.18 0.8.9 (unreleased)

Docstring improvements:

- Converted all module docstrings to numpy format across the entire codebase - CBXML.py: 68 docstrings converted - CBModel.py: 206 docstrings converted - CBCPLEX.py: 25 docstrings converted - CBGLPK.py: 16 docstrings converted - CBNetDB.py: 21 docstrings converted - CBRead.py: 6 docstrings converted - CBWrite.py: 31 docstrings converted
- All docstrings now use numpy format compatible with Sphinx Napoleon extension
- Sphinx documentation updated with all documented modules

Improvements:

- Updated Sphinx docs with new module API pages - `api/solvers.rst` - CBSolver, CBSolverX, CBCPLEX, CBGLPK, CBGLPKOLD - `api/io.rst` - CBRead, CBReadtxt, CBWrite, CBXML - `api/networking.rst` - CNetDB, miriamids - `api/visualization.rst` - CBPlot, GUI frontends, multi-processing, utilities
- Added missing core classes to docs (Species, Objective, GeneProteinAssociation, UserConstraint, Group)
- Fixed docstring style reference in CLAUDE.md (numpydoc -> numpy)

2.19 0.8.8 (2023)

- Support for Python 3.8 through 3.12
- SBML FBCv3 support
- Flux variability analysis improvements

2.20 0.8.4 (2022)

- Python 3.x compatibility update
- SBML3 FBC standard support improvements

2.21 0.8.2 (2021)

- Bug fixes for SymPy compatibility
- Improved requirements handling

INDICES AND TABLES

- genindex
- modindex
- search

PYTHON MODULE INDEX

C

- `cbmpy`, 17
- `cbmpy.CBCommon`, 82
- `cbmpy.CBConfig`, 81
- `cbmpy.CBDataStruct`, 115
- `cbmpy.CBDefaultModels`, 168
- `cbmpy.CBGLPK`, 117
- `cbmpy.CBGUI`, 162
- `cbmpy.CBModel`, 17
- `cbmpy.CBModelTools`, 165
- `cbmpy.CBMultiCore`, 163
- `cbmpy.CBMultiEnv`, 164
- `cbmpy.CBMultiModel`, 164
- `cbmpy.CBNetDB`, 155
- `cbmpy.CBPlot`, 161
- `cbmpy.CBReadtxt`, 125
- `cbmpy.CBSolve`, 65
- `cbmpy.CBSolver`, 115
- `cbmpy.CBSolverX`, 115
- `cbmpy.CBTools`, 98
- `cbmpy.CBWx`, 162
- `cbmpy.CBXML`, 138
- `cbmpy.fluxmodules`, 7
- `cbmpy.fluxmodules.decomposition`, 11
- `cbmpy.fluxmodules.matroid`, 7
- `cbmpy.fluxmodules.sparserationals`, 15
- `cbmpy.miriamids`, 160

INDEX

Symbols

<code>__array_type__</code> (<i>cbmpy.CBCommon.StructMatrixLP</i> attribute), 90	<code>__init__()</code> (<i>cbmpy.CBModel.GroupMemberAttributes</i> method), 48
<code>__init__()</code> (<i>cbmpy.CBCommon.ComboGen</i> method), 84	<code>__init__()</code> (<i>cbmpy.CBModel.Model</i> method), 22
<code>__init__()</code> (<i>cbmpy.CBCommon.MIRIAMAnnotation</i> method), 96	<code>__init__()</code> (<i>cbmpy.CBModel.Objective</i> method), 43
<code>__init__()</code> (<i>cbmpy.CBCommon.StructMatrix</i> method), 87	<code>__init__()</code> (<i>cbmpy.CBModel.Parameter</i> method), 53
<code>__init__()</code> (<i>cbmpy.CBCommon.StructMatrixLP</i> method), 90	<code>__init__()</code> (<i>cbmpy.CBModel.Reaction</i> method), 54
<code>__init__()</code> (<i>cbmpy.CBModel.Compartment</i> method), 47	<code>__init__()</code> (<i>cbmpy.CBModel.ReactionNew</i> method), 58
<code>__init__()</code> (<i>cbmpy.CBModel.ConstraintComponent</i> method), 47	<code>__init__()</code> (<i>cbmpy.CBModel.Reagent</i> method), 61
<code>__init__()</code> (<i>cbmpy.CBModel.FluxBound</i> method), 52	<code>__init__()</code> (<i>cbmpy.CBModel.Species</i> method), 59
<code>__init__()</code> (<i>cbmpy.CBModel.FluxBoundBase</i> method), 52	<code>__init__()</code> (<i>cbmpy.CBModel.UserDefinedConstraint</i> method), 46
<code>__init__()</code> (<i>cbmpy.CBModel.FluxBoundLower</i> method), 53	<code>__init__()</code> (<i>cbmpy.CBNetDB.DBTools</i> method), 156
<code>__init__()</code> (<i>cbmpy.CBModel.FluxBoundUpper</i> method), 53	<code>__init__()</code> (<i>cbmpy.CBNetDB.KeGGSequenceTools</i> method), 159
<code>__init__()</code> (<i>cbmpy.CBModel.FluxObjective</i> method), 45	<code>__init__()</code> (<i>cbmpy.CBNetDB.KeGGTools</i> method), 158
<code>__init__()</code> (<i>cbmpy.CBModel.FluxObjectiveQuadratic</i> method), 45	<code>__init__()</code> (<i>cbmpy.CBNetDB.RESTClient</i> method), 159
<code>__init__()</code> (<i>cbmpy.CBModel.Gene</i> method), 61	<code>__init__()</code> (<i>cbmpy.CBNetDB.SemanticSBML</i> method), 160
<code>__init__()</code> (<i>cbmpy.CBModel.GeneProteinAssociation</i> method), 62	<code>__init__()</code> (<i>cbmpy.CBSolve.LPbase</i> method), 66
<code>__init__()</code> (<i>cbmpy.CBModel.Group</i> method), 49	<code>__init__()</code> (<i>cbmpy.CBSolve.LPcplex</i> method), 66
	<code>__init__()</code> (<i>cbmpy.CBSolve.LPglpk</i> method), 66
	<code>__init__()</code> (<i>cbmpy.CBXML.CBMtoSBML3</i>

`method`), 147
`__init__()` (`cbmpy.CBXML.FBCconnect` `method`), 145
`__init__()` (`cbmpy.CBXML.MLStripper` `method`), 139
`__init__()` (`cbmpy.fluxmodules.decomposition.AddCombination` `method`), 11
`__init__()` (`cbmpy.fluxmodules.decomposition.Decomposition` `method`), 13
`__init__()` (`cbmpy.fluxmodules.decomposition.Edge` `method`), 11
`__init__()` (`cbmpy.fluxmodules.decomposition.EdgeWidth` `method`), 15
`__init__()` (`cbmpy.fluxmodules.decomposition.Lead` `method`), 12
`__init__()` (`cbmpy.fluxmodules.decomposition.NoFullyBoundedException` `method`), 14
`__init__()` (`cbmpy.fluxmodules.decomposition.Validate` `method`), 12
`__init__()` (`cbmpy.fluxmodules.matroid.Matroid` `method`), 8
`__init__()` (`cbmpy.fluxmodules.matroid.SimilarityMatrix` `method`), 10
`__init__()` (`cbmpy.fluxmodules.sparserationals.Matrix` `method`), 15

A

`AbstractVertex` (class in `cbmpy.fluxmodules.decomposition`), 11
`active` (`cbmpy.CBModel.Gene` attribute), 62
`active0` (`cbmpy.CBModel.Gene` attribute), 62
`activeObjIdx` (`cbmpy.CBModel.Model` attribute), 21
`addAssociation()` (`cbmpy.CBModel.GeneProteinAssociation` `method`), 63
`addAssociation()` (`cbmpy.CBModel.Parameter` `method`), 54
`addBounds()` (in module `cbmpy.CBModelTools`), 166
`addBoundsToReactions()` (in module `cbmpy.CBReadtxt`), 126
`addBoundsV1()` (`cbmpy.CBXML.CBMtoSBML3` `method`), 147
`addBoundsV2()` (`cbmpy.CBXML.CBMtoSBML3` `method`), 147
`addCombination()` (`cbmpy.CBCommon.ComboGen` `method`), 84
`addCompartment()` (`cbmpy.CBModel.Model` `method`), 27
`addConstraintComponent()` (`cbmpy.CBModel.UserDefinedConstraint` `method`), 47
`addFluxAsActiveObjective()` (in module `cbmpy.CBTools`), 111
`addFullyBoundedException()` (`cbmpy.CBModel.Model` `method`), 27
`addFluxObjective()` (`cbmpy.CBModel.Objective` `method`), 43
`addGene()` (`cbmpy.CBModel.Model` `method`), 27
`addGeneProteinAssociationsV1()` (`cbmpy.CBXML.CBMtoSBML3` `method`), 147
`addGeneref()` (`cbmpy.CBModel.GeneProteinAssociation` `method`), 63
`addGenesFromAnnotations()` (in module `cbmpy.CBTools`), 109
`addGenesV2()` (`cbmpy.CBXML.CBMtoSBML3` `method`), 147
`addGPRAssociation()` (`cbmpy.CBModel.Model` `method`), 29
`addGroup()` (`cbmpy.CBModel.Model` `method`), 42
`addIDorgURI()` (`cbmpy.CBCommon.MIRIAMannotation` `method`), 96
`addLinearConstraint()` (`cbmpy.CBSolverX.LPBase` `method`), 116
`addListRep()` (`cbmpy.fluxmodules.sparserationals.Matrix` `method`), 16
`addMember()` (`cbmpy.CBModel.Group` `method`), 147

`method`), 49
`addMetabolicNetwork()` (`cbmpy.fluxmodules.sparserationals.Matrix` `method`), 16
`addMIRIAMannotation()` (`cbmpy.CBCommon.MIRIAMannotation` `method`), 97
`addMIRIAMannotation()` (`cbmpy.CBModel.Fbase` `method`), 20
`addMIRIAMannotation()` (`cbmpy.CBModel.Model` `method`), 23
`addMIRIAMuri()` (`cbmpy.CBModel.Fbase` `method`), 20
`addModelAnnotation()` (`cbmpy.CBXML.CBMtoSBML3` `method`), 147
`addModelCreator()` (`cbmpy.CBModel.Model` `method`), 24
`addModelHistory()` (`cbmpy.CBXML.CBMtoSBML3` `method`), 147
`addObjective()` (`cbmpy.CBModel.Model` `method`), 24
`addObjective()` (`cbmpy.CBSolverX.LPBase` `method`), 116
`addObjectiveFunction()` (in module `cbmpy.CBModelTools`), 167
`addObjectives()` (`cbmpy.CBXML.CBMtoSBML3` `method`), 147
`addParameter()` (`cbmpy.CBModel.Model` `method`), 27
`addProteinCostAnnotation()` (in module `cbmpy.CBMultiEnv`), 165
`addReaction()` (`cbmpy.CBModel.Model` `method`), 28
`addReactions()` (in module `cbmpy.CBModelTools`), 166
`addReagent()` (`cbmpy.CBModel.Reaction` `method`), 55
`addReversibilityBounds()` (in module `cbmpy.CBModelTools`), 165
`addReversibilityBoundsIgnoreReversible()` (in module `cbmpy.CBModelTools`), 166
`addSharedMIRIAMannotation()` (`cbmpy.CBModel.Group` `method`), 50
`addSinkReaction()` (in module `cbmpy.CBTools`), 100
`addSourceReaction()` (in module `cbmpy.CBTools`), 100
`addSpecies()` (`cbmpy.CBModel.Model` `method`), 27
`addSpecies()` (in module `cbmpy.CBModelTools`), 166
`addStoichToFBAModel()` (in module `cbmpy.CBTools`), 100
`addUserConstraint()` (`cbmpy.CBModel.Model` `method`), 28
`addUserDefinedConstraint()` (`cbmpy.CBModel.Model` `method`), 27
`addUserDefinedConstraintsV3()` (`cbmpy.CBXML.CBMtoSBML3` `method`), 148
`annotation` (`cbmpy.CBModel.Fbase` `attribute`), 18
`annotation_ext` (`cbmpy.CBModel.Fbase` `attribute`), 18
`array` (`cbmpy.CBCommon.StructMatrix` `attribute`), 86, 87
`assignAllSharedPropertiesToMembers()` (`cbmpy.CBModel.Group` `method`), 51
`assignSharedAnnotationToMembers()` (`cbmpy.CBModel.Group` `method`), 51
`assignSharedMIRIAMannotationToMembers()` (`cbmpy.CBModel.Group` `method`), 51
`assignSharedNotesToMembers()` (`cbmpy.CBModel.Group` `method`), 51
`assignSharedSBOTermsToMembers()` (`cbmpy.CBModel.Group` `method`), 51
`assoc` (`cbmpy.CBModel.GeneProteinAssociation` `attribute`), 62
B
`binHash()` (in module `cbmpy.CBCommon`), 85

bound_registry
 (*cbmpy.CBXML.CBMtoSBML3* at-tribute), 147
 buildEvalFunc()
 (*cbmpy.CBModel.GeneProteinAssociation* method), 63
 buildFBAobj() (in module *cbmpy.CBReadtxt*), 126
 buildGeneDatabase()
 (*cbmpy.CBNetDB.KeGGSequenceTools* method), 159
 buildStoichMatrix()
 (*cbmpy.CBModel.Model* method), 41
C
 cbmpy
 module, 16
 cbmpy.CBCommon
 module, 81
 cbmpy.CBConfig
 module, 81
 cbmpy.CBDataStruct
 module, 114
 cbmpy.CBDefaultModels
 module, 168
 cbmpy.CBGLPK
 module, 117
 cbmpy.CBGUI
 module, 162
 cbmpy.CBModel
 module, 17
 cbmpy.CBModelTools
 module, 165
 cbmpy.CBMultiCore
 module, 163
 cbmpy.CBMultiEnv
 module, 164
 cbmpy.CBMultiModel
 module, 163
 cbmpy.CBNetDB
 module, 155
 cbmpy.CBPlot
 module, 161
 cbmpy.CBReadtxt
 module, 125
 cbmpy.CBSolve
 module, 65
 cbmpy.CBSolver
 module, 115
 cbmpy.CBSolverX
 module, 115
 cbmpy.CBTools
 module, 98
 cbmpy.CBWx
 module, 162
 cbmpy.CBXML
 module, 138
 cbmpy.fluxmodules
 module, 7
 cbmpy.fluxmodules.decomposition
 module, 11
 cbmpy.fluxmodules.matroid
 module, 7
 cbmpy.fluxmodules.sparserationals
 module, 15
 cbmpy.miriamids
 module, 160
 CBMtoSBML3 (class in *cbmpy.CBXML*), 146
 changeAllFluxBoundsWithValue()
 (*cbmpy.CBModel.Model* method), 32
 changeId() (*cbmpy.CBModel.Reaction* method), 56
 charge (*cbmpy.CBModel.Species* attribute), 59
 checkChemFormula() (in module *cbmpy.CBCommon*), 82
 checkEntity()
 (*cbmpy.CBCommon.MIRIAMannotation* method), 97
 checkEntityPattern()
 (*cbmpy.CBCommon.MIRIAMannotation* method), 96
 checkEntryInColumn()
 (*cbmpy.CBNetDB.DBTools* method), 157
 checkExchangeReactions() (in module *cbmpy.CBTools*), 112
 checkFluxBoundConsistency() (in module *cbmpy.CBTools*), 113

[checkId\(\)](#) (*cbmpy.CBCommon.MIRIAMAnnotation attribute*), [22](#)
[checkId\(\)](#) (*in module cbmpy.CBCommon*), [86](#)
[checkIds\(\)](#) (*in module cbmpy.CBTools*), [113](#)
[checkReactionBalanceElemental\(\)](#) (*in module cbmpy.CBTools*), [111](#)
[checkRows\(\)](#) (*cbmpy.fluxmodules.sparserationals.Matrix*
tribute), [159](#)
[checkRows\(\)](#) (*method*), [16](#)
[chemFormula](#) (*cbmpy.CBModel.Species at-*
tribute), [59](#)
[cidx](#) (*cbmpy.CBCommon.StructMatrix at-*
tribute), [86](#), [87](#)
[clone\(\)](#) (*cbmpy.CBModel.Fbase method*), [19](#)
[clone\(\)](#) (*cbmpy.CBModel.Group method*), [49](#)
[clone\(\)](#) (*cbmpy.CBModel.Model method*), [23](#)
[Close\(\)](#) (*cbmpy.CBNetDB.RESTClient*
method), [160](#)
[closeDB\(\)](#) (*cbmpy.CBNetDB.DBTools*
method), [158](#)
[CM](#) (*cbmpy.CBModel.Model attribute*), [21](#)
[coefficient](#) (*cbmpy.CBModel.ConstraintComponent*
attribute), [47](#)
[coefficient](#) (*cbmpy.CBModel.FluxObjective*
attribute), [45](#)
[coefficient](#) (*cbmpy.CBModel.FluxObjectiveQuadratic*
attribute), [45](#)
[coefficient](#) (*cbmpy.CBModel.Reagent*
attribute), [60](#)
[col](#) (*cbmpy.CBCommon.StructMatrix at-*
tribute), [86](#), [87](#)
[coladd\(\)](#) (*cbmpy.fluxmodules.sparserationals.Matrix*
method), [16](#)
[cols\(\)](#) (*cbmpy.fluxmodules.sparserationals.Matrix*
method), [15](#)
[combo](#) (*cbmpy.CBCommon.ComboGen at-*
tribute), [84](#)
[combo_int](#) (*cbmpy.CBCommon.ComboGen at-*
tribute), [84](#)
[ComboGen](#) (*class in cbmpy.CBCommon*), [84](#)
[commitDB\(\)](#) (*cbmpy.CBNetDB.DBTools*
method), [156](#)
[compartment](#) (*cbmpy.CBModel.Fbase at-*
tribute), [18](#)
[Compartment](#) (*class in cbmpy.CBModel*), [47](#)
[compartments](#) (*cbmpy.CBModel.Model*
attribute), [159](#)
[conn](#) (*cbmpy.CBNetDB.RESTClient attribute*),
[159](#)
[Connect\(\)](#) (*cbmpy.CBNetDB.RESTClient*
method), [159](#)
[CONNECTED](#) (*cbmpy.CBNetDB.RESTClient at-*
tribute), [159](#)
[connectivity\(\)](#)
(cbmpy.fluxmodules.matroid.Matroid
method), [10](#)
[connectSQLiteDB\(\)](#)
(cbmpy.CBNetDB.DBTools method),
[156](#)
[constant](#) (*cbmpy.CBModel.Parameter at-*
tribute), [54](#)
[constraint_components](#)
(cbmpy.CBModel.UserDefinedConstraint
attribute), [46](#)
[ConstraintComponent](#) (*class in*
cbmpy.CBModel), [47](#)
[containsReactions\(\)](#)
(cbmpy.CBModel.Compartment
method), [48](#)
[containsSpecies\(\)](#)
(cbmpy.CBModel.Compartment
method), [48](#)
[contract\(\)](#) (*cbmpy.fluxmodules.matroid.Matroid*
method), [8](#)
[convertUserConstraintsToUserDefinedConstraints\(\)](#)
(cbmpy.CBModel.Model method), [28](#)
[copy_parallel\(\)](#) (*cbmpy.fluxmodules.matroid.Matroid*
method), [8](#)
[copy\(\)](#) (*cbmpy.fluxmodules.matroid.Matroid*
method), [8](#)
[copy\(\)](#) (*cbmpy.fluxmodules.sparserationals.Matrix*
method), [15](#)
[copyReaction\(\)](#) (*in module*
cbmpy.CBMultiModel), [164](#)
[copySpecies\(\)](#) (*in module*
cbmpy.CBMultiModel), [164](#)
[copyUserDefinedConstraintsToUserConstraints\(\)](#)
(cbmpy.CBModel.Model method), [28](#)
[corank\(\)](#) (*cbmpy.fluxmodules.matroid.Matroid*
method), [9](#)
[countedPause\(\)](#) (*in module cbmpy.CBTools*),
[159](#)

109

`createAssociationAndGeneRefs()` (*cbmpy.CBModel.GeneProteinAssociation* method), 63

`createAssociationAndGeneRefsFromString()` (*cbmpy.CBModel.GeneProteinAssociation* method), 63

`createAssociationAndGeneRefsFromTree()` (*cbmpy.CBModel.GeneProteinAssociation* method), 63

`createAssociationDictFromNode()` (in module *cbmpy.CBCommon*), 83

`createCompartment()` (*cbmpy.CBModel.Model* method), 25

`createConstraintComponent()` (*cbmpy.CBModel.UserDefinedConstraint* method), 47

`createDBTable()` (*cbmpy.CBNetDB.DBTools* method), 156

`createFbParameterSharedV2()` (*cbmpy.CBXML.CBMtoSBML3* method), 148

`createFbParameterV2()` (*cbmpy.CBXML.CBMtoSBML3* method), 148

`createFluxBoundV1()` (*cbmpy.CBXML.FBCconnect* method), 146

`createFluxObjectives()` (*cbmpy.CBModel.Objective* method), 43

`createGeneAssociationsFromAnnotations()` (*cbmpy.CBModel.Model* method), 29

`createGeneAssociationV1()` (*cbmpy.CBXML.FBCconnect* method), 146

`createGeneProteinAssociation()` (*cbmpy.CBModel.Model* method), 29

`createGeneProteinAssociationFromTree()` (*cbmpy.CBModel.Model* method), 30

`createGroup()` (*cbmpy.CBModel.Model* method), 42

`createGroup()` (*cbmpy.CBXML.FBCconnect* method), 146

`createGroupLegacy()` (*cbmpy.CBXML.FBCconnect* method), 146

`createObjective()` (*cbmpy.CBXML.FBCconnect* method), 146

`createObjectiveFunction()` (*cbmpy.CBModel.Model* method), 24

`createParameter()` (*cbmpy.CBModel.Model* method), 25

`createParParameter()` (*cbmpy.CBXML.CBMtoSBML3* method), 148

`createQuadraticFluxObjectives()` (*cbmpy.CBModel.Objective* method), 43

`createReaction()` (*cbmpy.CBModel.Model* method), 25

`createReactionBounds()` (*cbmpy.CBModel.Model* method), 26

`createReactionLowerBound()` (*cbmpy.CBModel.Model* method), 26

`createReactionNew()` (*cbmpy.CBModel.Model* method), 26

`createReactionReagent()` (*cbmpy.CBModel.Model* method), 26

`createReactionUpperBound()` (*cbmpy.CBModel.Model* method), 26

`createReagent()` (*cbmpy.CBModel.Reaction* method), 55

`createSingleGeneEffectMap()` (*cbmpy.CBModel.Model* method), 41

`createSpecies()` (*cbmpy.CBModel.Model* method), 25

`createTempFileName()` (in module *cbmpy.CBTools*), 98

`createUserDefinedConstraint()`

(*cbmpy.CBModel.Model* method), [27](#)
[createZipArchive\(\)](#) (in module *cbmpy.CBTools*), [112](#)
ctype (*cbmpy.CBModel.ConstraintComponent* attribute), [47](#)
ctype (*cbmpy.CBModel.FluxObjective* attribute), [45](#)
ctype (*cbmpy.CBModel.FluxObjectiveQuadratic* attribute), [45](#)
ctypes (*cbmpy.CBModel.ConstraintComponent* attribute), [47](#)
ctypes (*cbmpy.CBModel.FluxObjective* attribute), [45](#)
ctypes (*cbmpy.CBModel.FluxObjectiveQuadratic* attribute), [45](#)
[current_version\(\)](#) (in module *cbmpy.CBConfig*), [81](#)
[current_version_tuple\(\)](#) (in module *cbmpy.CBConfig*), [81](#)
D
data (*cbmpy.CBNetDB.SemanticSBML* attribute), [160](#)
DATE_CREATED (*cbmpy.CBModel.Model* attribute), [22](#)
DATE_MODIFIED (*cbmpy.CBModel.Model* attribute), [22](#)
DB (*cbmpy.CBNetDB.KeGGSequenceTools* attribute), [159](#)
db_conn (*cbmpy.CBNetDB.DBTools* attribute), [156](#)
db_cursor (*cbmpy.CBNetDB.DBTools* attribute), [156](#)
db_tables (*cbmpy.CBNetDB.DBTools* attribute), [156](#)
DBTools (class in *cbmpy.CBNetDB*), [156](#)
[deactivateReaction\(\)](#) (*cbmpy.CBModel.Reaction* method), [57](#)
[deactivateReaction\(\)](#) (*cbmpy.CBModel.ReactionNew* method), [58](#)
Decomposition (class in *cbmpy.fluxmodules.decomposition*), [13](#)
[delete\(\)](#) (*cbmpy.fluxmodules.matroid.Matroid* method), [8](#)
[deleteAllFluxBoundsWithValue\(\)](#) (*cbmpy.CBModel.Model* method), [33](#)
[deleteAllFluxObjectives\(\)](#) (*cbmpy.CBModel.Objective* method), [43](#)
[deleteAnnotation\(\)](#) (*cbmpy.CBModel.Fbase* method), [19](#)
[deleteAssociation\(\)](#) (*cbmpy.CBModel.Parameter* method), [54](#)
[deleteBoundsForReactionId\(\)](#) (*cbmpy.CBModel.Model* method), [29](#)
[deleteCompartment\(\)](#) (*cbmpy.CBModel.Model* method), [33](#)
[deleteGene\(\)](#) (*cbmpy.CBModel.Model* method), [33](#)
[deleteGeneByLabel\(\)](#) (*cbmpy.CBModel.Model* method), [33](#)
[deleteGeneFromAssociation\(\)](#) (*cbmpy.CBModel.GeneProteinAssociation* method), [65](#)
[deleteGeneref\(\)](#) (*cbmpy.CBModel.GeneProteinAssociation* method), [63](#)
[deleteGPRAssociation\(\)](#) (*cbmpy.CBModel.Model* method), [33](#)
[deleteGroup\(\)](#) (*cbmpy.CBModel.Model* method), [42](#)
[deleteMember\(\)](#) (*cbmpy.CBModel.Group* method), [49](#)
[deleteMIRIAMannotation\(\)](#) (*cbmpy.CBCommon.MIRIAMannotation* method), [97](#)
[deleteMIRIAMannotation\(\)](#) (*cbmpy.CBModel.Fbase* method), [20](#)
[deleteNonReactingSpecies\(\)](#)

`(cbmpy.CBModel.Model method)`, 34
`deleteObjective()` (*cbmpy.CBModel.Model method*), 28
`deleteReactionAndBounds()` (*cbmpy.CBModel.Model method*), 28
`deleteReagentWithSpeciesRef()` (*cbmpy.CBModel.Reaction method*), 57
`deleteSpecies()` (*cbmpy.CBModel.Model method*), 33
`description` (*cbmpy.CBModel.Model attribute*), 21
`deSerialize()` (*in module cbmpy.CBTools*), 99
`deSerializeFromDisk()` (*in module cbmpy.CBTools*), 99
`dimensions` (*cbmpy.CBModel.Compartment attribute*), 48
`doc` (*cbmpy.CBXML.FBCconnect attribute*), 145
`doFBAMinSum()` (*in module cbmpy*), 17
`dual()` (*cbmpy.fluxmodules.matroid.Matroid method*), 8
`dumpReactionsToTxt()` (*in module cbmpy.CBReadtxt*), 126
`dumpSpeciesToTxt()` (*in module cbmpy.CBReadtxt*), 126
`dumpTableToCSV()` (*cbmpy.CBNetDB.DBTools method*), 158
`dumpTableToTxt()` (*cbmpy.CBNetDB.DBTools method*), 158

E

`Edge` (*class in cbmpy.fluxmodules.decomposition*), 11
`EdgeWidthException`, 15
`emptyUndelete()` (*cbmpy.CBModel.Model method*), 42
`encodes` (*cbmpy.CBCommon.MIRIAMannotation attribute*), 94, 95
`evalAssociation()` (*cbmpy.CBModel.GeneProteinAssociation method*), 63
`exchange()` (*cbmpy.fluxmodules.matroid.Matroid method*), 8
`executeSQL()` (*cbmpy.CBNetDB.DBTools method*), 157
`exportArray2CSV()` (*in module cbmpy.CBTools*), 104
`exportArray2TXT()` (*in module cbmpy.CBTools*), 105
`exportFVAdata()` (*cbmpy.CBModel.Model method*), 42
`exportLabelledArray()` (*in module cbmpy.CBTools*), 103
`exportLabelledArray2CSV()` (*in module cbmpy.CBTools*), 104
`exportLabelledArray2TXT()` (*in module cbmpy.CBTools*), 105
`exportLabelledArrayWithHeader()` (*in module cbmpy.CBTools*), 103
`exportLabelledArrayWithHeader2CSV()` (*in module cbmpy.CBTools*), 104
`exportLabelledArrayWithHeader2TXT()` (*in module cbmpy.CBTools*), 105
`exportLabelledLinkedList()` (*in module cbmpy.CBTools*), 103
`exportUserConstraints()` (*cbmpy.CBModel.Model method*), 42
`extractGeneIdsFromString()` (*in module cbmpy.CBCommon*), 82

F

`fba` (*cbmpy.CBXML.CBMtoSBML3 attribute*), 147
`Fbase` (*class in cbmpy.CBModel*), 17
`fbc` (*cbmpy.CBXML.FBCconnect attribute*), 146
`FBCconnect` (*class in cbmpy.CBXML*), 145
`fbcstrict` (*cbmpy.CBXML.FBCconnect attribute*), 146
`fbcversion` (*cbmpy.CBXML.FBCconnect attribute*), 145
`fetchAll()` (*cbmpy.CBNetDB.DBTools method*), 158

- [fetchSeqfromKeGG\(\)](#)
(cbmpy.CBNetDB.KeGGTools method), 158
- [findDeadEndMetabolites\(\)](#) *(in module cbmpy.CBTools)*, 101
- [findDeadEndReactions\(\)](#) *(in module cbmpy.CBTools)*, 101
- [findFluxesForConnectedSpecies\(\)](#)
(cbmpy.CBModel.Model method), 40
- [findModules\(\)](#)
(cbmpy.fluxmodules.matroid.Matroid method), 8
- [fixId\(\)](#) *(in module cbmpy.CBCommon)*, 85
- [fixReversibility\(\)](#) *(in module cbmpy.CBTools)*, 102
- [flux_bounds](#) *(cbmpy.CBModel.Model attribute)*, 22
- [flux_objectives](#) *(cbmpy.CBModel.Objective attribute)*, 43
- [FluxBound](#) *(class in cbmpy.CBModel)*, 52
- [FluxBoundBase](#) *(class in cbmpy.CBModel)*, 52
- [FluxBoundLower](#) *(class in cbmpy.CBModel)*, 53
- [FluxBoundUpper](#) *(class in cbmpy.CBModel)*, 53
- [FluxObjective](#) *(class in cbmpy.CBModel)*, 45
- [FluxObjectiveQuadratic](#) *(class in cbmpy.CBModel)*, 44
- [formatSbmlId\(\)](#) *(in module cbmpy.CBXML)*, 140
- [fromMatrix\(\)](#) *(in module cbmpy.fluxmodules.matroid)*, 10
- [func_getAssociationStrFromGprDict\(\)](#)
(in module cbmpy.CBCommon), 83
- [fundamentalCircuit\(\)](#)
(cbmpy.fluxmodules.matroid.Matroid method), 9
- [fundamentalCircuitValues\(\)](#)
(cbmpy.fluxmodules.matroid.Matroid method), 9
- [fva_max](#) *(cbmpy.CBModel.Reaction attribute)*, 54
- [fva_min](#) *(cbmpy.CBModel.Reaction attribute)*, 54
- [fva_status](#) *(cbmpy.CBModel.Reaction attribute)*, 54
- ## G
- [Gene](#) *(class in cbmpy.CBModel)*, 61
- [GeneProteinAssociation](#) *(class in cbmpy.CBModel)*, 62
- [generateInputScanReports\(\)](#) *(in module cbmpy.CBTools)*, 108
- [generefs](#) *(cbmpy.CBModel.GeneProteinAssociation attribute)*, 63
- [genes](#) *(cbmpy.CBModel.Model attribute)*, 22
- [Get\(\)](#) *(cbmpy.CBNetDB.RESTClient method)*, 159
- [get_data\(\)](#) *(cbmpy.CBXML.MLStripper method)*, 139
- [getActiveGenes\(\)](#)
(cbmpy.CBModel.GeneProteinAssociation method), 64
- [getActiveObjective\(\)](#)
(cbmpy.CBModel.Model method), 36
- [getActiveObjectiveReactionIds\(\)](#)
(cbmpy.CBModel.Model method), 36
- [getActiveObjectiveStoichiometry\(\)](#)
(cbmpy.CBModel.Model method), 36
- [getAdjacent\(\)](#)
(cbmpy.fluxmodules.decomposition.AbstractVertex method), 11
- [getAdjacent\(\)](#)
(cbmpy.fluxmodules.decomposition.Leaf method), 13
- [getAdjacent\(\)](#)
(cbmpy.fluxmodules.decomposition.Vertex method), 12
- [getAllFluxBounds\(\)](#)
(cbmpy.CBModel.Model method), 38
- [getAllGeneActivities\(\)](#)
(cbmpy.CBModel.Model method), 32
- [getAllGeneProteinAssociations\(\)](#)
(cbmpy.CBModel.Model method), 30
- [getAllMIRIAMUris\(\)](#)
(cbmpy.CBCommon.MIRIAMannotation method), 98

`getAllProteinActivities()`
 (*cbmpy.CBModel.Model* *method*), 32
`getAllProteinGeneAssociations()`
 (*cbmpy.CBModel.Model* *method*), 30
`getAllReactionsAssociatedWithGene()`
 (*in module cbmpy.CBTools*), 108
`getAndViewUrisForQualifier()`
 (*cbmpy.CBCommon.MIRIAMannotation* *method*), 98
`getAnnotation()` (*cbmpy.CBModel.Fbase* *method*), 18
`getAnnotations()` (*cbmpy.CBModel.Fbase* *method*), 18
`getAssociations()`
 (*cbmpy.CBModel.Parameter* *method*), 54
`getAssociationStr()`
 (*cbmpy.CBModel.GeneProteinAssociation* *method*), 64
`getBoundarySpeciesIds()`
 (*cbmpy.CBModel.Model* *method*), 40
`getBounds()` (*in module cbmpy.CBReadtxt*), 126
`getBoundsDict()` (*in module cbmpy.CBTools*), 106
`getByIdx()` (*cbmpy.CBCommon.StructMatrix* *method*), 89
`getByName()` (*cbmpy.CBCommon.StructMatrix* *method*), 89
`getCell()` (*cbmpy.CBNetDB.DBTools* *method*), 157
`getCharge()` (*cbmpy.CBModel.Species* *method*), 60
`getChemFormula()` (*cbmpy.CBModel.Species* *method*), 60
`getCoefficient()`
 (*cbmpy.CBModel.ConstraintComponent* *method*), 47
`getCoefficient()`
 (*cbmpy.CBModel.FluxObjective* *method*), 46
`getCoefficient()`
 (*cbmpy.CBModel.FluxObjectiveQuadratic* *method*), 45
`getCoefficient()` (*cbmpy.CBModel.Reagent* *method*), 61
`getCol()` (*cbmpy.fluxmodules.sparserationals.Matrix* *method*), 15
`getColsByIdx()`
 (*cbmpy.CBCommon.StructMatrix* *method*), 87
`getColsByName()`
 (*cbmpy.CBCommon.StructMatrix* *method*), 88
`getColumns()` (*cbmpy.CBNetDB.DBTools* *method*), 157
`getCompartment()` (*cbmpy.CBModel.Model* *method*), 34
`getCompartmentId()`
 (*cbmpy.CBModel.Fbase* *method*), 21
`getCompartmentIds()`
 (*cbmpy.CBModel.Model* *method*), 35
`getCompartmentObjects()`
 (*cbmpy.CBModel.Model* *method*), 35
`getConstraintComponent()`
 (*cbmpy.CBModel.UserDefinedConstraint* *method*), 46
`getConstraintComponentData()`
 (*cbmpy.CBModel.UserDefinedConstraint* *method*), 46
`getConstraintComponentForVariable()`
 (*cbmpy.CBModel.UserDefinedConstraint* *method*), 46
`getConstraintComponentIDs()`
 (*cbmpy.CBModel.UserDefinedConstraint* *method*), 46
`getConstraintComponents()`
 (*cbmpy.CBModel.UserDefinedConstraint* *method*), 47
`getConstraintComponentVariables()`
 (*cbmpy.CBModel.UserDefinedConstraint* *method*), 46
`getConstraintComponentVariableTypes()`
 (*cbmpy.CBModel.UserDefinedConstraint* *method*), 46

<code>getCopy()</code> (<i>cbmpy.CBCommon.StructMatrixLP method</i>), 91	<code>(cbmpy.CBModel.Objective method)</code> , 44
<code>getDescription()</code> (<i>cbmpy.CBModel.Model method</i>), 23	<code>getFluxObjectives()</code> (<i>cbmpy.CBModel.Objective method</i>), 44
<code>getDimensions()</code> (<i>cbmpy.CBModel.Compartment method</i>), 48	<code>getFVAdata()</code> (<i>cbmpy.CBModel.Reaction method</i>), 55
<code>getEquation()</code> (<i>cbmpy.CBModel.Reaction method</i>), 57	<code>getGene()</code> (<i>cbmpy.CBModel.GeneProteinAssociation method</i>), 64
<code>getExchangeReactionIds()</code> (<i>cbmpy.CBModel.Model method</i>), 40	<code>getGene()</code> (<i>cbmpy.CBModel.Model method</i>), 30
<code>getExchangeReactions()</code> (<i>cbmpy.CBModel.Model method</i>), 40	<code>getGeneByLabel()</code> (<i>cbmpy.CBModel.Model method</i>), 37
<code>getExchBoundsDict()</code> (<i>in module cbmpy.CBTools</i>), 106	<code>getGeneIdFromLabel()</code> (<i>cbmpy.CBModel.Model method</i>), 36
<code>getFluxBoundByID()</code> (<i>cbmpy.CBModel.Model method</i>), 34	<code>getGeneIds()</code> (<i>cbmpy.CBModel.GeneProteinAssociation method</i>), 64
<code>getFluxBoundByReactionID()</code> (<i>cbmpy.CBModel.Model method</i>), 35	<code>getGeneIds()</code> (<i>cbmpy.CBModel.Model method</i>), 31
<code>getFluxBoundIds()</code> (<i>cbmpy.CBModel.Model method</i>), 36	<code>getGeneLabels()</code> (<i>cbmpy.CBModel.GeneProteinAssociation method</i>), 64
<code>getFluxBoundsByReactionID()</code> (<i>cbmpy.CBModel.Model method</i>), 35	<code>getGeneLabels()</code> (<i>cbmpy.CBModel.Model method</i>), 32
<code>getFluxesAssociatedWithCompartments()</code> (<i>cbmpy.CBModel.Model method</i>), 41	<code>getGeneObjects()</code> (<i>cbmpy.CBModel.Model method</i>), 31
<code>getFluxesAssociatedWithSpecies()</code> (<i>cbmpy.CBModel.Model method</i>), 40	<code>getGeneObjectsByLabel()</code> (<i>cbmpy.CBModel.Model method</i>), 31
<code>getFluxObjective()</code> (<i>cbmpy.CBModel.Objective method</i>), 44	<code>getGenes()</code> (<i>cbmpy.CBModel.GeneProteinAssociation method</i>), 64
<code>getFluxObjectiveData()</code> (<i>cbmpy.CBModel.Objective method</i>), 44	<code>getGPRasDictFromString()</code> (<i>in module cbmpy.CBCommon</i>), 83
<code>getFluxObjectiveForReaction()</code> (<i>cbmpy.CBModel.Objective method</i>), 44	<code>getGPRassociation()</code> (<i>cbmpy.CBModel.Model method</i>), 30
<code>getFluxObjectiveIDs()</code> (<i>cbmpy.CBModel.Objective method</i>), 44	<code>getGPRassociationString()</code> (<i>cbmpy.CBModel.Reaction method</i>), 57
<code>getFluxObjectiveReactions()</code>	<code>getGPRforReaction()</code> (<i>cbmpy.CBModel.Model method</i>), 30
	<code>getGPRforReactionAsDict()</code> (<i>cbmpy.CBModel.Model method</i>), 31

[getGPRIIdAssociatedWithGeneId\(\)](#) (*cbmpy.CBModel.Model* method), 31
[getGPRIIdAssociatedWithGeneLabel\(\)](#) (*cbmpy.CBModel.Model* method), 31
[getGPRIds\(\)](#) (*cbmpy.CBModel.Model* method), 31
[getGPRObjects\(\)](#) (*cbmpy.CBModel.Model* method), 32
[getGroup\(\)](#) (*cbmpy.CBModel.Model* method), 42
[getGroupIds\(\)](#) (*cbmpy.CBModel.Model* method), 42
[getGroupMembership\(\)](#) (*cbmpy.CBModel.Model* method), 42
[getGroupNames\(\)](#) (*cbmpy.CBModel.Model* method), 42
[getId\(\)](#) (*cbmpy.CBModel.Fbase* method), 18
[getIndexs\(\)](#) (*cbmpy.CBCommon.StructMatrix* method), 89
[getInterface\(\)](#) (*cbmpy.fluxmodules.matroid.Matroid* method), 10
[getIrreversibleReactionIds\(\)](#) (*cbmpy.CBModel.Model* method), 40
[getKind\(\)](#) (*cbmpy.CBModel.Group* method), 50
[getLabel\(\)](#) (*cbmpy.CBModel.Gene* method), 62
[getLabels\(\)](#) (*cbmpy.CBCommon.StructMatrix* method), 88
[getLandRHS\(\)](#) (*cbmpy.CBCommon.StructMatrixLP* method), 91
[getLeaves\(\)](#) (*cbmpy.fluxmodules.decomposition.AbstractVertex* method), 12
[getLeaves\(\)](#) (*cbmpy.fluxmodules.decomposition.Leaf* method), 13
[getLeaves\(\)](#) (*cbmpy.fluxmodules.decomposition.Vertex* method), 12
[getLinearFluxObjectives\(\)](#) (*cbmpy.CBModel.Objective* method), 44
[GetLog\(\)](#) (*cbmpy.CBNetDB.RESTClient* method), 159
[getLowerBound\(\)](#) (*cbmpy.CBModel.Reaction* method), 57
[getLowerBound\(\)](#) (*cbmpy.CBModel.ReactionNew* method), 58
[getLowerBound\(\)](#) (*cbmpy.CBModel.UserDefinedConstraint* method), 46
[getMember\(\)](#) (*cbmpy.CBModel.Group* method), 49
[getMemberIds\(\)](#) (*cbmpy.CBModel.Group* method), 49
[getMembers\(\)](#) (*cbmpy.CBModel.Group* method), 49
[getMetaId\(\)](#) (*cbmpy.CBModel.Fbase* method), 18
[getMIRIAMAnnotations\(\)](#) (*cbmpy.CBModel.Fbase* method), 20
[getMIRIAMUrisForQualifier\(\)](#) (*cbmpy.CBCommon.MIRIAMAnnotation* method), 97
[getModel\(\)](#) (*cbmpy.CBModel.Fbase* method), 18
[getModel\(\)](#) (*cbmpy.CBModel.Model* method), 23
[getModelCreators\(\)](#) (*cbmpy.CBModel.Model* method), 24
[getModelGenesPerReaction\(\)](#) (in module *cbmpy.CBTools*), 110
[getName\(\)](#) (*cbmpy.CBModel.Fbase* method), 18
[getNotes\(\)](#) (*cbmpy.CBModel.Fbase* method), 18
[getObject\(\)](#) (*cbmpy.CBModel.Model* method), 23
[getObjectiveIds\(\)](#) (*cbmpy.CBModel.Model* method), 36
[getObjFuncValue\(\)](#) (*cbmpy.CBModel.Model* method), 36
[getOperation\(\)](#) (*cbmpy.CBModel.Objective* method), 43
[getOperatorbyIdx\(\)](#)

<code>(cbmpy.CBCommon.StructMatrixLP method), 93</code>	<code>getReactionId2()</code> <code>(cbmpy.CBModel.FluxObjectiveQuadratic method), 45</code>
<code>getOperatorByName()</code> <code>(cbmpy.CBCommon.StructMatrixLP method), 92</code>	<code>getReactionIds()</code> <code>(cbmpy.CBModel.FluxObjectiveQuadratic method), 45</code>
<code>getOptimalValue()</code> <code>(cbmpy.CBModel.Model method), 36</code>	<code>getReactionIds()</code> <code>(cbmpy.CBModel.Model method), 35</code>
<code>getOther()</code> <code>(cbmpy.fluxmodules.decomposition.Edge method), 11</code>	<code>getReactionIdsAssociatedWithSpecies()</code> <code>(cbmpy.CBModel.Model method), 41</code>
<code>getParameter()</code> <code>(cbmpy.CBModel.Model method), 34</code>	<code>getReactionLowerBound()</code> <code>(cbmpy.CBModel.Model method), 34</code>
<code>getPeptideLengthsFromDB()</code> <code>(cbmpy.CBNetDB.KeGGSequenceTools method), 159</code>	<code>getReactionNames()</code> <code>(cbmpy.CBModel.Model method), 36</code>
<code>getPid()</code> <code>(cbmpy.CBModel.Fbase method), 18</code>	<code>getReactionObjects()</code> <code>(cbmpy.CBModel.Model method), 35</code>
<code>getProductIds()</code> <code>(cbmpy.CBModel.Reaction method), 56</code>	<code>getReactions()</code> <code>(in module cbmpy.CBReadtxt), 126</code>
<code>getProtein()</code> <code>(cbmpy.CBModel.GeneProteinAssociation method), 64</code>	<code>getReactions_old_format()</code> <code>(in module cbmpy.CBReadtxt), 126</code>
<code>getQuadraticBivariateFluxObjectives()</code> <code>(cbmpy.CBModel.Objective method), 44</code>	<code>getReactionsPerGene()</code> <code>(in module cbmpy.CBTools), 110</code>
<code>getQuadraticFluxObjectives()</code> <code>(cbmpy.CBModel.Objective method), 44</code>	<code>getReactionUpperBound()</code> <code>(cbmpy.CBModel.Model method), 34</code>
<code>getReaction()</code> <code>(cbmpy.CBModel.Model method), 34</code>	<code>getReactionValues()</code> <code>(cbmpy.CBModel.Model method), 40</code>
<code>getReactionActivity()</code> <code>(cbmpy.CBModel.Model method), 31</code>	<code>getReagent()</code> <code>(cbmpy.CBModel.Reaction method), 55</code>
<code>getReactionBounds()</code> <code>(cbmpy.CBModel.Model method), 34</code>	<code>getReagentObjIds()</code> <code>(cbmpy.CBModel.Reaction method), 55</code>
<code>getReactionId()</code> <code>(cbmpy.CBModel.FluxBound method), 52</code>	<code>getReagentOf()</code> <code>(cbmpy.CBModel.Species method), 60</code>
<code>getReactionId()</code> <code>(cbmpy.CBModel.FluxBoundBase method), 53</code>	<code>getReagentRefs()</code> <code>(cbmpy.CBModel.Reaction method), 55</code>
<code>getReactionId()</code> <code>(cbmpy.CBModel.FluxObjective method), 46</code>	<code>getReagentWithSpeciesRef()</code> <code>(cbmpy.CBModel.Reaction method), 56</code>
<code>getReactionId()</code> <code>(cbmpy.CBModel.FluxObjectiveQuadratic method), 45</code>	<code>getReducedCosts()</code> <code>(in module cbmpy.CBGLPK), 119</code>
	<code>getReversibleReactionIds()</code> <code>(cbmpy.CBModel.Model method),</code>

40
getRHSbyIdx() (*cbmpy.CBCommon.StructMatrixLP*
method), 92
getRHSbyName() (*cbmpy.CBCommon.StructMatrixLP*
method), 92
getRole() (*cbmpy.CBModel.Reagent* method),
61
getRow() (*cbmpy.CBNetDB.DBTools* method),
157
getRowsByIdx() (*cbmpy.CBCommon.StructMatrix*
method), 87
getRowsByName() (*cbmpy.CBCommon.StructMatrix*
method), 88
getSB0term() (*cbmpy.CBModel.Fbase*
method), 21
getSharedAnnotations() (*cbmpy.CBModel.Group* method),
51
getSharedMIRIAMannotations() (*cbmpy.CBModel.Group* method),
50
getSharedNotes() (*cbmpy.CBModel.Group*
method), 51
getSharedSB0term() (*cbmpy.CBModel.Group* method),
51
getSize() (*cbmpy.CBModel.Compartment*
method), 48
getSolutionVector() (*cbmpy.CBModel.Model* method),
40
getSpecies() (*cbmpy.CBModel.Model*
method), 34
getSpecies() (*cbmpy.CBModel.Reagent*
method), 61
getSpecies() (in module *cbmpy.CBReadtxt*),
126
getSpeciesIds() (*cbmpy.CBModel.Model*
method), 35
getSpeciesIds() (*cbmpy.CBModel.Reaction*
method), 55
getSpeciesObj() (*cbmpy.CBModel.Reaction*
method), 55
getSpeciesObjects() (*cbmpy.CBModel.Model* method),
35
getStoichiometry() (*cbmpy.CBModel.Reaction* method), 56
getSubstrateIds() (*cbmpy.CBModel.Reaction* method), 56
getTable() (*cbmpy.CBNetDB.DBTools*
method), 158
getTree() (*cbmpy.CBModel.GeneProteinAssociation*
method), 65
getTreeCopy() (*cbmpy.CBModel.GeneProteinAssociation*
method), 65
getType() (*cbmpy.CBModel.ConstraintComponent*
method), 47
getType() (*cbmpy.CBModel.FluxBound*
method), 52
getType() (*cbmpy.CBModel.FluxBoundBase*
method), 52
getType() (*cbmpy.CBModel.FluxObjective*
method), 46
getType() (*cbmpy.CBModel.FluxObjectiveQuadratic*
method), 45
getUpperBound() (*cbmpy.CBModel.Reaction*
method), 57
getUpperBound() (*cbmpy.CBModel.ReactionNew*
method), 58
getUpperBound() (*cbmpy.CBModel.UserDefinedConstraint*
method), 46
getValue() (*cbmpy.CBModel.FluxBound*
method), 52
getValue() (*cbmpy.CBModel.FluxBoundBase*
method), 53
getValue() (*cbmpy.CBModel.Objective*
method), 44
getValue() (*cbmpy.CBModel.Parameter*
method), 54
getValue() (*cbmpy.CBModel.Reaction*
method), 56
getValue() (*cbmpy.CBModel.Species*

- method), 59
- getVariable() (cbmpy.CBModel.ConstraintComponent method), 47
- getVariableType() (cbmpy.CBModel.ConstraintComponent method), 47
- getVertex() (cbmpy.fluxmodules.decomposition.Decomposition method), 13
- getWidth() (cbmpy.fluxmodules.decomposition.Decomposition method), 14
- glpk_analyzeModel() (in module cbmpy.CBGLPK), 118
- glpk_constructLPfromFBA() (in module cbmpy.CBGLPK), 117
- glpk_FluxVariabilityAnalysis() (in module cbmpy.CBGLPK), 119
- glpk_func_GetCPXandPresolve() (in module cbmpy.CBGLPK), 120
- glpk_func_SetObjectiveFunctionAsConstraint() (in module cbmpy.CBGLPK), 121
- glpk_getOptimalSolution() (in module cbmpy.CBGLPK), 119
- glpk_getReducedCosts() (in module cbmpy.CBGLPK), 119
- glpk_getSolutionStatus() (in module cbmpy.CBGLPK), 117
- glpk_setFBAsolutionToModel() (in module cbmpy.CBGLPK), 119
- glpk_setObjective() (in module cbmpy.CBGLPK), 121
- glpk_setSingleConstraint() (in module cbmpy.CBGLPK), 121
- glpk_setSolutionStatusToModel() (in module cbmpy.CBGLPK), 118
- glpk_Solve() (in module cbmpy.CBGLPK), 117
- gpr (cbmpy.CBModel.Model attribute), 22
- Group (class in cbmpy.CBModel), 49
- grouper() (in module cbmpy.CBMultiCore), 163
- groupList (cbmpy.CBXML.FBCconnect attribute), 145
- GroupMemberAttributes (class in cbmpy.CBModel), 48
- groups (cbmpy.CBModel.Model attribute), 22
- groups (cbmpy.CBXML.FBCconnect attribute), 146
- GROUPS_AVAILABLE (cbmpy.CBXML.FBCconnect attribute), 146
- groupsversion (cbmpy.CBXML.FBCconnect attribute), 145
- ## H
- handle_data() (cbmpy.CBXML.MLStripper method), 139
- hasAnnotation() (cbmpy.CBModel.Fbase method), 18
- hasMember() (cbmpy.CBModel.Group method), 49
- hasObject() (cbmpy.CBModel.Model method), 23
- hasPart (cbmpy.CBCommon.MIRIAMannotation attribute), 94, 95
- hasProperty (cbmpy.CBCommon.MIRIAMannotation attribute), 94, 95
- hasTaxon (cbmpy.CBCommon.MIRIAMannotation attribute), 94, 95
- hasVersion (cbmpy.CBCommon.MIRIAMannotation attribute), 94, 95
- history (cbmpy.CBNetDB.RESTClient attribute), 159
- ## I
- id (cbmpy.CBModel.Fbase attribute), 17
- id (cbmpy.CBSolve.LPproperties property), 65
- id (cbmpy.CBSolverX.LPBase attribute), 116
- id() (cbmpy.fluxmodules.decomposition.AbstractVertex method), 11
- importUserConstraints() (cbmpy.CBModel.Model method), 42
- insertData() (cbmpy.CBNetDB.DBTools method), 156
- is_balanced (cbmpy.CBModel.Reaction attribute), 54
- is_bound (cbmpy.CBModel.FluxBound attribute), 52

- `is_boundary` (*cbmpy.CBModel.Species* attribute), 59
 - `is_exchange` (*cbmpy.CBModel.Reaction* attribute), 54
 - `isA` (*cbmpy.CBCommon.MIRIAMannotation* attribute), 93, 95
 - `isActive()` (*cbmpy.CBModel.Gene* method), 62
 - `isCoIndependent()` (*cbmpy.fluxmodules.matroid.Matroid* method), 9
 - `isCoparallel()` (*cbmpy.fluxmodules.matroid.Matroid* method), 9
 - `isDerivedFrom` (*cbmpy.CBCommon.MIRIAMannotation* attribute), 95, 96
 - `isDescribedBy` (*cbmpy.CBCommon.MIRIAMannotation* attribute), 94, 95
 - `isEncodedBy` (*cbmpy.CBCommon.MIRIAMannotation* attribute), 94, 95
 - `isHomologTo` (*cbmpy.CBCommon.MIRIAMannotation* attribute), 94, 95
 - `isIndependent()` (*cbmpy.fluxmodules.matroid.Matroid* method), 9
 - `isParallel()` (*cbmpy.fluxmodules.matroid.Matroid* method), 9
 - `isParallelSeries()` (*cbmpy.fluxmodules.decomposition.Decomposition* method), 14
 - `isPartOf` (*cbmpy.CBCommon.MIRIAMannotation* attribute), 95
 - `isPropertyOf` (*cbmpy.CBCommon.MIRIAMannotation* attribute), 95, 96
 - `isProteinActive()` (*cbmpy.CBModel.GeneProteinAssociation* method), 64
 - `isReagentOf()` (*cbmpy.CBModel.Species* method), 60
 - `isSimple()` (*cbmpy.fluxmodules.decomposition.Vertex* method), 12
 - `isVersionOf` (*cbmpy.CBCommon.MIRIAMannotation* attribute), 95, 96
 - `isZero()` (*cbmpy.fluxmodules.sparserationals.Matrix* method), 16
 - `item_re` (*cbmpy.CBNetDB.SemanticSBML* attribute), 160
- ## K
- `Kclient` (*cbmpy.CBNetDB.KeGGTools* attribute), 158
 - `KEGG` (*cbmpy.CBNetDB.KeGGSequenceTools* attribute), 159
 - `KeGGSequenceTools` (class in *cbmpy.CBNetDB*), 159
 - `KeGGTools` (class in *cbmpy.CBNetDB*), 158
 - `kind` (*cbmpy.CBModel.Group* attribute), 49
- ## L
- `label` (*cbmpy.CBModel.Gene* attribute), 61
 - `label()` (*cbmpy.fluxmodules.decomposition.AbstractVertex* method), 11
 - `label()` (*cbmpy.fluxmodules.decomposition.Leaf* method), 13
 - `label()` (*cbmpy.fluxmodules.decomposition.Vertex* method), 12
 - `lb` (*cbmpy.CBModel.ReactionNew* attribute), 58
 - `lb` (*cbmpy.CBModel.UserDefinedConstraint* attribute), 46
 - `Leaf` (class in *cbmpy.fluxmodules.decomposition*), 12
 - `linear_constraints` (*cbmpy.CBSolverX.LPBase* attribute), 116
 - `listNonLeaves()` (*cbmpy.fluxmodules.decomposition.Decomposition* method), 13
 - `listNonLeavesRec()` (*cbmpy.fluxmodules.decomposition.Decomposition* method), 13
 - `loadCBGUI()` (in module *cbmpy.CBGUI*), 162
 - `loadObj()` (in module *cbmpy.CBTools*), 99
 - `Log()` (*cbmpy.CBNetDB.RESTClient* method), 159
 - `LPbase` (class in *cbmpy.CBSolve*), 65
 - `LPBase` (class in *cbmpy.CBSolverX*), 116
 - `LPcplex` (class in *cbmpy.CBSolve*), 66

- LPCPLEX (class in *cbmpy.CBSolverX*), 117
- LPECYGLPKI (class in *cbmpy.CBSolverX*), 117
- LPglpk (class in *cbmpy.CBSolve*), 66
- LPproperties (class in *cbmpy.CBSolve*), 65
- ## M
- makeGraphViz() (cbmpy.fluxmodules.decomposition.Decomposition method), 14
- makeSubcubic() (cbmpy.fluxmodules.decomposition.Decomposition method), 14
- Matrix (class in *cbmpy.fluxmodules.sparses*), 15
- Matroid (class in *cbmpy.fluxmodules.matroid*), 7
- max() (cbmpy.fluxmodules.matroid.SimilarityMatrix method), 10
- maxId (cbmpy.fluxmodules.decomposition.AbstractVertex attribute), 11
- maxobjname (cbmpy.CBXML.FBCconnect attribute), 145
- member_ids (cbmpy.CBModel.Group attribute), 49
- members (cbmpy.CBModel.Group attribute), 49
- merge2Models() (in module *cbmpy.CBTools*), 114
- mergeGroups() (in module *cbmpy.CBTools*), 113
- metaprefix (cbmpy.CBXML.FBCconnect attribute), 145
- MIDS (cbmpy.CBCommon.MIRIAMannotation attribute), 93, 96
- MIDS1c (cbmpy.CBCommon.MIRIAMannotation attribute), 93, 96
- MILPBase (class in *cbmpy.CBSolverX*), 116
- MinimizeSumOfAbsFluxes() (in module *cbmpy*), 17
- minobjname (cbmpy.CBXML.FBCconnect attribute), 145
- MIRIAM (cbmpy.CBCommon.MIRIAMannotation attribute), 93, 96
- miriam (cbmpy.CBModel.Fbase attribute), 18
- MIRIAMannotation (class in *cbmpy.CBCommon*), 93
- MIRIAMTools (class in *cbmpy.CBNetDB*), 160
- MiriamURN2IdentifiersURL() (cbmpy.CBNetDB.MIRIAMTools method), 160
- MLStripper (class in *cbmpy.CBXML*), 139
- model (cbmpy.CBSolve.LPproperties property), 65
- model (cbmpy.CBXML.FBCconnect attribute), 145
- Model (class in *cbmpy.CBModel*), 21
- MODEL_CREATORS (cbmpy.CBModel.Model attribute), 22
- module
- cbmpy, 16
 - cbmpy.CBCommon, 81
 - cbmpy.CBConfig, 81
 - cbmpy.CBDataStruct, 114
 - cbmpy.CBDefaultModels, 168
 - cbmpy.CBGLPK, 117
 - cbmpy.CBGUI, 162
 - cbmpy.CBModel, 17
 - cbmpy.CBModelTools, 165
 - cbmpy.CBMultiCore, 163
 - cbmpy.CBMultiEnv, 164
 - cbmpy.CBMultiModel, 163
 - cbmpy.CBNetDB, 155
 - cbmpy.CBPlot, 161
 - cbmpy.CBReadtxt, 125
 - cbmpy.CBSolve, 65
 - cbmpy.CBSolver, 115
 - cbmpy.CBSolverX, 115
 - cbmpy.CBTools, 98
 - cbmpy.CBWx, 162
 - cbmpy.CBXML, 138
 - cbmpy.fluxmodules, 7
 - cbmpy.fluxmodules.decomposition, 11
 - cbmpy.fluxmodules.matroid, 7
 - cbmpy.fluxmodules.sparses, 15
 - cbmpy.miriamids, 160

N

`N` (*cbmpy.CBModel.Model* attribute), 21
`name` (*cbmpy.CBModel.Fbase* attribute), 18
`name` (*cbmpy.CBSolve.LPproperties* property), 65
`name` (*cbmpy.CBSolverX.LPBase* attribute), 116
`NetDBbase` (class in *cbmpy.CBNetDB*), 155
`notes` (*cbmpy.CBModel.Fbase* attribute), 18
`NotFullyBranchedException`, 14
`numberifyComb2Int()` (*cbmpy.CBCommon.ComboGen* method), 85

O

`obj` (*cbmpy.CBSolverX.LPBase* attribute), 116
`obj_func` (*cbmpy.CBModel.Model* attribute), 21
`objective` (*cbmpy.CBSolverX.LPBase* attribute), 116
`Objective` (class in *cbmpy.CBModel*), 43
`objective_sense` (*cbmpy.CBSolverX.LPBase* attribute), 116
`objectives` (*cbmpy.CBModel.Model* attribute), 22
`occursIn` (*cbmpy.CBCommon.MIRIAMannotation* attribute), 95, 96
`operation` (*cbmpy.CBModel.FluxBound* attribute), 52
`operation` (*cbmpy.CBModel.Objective* attribute), 43
`operator` (*cbmpy.CBModel.FluxBoundBase* attribute), 52
`operators` (*cbmpy.CBCommon.StructMatrixLP* attribute), 90, 91

P

`parallel()` (*cbmpy.fluxmodules.matroid.Matroid* method), 8
`Parameter` (class in *cbmpy.CBModel*), 53
`parameter_cntr` (*cbmpy.CBXML.CBMtoSBML3* attribute), 147
`parameter_map` (*cbmpy.CBXML.CBMtoSBML3* attribute), 147

`parameters` (*cbmpy.CBModel.Model* attribute), 22
`parseGeneAssociation()` (in module *cbmpy.CBCommon*), 82
`parseReactions()` (in module *cbmpy.CBReadtxt*), 126
`parseXMLtoText()` (*cbmpy.CBNetDB.SemanticSBML* method), 160
`plotFluxVariability()` (in module *cbmpy.CBPlot*), 161
`poolmanMethod()` (*cbmpy.fluxmodules.decomposition.Decomposition* method), 14
`prefix` (*cbmpy.CBModel.Model* attribute), 21
`printSolution()` (in module *cbmpy.CBModelTools*), 167
`processBiGGannotationNote()` (in module *cbmpy.CBTools*), 107
`processBiGGchemFormula()` (in module *cbmpy.CBTools*), 106
`processExchangeReactions()` (in module *cbmpy.CBTools*), 108
`processSBMLAnnotationNotes()` (in module *cbmpy.CBTools*), 107
`processSpeciesChargeChemFormulaAnnot()` (in module *cbmpy.CBCommon*), 85
`protein` (*cbmpy.CBModel.GeneProteinAssociation* attribute), 63

Q

`QPbase` (class in *cbmpy.CBSolverX*), 116
`QUALIFIERS` (*cbmpy.CBCommon.MIRIAMannotation* attribute), 93, 96
`quickDefaultBuild()` (in module *cbmpy.CBModelTools*), 167
`quickLookup()` (*cbmpy.CBNetDB.SemanticSBML* method), 160

R

`rank()` (*cbmpy.fluxmodules.matroid.Matroid* method), 9
`reaction` (*cbmpy.CBModel.FluxBound* attribute), 52

[reaction](#) (*cbmpy.CBModel.FluxObjective* attribute), [45](#)
[reaction](#) (*cbmpy.CBModel.FluxObjectiveQuadratic* attribute), [45](#)
[Reaction](#) (class in *cbmpy.CBModel*), [54](#)
[reaction2](#) (*cbmpy.CBModel.FluxObjectiveQuadratic* attribute), [45](#)
[ReactionNew](#) (class in *cbmpy.CBModel*), [58](#)
[reactions](#) (*cbmpy.CBModel.Model* attribute), [22](#)
[reactivateReaction\(\)](#) (*cbmpy.CBModel.Reaction* method), [57](#)
[reactivateReaction\(\)](#) (*cbmpy.CBModel.ReactionNew* method), [59](#)
[readCSV\(\)](#) (in module *cbmpy.CBReadtxt*), [126](#)
[Reagent](#) (class in *cbmpy.CBModel*), [60](#)
[reagent_of](#) (*cbmpy.CBModel.Species* attribute), [59](#)
[reagents](#) (*cbmpy.CBModel.Reaction* attribute), [55](#)
[reduced_cost](#) (*cbmpy.CBModel.Reaction* attribute), [54](#)
[registerObjectInGlobalStore\(\)](#) (*cbmpy.CBModel.Model* method), [22](#)
[removeFixedSpeciesReactions\(\)](#) (in module *cbmpy.CBTools*), [111](#)
[renameObjectIds\(\)](#) (*cbmpy.CBModel.Model* method), [39](#)
[replaceAdjacent\(\)](#) (*cbmpy.fluxmodules.decomposition.AbstractVertexule* *cbmpy.CBXML*), [11](#)
[replaceAdjacent\(\)](#) (*cbmpy.fluxmodules.decomposition.Leaf* method), [13](#)
[replaceAdjacent\(\)](#) (*cbmpy.fluxmodules.decomposition.Vertex* method), [12](#)
[reset\(\)](#) (*cbmpy.fluxmodules.sparserationals.Matrix* method), [16](#)
[resetActivity\(\)](#) (*cbmpy.CBModel.Gene* method), [62](#)
[resetAllGenes\(\)](#) (*cbmpy.CBModel.Model* method), [37](#)
[resetAllInactiveGPRBounds\(\)](#) (*cbmpy.CBModel.Model* method), [32](#)
[RESTClient](#) (class in *cbmpy.CBNetDB*), [159](#)
[reversible](#) (*cbmpy.CBModel.Reaction* attribute), [55](#)
[revslice\(\)](#) (in module *cbmpy.fluxmodules.sparserationals*), [16](#)
[RHS](#) (*cbmpy.CBCommon.StructMatrixLP* attribute), [90](#), [91](#)
[ridx](#) (*cbmpy.CBCommon.StructMatrix* attribute), [86](#), [87](#)
[role](#) (*cbmpy.CBModel.Reagent* attribute), [61](#)
[roundOffWithSense\(\)](#) (in module *cbmpy.CBTools*), [113](#)
[row](#) (*cbmpy.CBCommon.StructMatrix* attribute), [86](#), [87](#)
[rows\(\)](#) (*cbmpy.fluxmodules.sparserationals.Matrix* method), [15](#)
[runMultiCoreFVA\(\)](#) (in module *cbmpy.CBMultiCore*), [163](#)
[runMultiStateFBA\(\)](#) (in module *cbmpy.CBMultiEnv*), [165](#)

S

[sbml](#) (*cbmpy.CBXML.FBCconnect* attribute), [145](#)
[sbml_convertCOBRASBMLtoFBC\(\)](#) (in module *cbmpy.CBXML*), [152](#)
[sbml_convertSBML3FBCToCOBRA\(\)](#) (in module *cbmpy.CBXML*), [153](#)
[sbml_createAssociationFromAST\(\)](#) (in module *cbmpy.CBXML*), [150](#)
[sbml_createAssociationFromTreeV2\(\)](#) (in module *cbmpy.CBXML*), [151](#)
[sbml_createModelL2\(\)](#) (in module *cbmpy.CBXML*), [142](#)
[sbml_exportSBML2FBAModel\(\)](#) (in module *cbmpy.CBXML*), [144](#)
[sbml_fileFindVersion\(\)](#) (in module *cbmpy.CBXML*), [153](#)
[sbml_fileValidate\(\)](#) (in module *cbmpy.CBXML*), [153](#)

<code>sbml_getCVterms()</code> (in module <code>cbmpy.CBXML</code>), 155	<code>sbml_setSpeciesL2()</code> (in module <code>cbmpy.CBXML</code>), 144
<code>sbml_getGeneRefs()</code> (in module <code>cbmpy.CBXML</code>), 150	<code>sbml_setSpeciesL3()</code> (in module <code>cbmpy.CBXML</code>), 149
<code>sbml_getGPRasDictFBCv1()</code> (in module <code>cbmpy.CBXML</code>), 150	<code>sbml_setUnits()</code> (in module <code>cbmpy.CBXML</code>), 143
<code>sbml_getGPRasDictFBCv2()</code> (in module <code>cbmpy.CBXML</code>), 150	<code>sbml_setValidationOptions()</code> (in module <code>cbmpy.CBXML</code>), 152
<code>sbml_getNotes()</code> (in module <code>cbmpy.CBXML</code>), 143	<code>sbml_validateDocument()</code> (in module <code>cbmpy.CBXML</code>), 153
<code>SBML_NS</code> (in module <code>cbmpy.CBXML</code>), 139	<code>sbml_writeAnnotationsAsCOBRANote()</code> (in module <code>cbmpy.CBXML</code>), 143
<code>sbml_readCOBRANote()</code> (in module <code>cbmpy.CBXML</code>), 154	<code>sbml_writeCOBRASBML()</code> (in module <code>cbmpy.CBXML</code>), 150
<code>sbml_readCOBRASBML()</code> (in module <code>cbmpy.CBXML</code>), 152	<code>sbml_writeKeyValueDataAnnotation()</code> (in module <code>cbmpy.CBXML</code>), 148
<code>sbml_readFBCv3KeyValuePairs()</code> (in module <code>cbmpy.CBXML</code>), 148	<code>sbml_writeSBML2FBA()</code> (in module <code>cbmpy.CBXML</code>), 145
<code>sbml_readKeyValueDataAnnotation()</code> (in module <code>cbmpy.CBXML</code>), 148	<code>sbml_writeSBML3FBC()</code> (in module <code>cbmpy.CBXML</code>), 151
<code>sbml_readSBML2FBA()</code> (in module <code>cbmpy.CBXML</code>), 140	<code>sbmllevel</code> (<code>cbmpy.CBXML.FBCconnect</code> attribute), 145
<code>sbml_readSBML3FBC()</code> (in module <code>cbmpy.CBXML</code>), 154	<code>sbmlns</code> (<code>cbmpy.CBXML.FBCconnect</code> attribute), 145
<code>sbml_setAnnotationsL3Fbc()</code> (in module <code>cbmpy.CBXML</code>), 142	<code>sbmlversion</code> (<code>cbmpy.CBXML.FBCconnect</code> attribute), 145
<code>sbml_setCompartmentsL2()</code> (in module <code>cbmpy.CBXML</code>), 142	<code>SCALED_REDUCED_COSTS</code> (<code>cbmpy.CBModel.Model</code> attribute), 22
<code>sbml_setCompartmentsL3()</code> (in module <code>cbmpy.CBXML</code>), 142	<code>scaleProteinCostAnnotation()</code> (in module <code>cbmpy.CBMultiEnv</code>), 165
<code>sbml_setCVterms()</code> (in module <code>cbmpy.CBXML</code>), 155	<code>scanForReactionDuplicates()</code> (in module <code>cbmpy.CBTools</code>), 109
<code>sbml_setDescription()</code> (in module <code>cbmpy.CBXML</code>), 142	<code>scanForUnbalancedReactions()</code> (in module <code>cbmpy.CBTools</code>), 111
<code>sbml_setFBCv3KeyValuePairs()</code> (in module <code>cbmpy.CBXML</code>), 149	<code>SemanticSBML</code> (class in <code>cbmpy.CBNetDB</code>), 160
<code>sbml_setGroupsL3()</code> (in module <code>cbmpy.CBXML</code>), 150	<code>sensitivity</code> (<code>cbmpy.CBModel.Model</code> attribute), 22
<code>sbml_setNotes3()</code> (in module <code>cbmpy.CBXML</code>), 143	<code>serialize()</code> (<code>cbmpy.CBModel.Fbase</code> method), 20
<code>sbml_setParametersL3Fbc()</code> (in module <code>cbmpy.CBXML</code>), 142	<code>serialize()</code> (<code>cbmpy.CBModel.Group</code> method), 51
<code>sbml_setReactionsL2()</code> (in module <code>cbmpy.CBXML</code>), 144	<code>serialize()</code> (<code>cbmpy.CBModel.Model</code> method), 23
<code>sbml_setReactionsL3Fbc()</code> (in module <code>cbmpy.CBXML</code>), 149	<code>serializeToDisk()</code> (<code>cbmpy.CBModel.Fbase</code>

`method`), 20
`serializeToDisk()` (*cbmpy.CBModel.Group* `method`), 51
`serializeToDisk()` (*cbmpy.CBModel.Model* `method`), 23
`setActive()` (*cbmpy.CBModel.Gene* `method`), 62
`setActiveObjective()` (*cbmpy.CBModel.Model* `method`), 36
`setAllFluxBounds()` (*cbmpy.CBModel.Model* `method`), 38
`setAllGenesActive()` (*cbmpy.CBModel.GeneProteinAssociation* `method`), 64
`setAllGenesInactive()` (*cbmpy.CBModel.GeneProteinAssociation* `method`), 64
`setAllInactiveGPRBounds()` (*cbmpy.CBModel.Model* `method`), 32
`setAllProteinActivities()` (*cbmpy.CBModel.Model* `method`), 32
`setAnnotation()` (*cbmpy.CBModel.Fbase* `method`), 19
`setBoundary()` (*cbmpy.CBModel.Species* `method`), 60
`setBoundValueByName()` (*cbmpy.CBModel.Model* `method`), 36
`setByIdx()` (*cbmpy.CBCommon.StructMatrix* `method`), 89
`setByName()` (*cbmpy.CBCommon.StructMatrix* `method`), 89
`setCBSB0term()` (in module *cbmpy.CBXML*), 153
`setCharge()` (*cbmpy.CBModel.Species* `method`), 60
`setChemFormula()` (*cbmpy.CBModel.Species* `method`), 60
`setCoefficient()` (*cbmpy.CBModel.ConstraintComponent* `method`), 47
`setCoefficient()` (*cbmpy.CBModel.FluxObjective* `method`), 46
`setCoefficient()` (*cbmpy.CBModel.FluxObjectiveQuadratic* `method`), 45
`setCoefficient()` (*cbmpy.CBModel.Reagent* `method`), 61
`setCol()` (*cbmpy.CBCommon.StructMatrix* `method`), 88
`setCompartmentId()` (*cbmpy.CBModel.Fbase* `method`), 21
`setCreatedDate()` (*cbmpy.CBModel.Model* `method`), 24
`setDescription()` (*cbmpy.CBModel.Model* `method`), 23
`setDimensions()` (*cbmpy.CBModel.Compartment* `method`), 48
`setFluxBoundsFromDict()` (*cbmpy.CBModel.Model* `method`), 38
`setGeneActive()` (*cbmpy.CBModel.GeneProteinAssociation* `method`), 64
`setGeneActive()` (*cbmpy.CBModel.Model* `method`), 37
`setGeneInactive()` (*cbmpy.CBModel.GeneProteinAssociation* `method`), 64
`setGeneInactive()` (*cbmpy.CBModel.Model* `method`), 37
`setId()` (*cbmpy.CBModel.Compartment* `method`), 48
`setId()` (*cbmpy.CBModel.Fbase* `method`), 19
`setId()` (*cbmpy.CBModel.Gene* `method`), 62
`setId()` (*cbmpy.CBModel.Reaction* `method`), 56
`setId()` (*cbmpy.CBModel.ReactionNew* `method`), 58
`setId()` (*cbmpy.CBModel.Species* `method`), 59
`setInactive()` (*cbmpy.CBModel.Gene* `method`), 62
`setKind()` (*cbmpy.CBModel.Group* `method`),

[50](#)
[setLabel\(\)](#) (*cbmpy.CBModel.Gene method*), [62](#)
[setLowerBound\(\)](#) (*cbmpy.CBModel.Reaction method*), [57](#)
[setLowerBound\(\)](#) (*cbmpy.CBModel.ReactionNew method*), [58](#)
[setLowerBound\(\)](#) (*cbmpy.CBModel.UserDefinedConstraint method*), [46](#)
[setMetaId\(\)](#) (*cbmpy.CBModel.Fbase method*), [19](#)
[setModifiedDate\(\)](#) (*cbmpy.CBModel.Model method*), [24](#)
[setName\(\)](#) (*cbmpy.CBModel.Fbase method*), [18](#)
[setNotes\(\)](#) (*cbmpy.CBModel.Fbase method*), [19](#)
[setObjectiveFlux\(\)](#) (*cbmpy.CBModel.Model method*), [39](#)
[setObjectiveSense\(\)](#) (*cbmpy.CBSolverX.LPBase method*), [116](#)
[setObjRefOnComponents\(\)](#) (*cbmpy.CBModel.ReactionNew method*), [59](#)
[setObjRefOnComponents\(\)](#) (*cbmpy.CBModel.UserDefinedConstraint method*), [47](#)
[setOperation\(\)](#) (*cbmpy.CBModel.Objective method*), [43](#)
[setOperatorbyIdx\(\)](#) (*cbmpy.CBCommon.StructMatrixLP method*), [92](#)
[setOperatorbyName\(\)](#) (*cbmpy.CBCommon.StructMatrixLP method*), [92](#)
[setPid\(\)](#) (*cbmpy.CBModel.Fbase method*), [19](#)
[setPrefix\(\)](#) (*cbmpy.CBModel.Model method*), [39](#)
[setProtein\(\)](#) (*cbmpy.CBModel.GeneProteinAssociation method*), [64](#)
[setReactionBound\(\)](#) (*cbmpy.CBModel.Model method*), [37](#)
[setReactionBounds\(\)](#) (*cbmpy.CBModel.Model method*), [38](#)
[setReactionId\(\)](#) (*cbmpy.CBModel.FluxBound method*), [52](#)
[setReactionId\(\)](#) (*cbmpy.CBModel.FluxObjective method*), [46](#)
[setReactionId\(\)](#) (*cbmpy.CBModel.FluxObjectiveQuadratic method*), [45](#)
[setReactionId2\(\)](#) (*cbmpy.CBModel.FluxObjectiveQuadratic method*), [45](#)
[setReactionIds\(\)](#) (*cbmpy.CBModel.FluxObjectiveQuadratic method*), [45](#)
[setReactionLowerBound\(\)](#) (*cbmpy.CBModel.Model method*), [38](#)
[setReactionUpperBound\(\)](#) (*cbmpy.CBModel.Model method*), [38](#)
[setReagentOf\(\)](#) (*cbmpy.CBModel.Species method*), [60](#)
[setReducedCosts\(\)](#) (*in module cbmpy.CBGLPK*), [119](#)
[setRHSbyIdx\(\)](#) (*cbmpy.CBCommon.StructMatrixLP method*), [91](#)
[setRHSbyName\(\)](#) (*cbmpy.CBCommon.StructMatrixLP method*), [91](#)
[setRow\(\)](#) (*cbmpy.CBCommon.StructMatrix method*), [88](#)
[setSB0term\(\)](#) (*cbmpy.CBModel.Fbase method*), [21](#)
[setSharedAnnotation\(\)](#) (*cbmpy.CBModel.Group method*), [50](#)
[setSharedNotes\(\)](#) (*cbmpy.CBModel.Group method*), [50](#)

[setSharedSBOTerm\(\)](#) (*cbmpy.CBModel.Group* *method*), [50](#)
[setSize\(\)](#) (*cbmpy.CBModel.Compartment* *method*), [48](#)
[setSpecies\(\)](#) (*cbmpy.CBModel.Reagent* *method*), [61](#)
[setSpeciesPropertiesFromAnnotations\(\)](#) (*in module cbmpy.CBTools*), [101](#)
[setStoichCoefficient\(\)](#) (*cbmpy.CBModel.Reaction* *method*), [56](#)
[setSuffix\(\)](#) (*cbmpy.CBModel.Model* *method*), [39](#)
[setTree\(\)](#) (*cbmpy.CBModel.GeneProteinAssociation* *method*), [64](#)
[setType\(\)](#) (*cbmpy.CBModel.ConstraintComponent* *method*), [47](#)
[setType\(\)](#) (*cbmpy.CBModel.FluxObjective* *method*), [46](#)
[setType\(\)](#) (*cbmpy.CBModel.FluxObjectiveQuadratic* *method*), [45](#)
[setUpperBound\(\)](#) (*cbmpy.CBModel.Reaction* *method*), [57](#)
[setUpperBound\(\)](#) (*cbmpy.CBModel.ReactionNew* *method*), [58](#)
[setUpperBound\(\)](#) (*cbmpy.CBModel.UserDefinedConstraint* *method*), [46](#)
[setUpRoot\(\)](#) (*cbmpy.fluxmodules.decomposition.Vertex* *method*), [12](#)
[setValue\(\)](#) (*cbmpy.CBModel.FluxBound* *method*), [52](#)
[setValue\(\)](#) (*cbmpy.CBModel.FluxBoundBase* *method*), [53](#)
[setValue\(\)](#) (*cbmpy.CBModel.Objective* *method*), [44](#)
[setValue\(\)](#) (*cbmpy.CBModel.Parameter* *method*), [54](#)
[setValue\(\)](#) (*cbmpy.CBModel.Reaction* *method*), [56](#)
[setValue\(\)](#) (*cbmpy.CBModel.Species* *method*), [59](#)
[setVariable\(\)](#) (*cbmpy.CBModel.ConstraintComponent* *method*), [47](#)
[shadow_price](#) (*cbmpy.CBModel.Species* *attribute*), [59](#)
[shape](#) (*cbmpy.CBCommon.StructMatrix* *attribute*), [86, 87](#)
[SimilarityMatrix](#) (*class in cbmpy.fluxmodules.matroid*), [10](#)
[similarityMatrix\(\)](#) (*cbmpy.fluxmodules.matroid.Matroid* *method*), [10](#)
[site_root](#) (*cbmpy.CBNetDB.RESTClient* *attribute*), [159](#)
[size](#) (*cbmpy.CBModel.Compartment* *attribute*), [48](#)
[solution](#) (*cbmpy.CBModel.Objective* *attribute*), [43](#)
[solution](#) (*cbmpy.CBModel.UserDefinedConstraint* *attribute*), [46](#)
[SOLUTION_STATUS](#) (*cbmpy.CBModel.Model* *attribute*), [22](#)
[SOLUTION_STATUS_INT](#) (*cbmpy.CBModel.Model* *attribute*), [22](#)
[sortReactionsById\(\)](#) (*cbmpy.CBModel.Model* *method*), [39](#)
[sortSpeciesById\(\)](#) (*cbmpy.CBModel.Model* *method*), [40](#)
[sourcefile](#) (*cbmpy.CBModel.Model* *attribute*), [21](#)
[species](#) (*cbmpy.CBModel.Model* *attribute*), [22](#)
[Species](#) (*class in cbmpy.CBModel*), [59](#)
[species_ref](#) (*cbmpy.CBModel.Reagent* *attribute*), [61](#)
[splitEqualityFluxBounds\(\)](#) (*cbmpy.CBModel.Model* *method*), [41](#)
[splitOff\(\)](#) (*cbmpy.fluxmodules.decomposition.Vertex* *method*), [12](#)
[splitOneModules\(\)](#) (*cbmpy.fluxmodules.decomposition.Decomposition* *method*), [13](#)
[splitReversibleReactions\(\)](#) (*in module cbmpy.CBTools*), [102](#)
[splitSimple\(\)](#)

`(cbmpy.fluxmodules.decomposition.DecompositionData()` `(cbmpy.CBNetDB.DBTools`
`method), 13` `method), 157`
`splitSingleReversibleReaction()` `(in module cbmpy.CBTools), 102` `updateNetwork()` `(cbmpy.CBModel.Model`
`method), 37`
`splitZeroModules()` `URLDecode()` `(cbmpy.CBNetDB.NetDBbase`
`(cbmpy.fluxmodules.decomposition.Decomposition` `method), 156`
`method), 13` `URLEncode()` `(cbmpy.CBNetDB.NetDBbase`
`method), 155`
`sqlite (cbmpy.CBNetDB.DBTools attribute),` `use_compiled(cbmpy.CBModel.GeneProteinAssociation`
`156` `attribute), 63`
`sqlite_version (cbmpy.CBNetDB.DBTools` `USER_AGENT (cbmpy.CBNetDB.RESTClient at-`
`attribute), 156` `tribute), 159`
`storeObj()` `(in module cbmpy.CBTools), 99` `user_constraints (cbmpy.CBModel.Model`
`stringReplace()` `(in module cbmpy.CBTools), 105` `attribute), 22`
`StructMatrix (class in cbmpy.CBCommon),` `user_defined_constraints`
`86` `(cbmpy.CBModel.Model attribute),`
`StructMatrixLP (class in` `22`
`cbmpy.CBCommon), 90` `UserDefinedConstraint (class in`
`suffix (cbmpy.CBModel.Model attribute), 21` `cbmpy.CBModel), 46`

T

`test()` `(in module cbmpy), 17`
`testGeneProteinAssociations()`
`(cbmpy.CBModel.Model method),`
`42`
`text_encoding (cbmpy.CBNetDB.NetDBbase`
`attribute), 155`
`toNumpy()` `(cbmpy.fluxmodules.sparserationals.Matrix`
`method), 16`
`transpose()` `(cbmpy.fluxmodules.sparserationals.Matrix`
`method), 15`
`tree (cbmpy.CBModel.GeneProteinAssociation`
`attribute), 62`

U

`ub (cbmpy.CBModel.ReactionNew attribute), 58`
`ub (cbmpy.CBModel.UserDefinedConstraint at-`
`tribute), 46`
`uniqueCombinations()`
`(cbmpy.CBCommon.ComboGen`
`method), 84`
`unRegisterObjectInGlobalStore()`
`(cbmpy.CBModel.Model method), 22`
`unsetBoundary()` `(cbmpy.CBModel.Species`
`method), 60`

V

`value (cbmpy.CBModel.FluxBound attribute),`
`52`
`value (cbmpy.CBModel.FluxBoundBase prop-`
`erty), 53`
`value (cbmpy.CBModel.Objective attribute),`
`43`
`value (cbmpy.CBModel.Parameter property),`
`54`
`value (cbmpy.CBModel.Reaction attribute), 54`
`value (cbmpy.CBModel.Species attribute), 59`
`variable (cbmpy.CBModel.ConstraintComponent`
`attribute), 47`
`verifyEdgeWidth()`
`(cbmpy.fluxmodules.decomposition.Decomposition`
`method), 14`
`verifyFullyBranched()`
`(cbmpy.fluxmodules.decomposition.Decomposition`
`method), 14`
`Vertex (class in`
`cbmpy.fluxmodules.decomposition),`
`12`
`viewDataInWebbrowser()`
`(cbmpy.CBNetDB.SemanticSBML`
`method), 160`

`viewURL()` (*cbmpy.CBCommon.MIRIAMannotation*
method), 98

`volume` (*cbmpy.CBModel.Compartment* at-
tribute), 48

W

`with_metaclass()` (in module
cbmpy.CBXML), 154

`write()` (*cbmpy.fluxmodules.decomposition.Decomposition*
method), 14

`writeToFile()` (*cbmpy.CBXML.FBCconnect*
method), 146

X

`xml_addSBML2FBAFluxBound()` (in module
cbmpy.CBXML), 140

`xml_addSBML2FBAObjective()` (in module
cbmpy.CBXML), 141

`xml_createListOfFluxObjectives()` (in
module *cbmpy.CBXML*), 141

`xml_createSBML2FBADoc()` (in module
cbmpy.CBXML), 140

`xml_createSBML2FBAObjective()` (in mod-
ule *cbmpy.CBXML*), 141

`xml_getSBML2FBAannotation()` (in module
cbmpy.CBXML), 141

`xml_stripTags()` (in module
cbmpy.CBXML), 139

`xml_viewSBML2FBAXML()` (in module
cbmpy.CBXML), 140